

# 第一部分

***DNP V3.00***

**数据链路层**

# 目 录

---

|                      |    |
|----------------------|----|
| 第一章 概论.....          | 4  |
| 第二章 与 IEC 的一致性.....  | 5  |
| 2.1 通道的故障切换.....     | 5  |
| 2.2 帧格式与规程.....      | 5  |
| 2.3 长度、控制与地址段.....   | 5  |
| 第三章 DNP 数据链路的说明..... | 7  |
| 3.1 数据链路层的目的.....    | 7  |
| 3.2 数据链路报头帧的格式.....  | 8  |
| 3.3 原方通信事务.....      | 12 |
| 3.4 副方通信事务.....      | 14 |
| 3.5 对用户过程的复位.....    | 14 |
| 3.6 测试.....          | 14 |
| 3.7 用户数据.....        | 14 |
| 3.8 无需确认的用户数据.....   | 16 |
| 3.9 传输过程.....        | 16 |
| 3.10 对用户过程之复位.....   | 17 |
| 3.11 发送/确认用户数据.....  | 17 |
| 3.12 发送/不期待回信.....   | 17 |
| 3.13 发送/NACK.....    | 20 |
| 3.14 查询/响应.....      | 21 |
| 第四章 数据链路的服务与职责.....  | 22 |
| 4.1 数据链路的功能.....     | 22 |
| 4.2 接口说明.....        | 23 |
| 第五章 物理层的接口.....      | 26 |
| 5.1 物理层的说明.....      | 26 |

|                 |    |
|-----------------|----|
| 第六章 物理层特性.....  | 29 |
| 6.1 线路配置.....   | 29 |
| 6.2 传输的模式.....  | 29 |
| 6.3 本地环路.....   | 29 |
| 第七章 物理层的规程..... | 31 |
| 7.1 基本概念.....   | 31 |
| 7.2 半双工规程.....  | 31 |
| 7.3 全双工规程.....  | 32 |

## 第一章：概 论

---

本文件规定了分布网络规约(DNP)V3.00 版的数据链路层。链路规约数据单元(LPDU)以及数据链路层服务和传输规程。主站、分主站，外站和智能电子设备 (IEDs) 都能使用本数据链路在原方(始发)站和副方(接收)站之间传递报文。在本规约中主站、分主站、外站和 IEDs 都是既可作始发(原方)站也可以作接收(副方)站。

IEC TC57 所推出的 IEC 870-5-1 和 IEC870-5-2 关于远动系统内数据传输的标准是开发 DNP V3.00 数据链路层的基础。

DNP V3.00 的数据链路层支持被查询的和寂静的远动系统，并且在设计上可适用于面向连接的和面向非连接的操作，异步的或同步的位串型物理层，诸如 RS-232C，RS-485 以及光纤收发器，采用全部平衡的传输规程去支持来自外站，IEDs 或非作为主站的分主站之自发型传输。

本规约所支持的 ISO OSI 模型仅规定了物理层，数据链路层和应用层。这种特定的模型被称之为性能增强了的体系结构 (EPA)。然而，为了支持高级的 RTU 功能和大于最大帧式(如 IEC870-5-1 所规定的最大帧长)的报文，DNP V3.00 的数据链路打算用一个伪传输层去完成最短报文的组装与分解。

该伪传输层说明于文件 DNP V3.00 Transport Function(P009-OPD.TF) 内，然而，要强调一下，这些传输功能并非数据链路的一部份而是支持高级 RTU 功能之所需。

## 第二章: 与 IEC 的一致性

本章说明 DNP 规约与 IEC TC-57 (870-5) 运动数据链路层规约在规范上的不同点。

### 2.1 通道的故障切换

DNP 链路层仅对一个物理层(或信道)通信。在 OSI 模型中,会话层才负责维持信道的连接。在 DNP 中,在数据链路以上的层,根据数据链路上的通信失败,负责提供信道的故障切换。该层可以是网络/传输层或应用层。所以 IEC 关于信道故障切换的 870-5-1 的第 13 项要求在应用层中得到满足。

### 2.2 帧格式与规程

数据链路层采用一种可变帧长格式,如 IEC870-5-1 (传输帧格式)文件中所述。FT3 帧格式很适合于中等传输速率和低残余误码率的站间数据传输。采用其作为基本帧格式,遵守其传输规则第 R1, R2, R3 和 R4 条。在原则上遵守规则的 R5 及 R6 条,尽管没有采用它所推荐的精确时间值,但在每个实施中可以组态。遵循在 IEC870-5-2 中所提出的帧定义,所不同者这里用了两字节长的地址段指定目的站地址,以及将(链路用户数据段)用作两个字节的源方站地址。

采用 IEC-870-5-2 所规定的全平衡传输规程去处理在半双工或全双工系统内来自非主站的非所请求的信息传输。全平衡意味每个站都可以同时作为原方站(发方)和副方站(收方)。这种配置要求一个全双工信道去正确地运行。在半双工环境中,将使用同一个规程,所不同者一个站不能同时既是原方站又是副方站。这就是说,在主站与外站间数据链路层的一个完整的数据传输回合必须完成后,才能开始来自任何一方的任何其它的通信事务处理。在半双工和全双工两者的组态中,每个设备的责任都要实现一个可兼容的防免碰撞的模式。

### 2.3 长度、控制和地址段

DNP 数据链使用的(长度)段和 IEC870-5-1 的 6.2.4 款所定义的相同(参见第三章,关于此段的进一步资料)。

(控制)段采用于平衡传输的 IEC CONTROL 段,如 IEC870-5-2 的 6.1.2 款所规定者。在 IEC870-5-3 的 6.2.1 款,表 III 中的全部功能码也是被支持的。

(地址)段是一个 16 位(二字节)的段。DNP 数据链路帧之报头有两个 IEC 地址段。第一段是 A(地址)段,它被用以表示目的站地,第二段在(链路用户数据)段,它被用

以表示源主站地址。（关于此两段的更多资料参见第三章）。

DocuCom PDF Trial  
www.pdfwizard.com

## 第三章： DNP 数据链路的说明

数据链路层是 OSI 模型的第二层。数据链路层接受，执行并控制高层所要求的传输服务功能。

### 3. 1 数据链路层的目的

DNP 数据链路层的目的有两重。第一，数据链路层必须提供信息的传送或提供进入物理链路的（链路服务数据单元（LSDU），如 ISO-OSI 的标准所述。这就是说由较高层供给用户数据 LSDU 必须转换成一个帧（或如第二章所述的 LPDU），并送至物理层以便发送。反过来，数据链路层所收到的 LPDUS 也必须被组装入一个 LSDU 并传递给较高的一层。该层还提供帧同步和链路控制。

第二，在 DNPV3.00 内，数据链路还为其它事件提供信号，诸如链路的状态。

OSI 参考模型既实施了一个无连接系统，也实施了一个面向连接的系统。然而 EPA 模型既不蕴含非连接系统，也不蕴含面向连接的系统。DNP 第 3 版在 IEC 数据链路中既管理非连接的系统也管理面向连接的系统（即在数据能被传输到目的设备之前，先要求拨号或挂号的物理网络），但不需要提供连接服务。实际的物理网络对于使用数据链路的应用是透明的，因为数据链路负责对任何物理网络的连接和断开，没有与较高层次的交互作用（即应用层）。这就是说数据链路（给定的目的站址）将连接到正确的物理电路无需更高层的控制。如此则物理介质对于链路层服务的用户就全然透明了。

#### FT3 帧格式

本节说明 LPDU 格式。一个 FT3 型的帧被定义为一个固定长度的报头，随之以可选用的数据块，每个数据块附有一个 16 位的 CRC。IEC 规定那个报头段含有

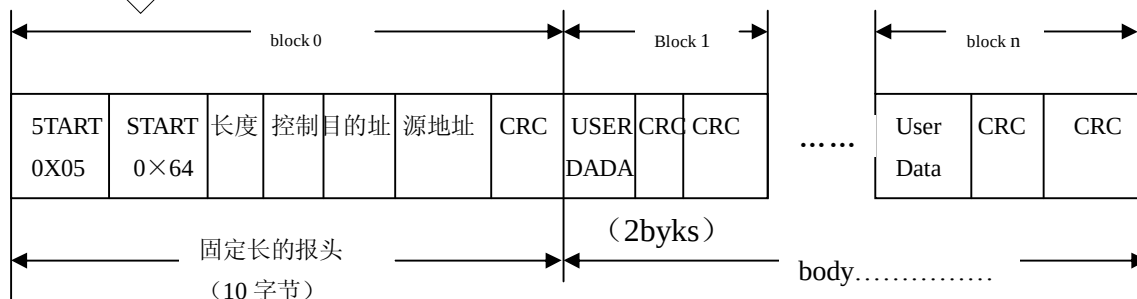


图 3-1 FT3 帧格式

2 个 start 字节, 1 个字节的长度, 1 个字节的控制字, 1 个目的址和一个可选用的定帧长的用户数据段。在这为首的数据块中定长的用户数据段被定义为一个源地址。

START (起始) — 报头的 2 个起始字节

LENGTH (长度) — 长度为报头中的 USER DATA 长 (5 字节) (即控制字与目的址和源地址之长); 加上主体 (Body) 中的 UER DATA 长度。故 L 之最短为 5。L 之最长为 255。

CONTROL (控制) — 帧的控制字节

SOURCE (源地址) — 2 个字节, 第一个为 LSB, 第二个为 MSB。

DESTINATION (目的址) — 2 个字节, 第一个字节为 LSB, 第二个为 MSB。

CRC (循环冗余码) — 2 个字节

USER DATA (用户数据) — 在报头之后, 每一块为 16 个字节; 最后那个块的字节数则为余数, 在 1 至 16 之间, 根据需要而定

### 3.2 数据段链路报头帧的格式

本节说明一个数据链路帧的#0 字块 (或称报头)。(按: 本节原文内容有重复之处, 又对 CRC 的常识介绍过于详细, 此处应予节删。)

图 3-2 定义了该字段的控制字节。站 A 被定义为主站。站 B 被定义为非主站 (即从站或外站)。而原方站 (primary) 为报文的原发站; 副方站 (secondary) 为报文的目的地站。

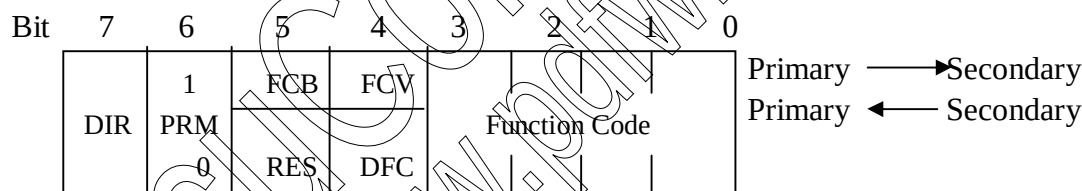


图 3-2 控制字节位定义

DIR: 方向 (primary) 位

1= 表示此帧自 A 发向 B (即主站发出方向)

0= 表示此帧自 B 发向 A (即发向主站方向)

PRM: 原发标志位 (direction) 位

1= 表示此帧来自原发站

0= 表示此帧来自响应站

FCB- 帧记数位

FCV- 帧记数有效位

1=有效

0=无效

DFC — 数据流控位

RES — 保留 =0

FUNCTION CODE – 功能码表明了帧的类型

DIR: 方向位

DIR =1 表示此帧自 A 发向 B (即主站发出方向)

DIR =0 表示此帧自 B 发向 A (即发向主站方向)

注意: 原发站不一定是主站; 外站也可以是原发站。

PRM: 原发标志位

PRM =1 表示此帧来自原发站

PRM =0 表示此帧来自响应站

**FCB: 帧的计数位 (frame count bit)** 系用以防止对同一个副方站丢失或重复信息帧。每次成功地完成一次发送并确认 (**SEND-CONFIRM**) 服务 (即由同一个原方站起动, 向同一个副方站的发送服务) 后该位就翻转一次。

在对一个副方站的通信起始之前或在通信失败之后, 原方站 (不论它是主站或外站) 对它所要通信的每个副方站都必须使其数据链路复位。在对所有副方站一旦建立了数据链路或在需要时, 应可以这样复位一次。

在数据链路建立之后或通信失败后, 每个副方站必须拒绝接受原方 FCB 置位的任何 **SEND-CONFIRM** 报文, 直至已收到一条 **RESET** 令并发出一条 **CONFIRM** 报文为止。

**FCV: 帧计数有效位 (frame count valid)**, 它可使 FCB 位生效。

0=表示 FCB 位无效。

1=向副方站指示本帧的 FCB 位状态有效, 必须和 FCV 置位的最近一帧之 FCB 位状态进行校核。

**DFC: 数据流控制位 (data flow control bit)** 系用以防止在副方站内的缓存器溢出。如果再将用户数据送入该副方站将导致其缓存溢出时, 副方站的回文中就将这一位 (DFC) 置位。原方站必须用 **REQUEST-RESPOND** 去查询链路状态, 直到 DFC 复位为止。以后, 原方站又可以继续向它发送数据。图 3-16 例举 DFC 位之用法。

功能码: 功能码标明帧之类型。

发自原方站的控制字有帧计数位 (FCB);

发自副方站的控制字无帧计数位。

原方发送的控制字中的 FCB 位仅在测试链路或传送用户数据时有效。

原方发送的控制字其功能码为（PRM=1）

| 功能码   | 帧类型             | 服务功能     | FCV 位 |
|-------|-----------------|----------|-------|
| 0     | SEND/期待 Confirm | 使远方链路复位  | 0     |
| 1     | SEND/期待 Confirm | 使用户过程复位  | 0     |
| 2     | SEND/期待 Confirm | 对链路的测试功能 | 1     |
| 3     | SEND/期待 Confirm | 用户数据     | 1     |
| 4     | SEND/不期待回信      | 非确认的用户数据 | 0     |
| 9     | 查询/期待响应         | 查询链路状态   | 0     |
| 5-8   | 未用              |          |       |
| 10-15 | 未用              |          |       |

副方发送的控制字功能码（PRM=0）

| 功能码   | 帧类型 | 服务功能             |
|-------|-----|------------------|
| 0     | 确认  | ACK=肯定的确认        |
| 1     | 确认  | NACK=报文未收到；链路忙   |
| 11    | 响应  | 链路的状态（DFC=0 或 1） |
| 2-10  | 未用  | 链路服务不工作了         |
| 12-13 | 未用  | 未用链路服务不工作了       |
| 14    |     | 未用链路服务或未实现链路服务   |
| 15    |     |                  |

图 3-3 原方与副方功能码表

**DESTINATION**（目的址）两个字节，低字节居前。

0XFFFF 为全站址。目的址为此值者，所有的站都要接收它的帧。

|          |               |
|----------|---------------|
| 低字节（LSB） | 高字节（MSB）（MSB） |
|----------|---------------|

图 3-4 目的地址的格式

**SOURCE**（源地址）两个字节，低字节居前。

注意，这个字节不包括在 **USER DATA** 之内，但必须由数据链路服务的原语传送给更高层，作为一个返回（return）

|          |          |
|----------|----------|
| 低字节（LSB） | 高字节（MSB） |
|----------|----------|

图 3-5 源地址的格式

### 3.3 USER DATA (用户数据)

跟在报头之后的数据块，每个者 16 个字节长，最后一个块包含剩下的字节，可以是 16 个字节。每个数据块都有一个 CRC 挂在后面。当收到一个 SEND (始发方) 的用户数据帧时，数据链路层就将全部用户数据和报头中的源地址都传递给更高层。数据链路服务的原语提供一个地址放置源地址。

#### CRC 循环冗余码

2 个字节长。在一个帧内，挂在每个数据块之后。报头内的 CRC，将 START、LENGTH、CONTRLL、DESTINATION 和 SOURCE 五个段全计算进去了。

CRC 的生成函数为

$$P=G(X)=X^{16}+X^{13}+X^{12}+X^{11}+X^{10}+X^8+X^6+X^5+X^2+1$$

$CRC=M \cdot 2^{16}/G(X)$  之余数取反

以下为发送与传输的次序：

发送一个数据块：

- (1) 取用户数据块 M，其长度为 K 比特。
- (2) 将 M 乘以  $2^{16}$  得  $2^{16}M$
- (3) 将此数除以 (模 2 除) P (17 比特)，得余数 R (16 比特)
- (4) 对 R 取反，得  $R'$
- (5) 将  $R'$  挂在  $2^{16}M$  后，并作为数据块 (T) 发送出去。

接收一个数据块：

- (1) 接收一个数据块 (T)，它具有 (K+16) 比特。
- (2) 将 T 中的  $R'$  取反后得 T，仍为 (K+16) 比特。
- (3) 将 T 除以 P，得出它的余数。
- (4) 若余数非为零，则数据块出错，否则数据块无误。

采用了 FT3 帧格式和 CRC，帧的汉明码距为 6。

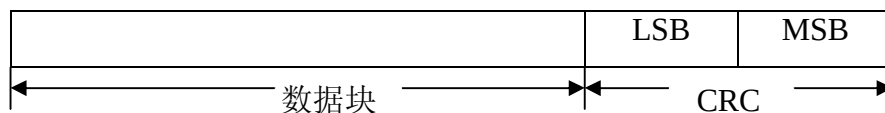


图 3—6 CRC 的次序。

### 3.4 数据链功能码

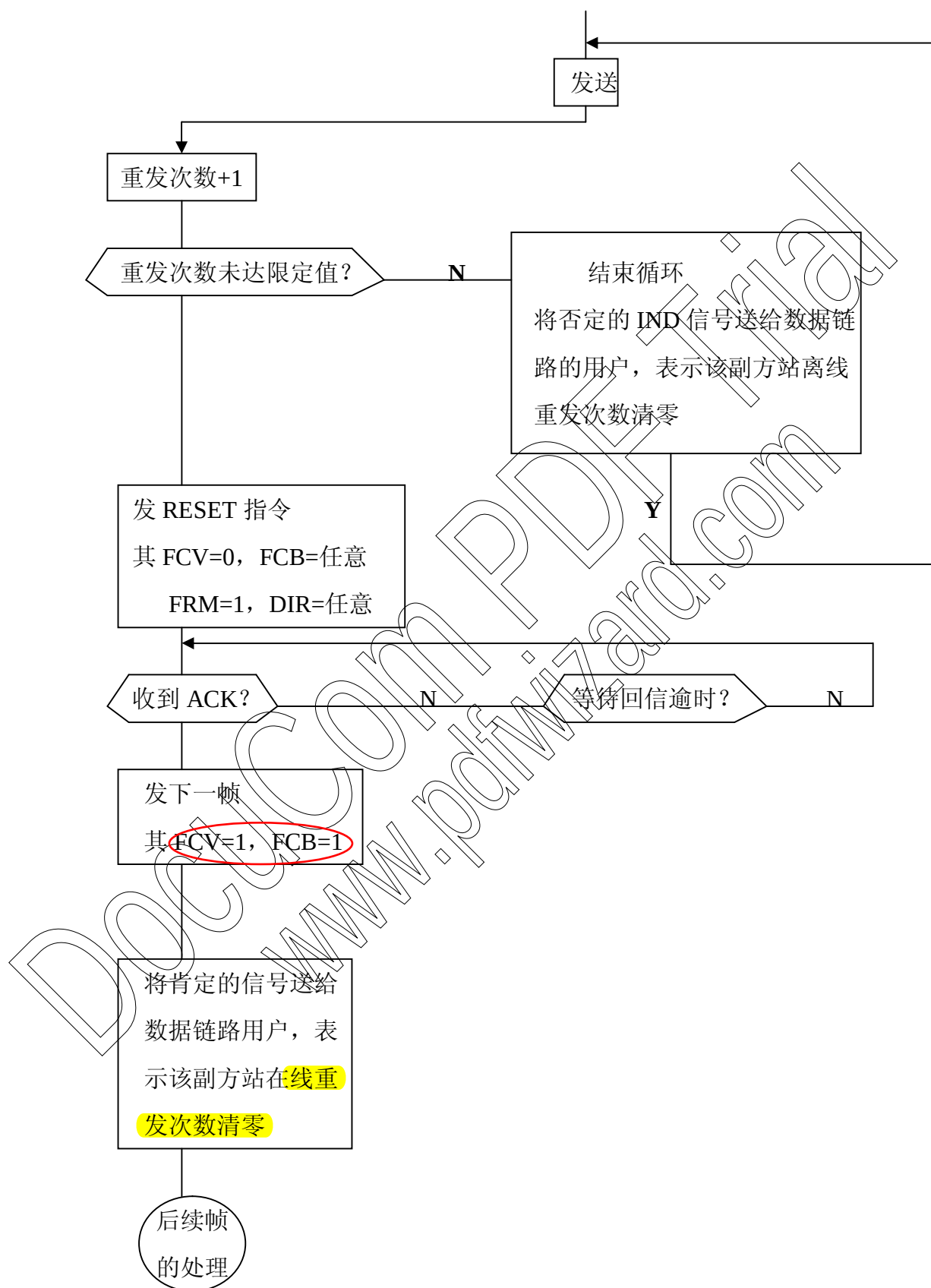
Reset (复位)

该功能码的目的是为下次 SEND/CONFIRM 的通信回合，而使原方与副方站整步。在收到了并回答了 RESET 命令之后，副方站就将开始自原方站接收带有 FCB=1 的原方报文。RESET 命令仅开通了自原方向副方的单向通信。这是因为一次成功的通信仅保证原方的发信机与副方站的收信机进行通信。原方站，当它想和副方的单向通信。这是因为一次成功的通信仅保证原方的发信机与副方站的收信机进行通信。原方站，当它想和副方站作第一次通信时或当它查觉到一次通信失败后，就必须发送这个功能码。当一个副方站已被再起动作后或副方站已经识别出一次通信失败后，该副方站将被认为是非复位的 (un-reset)。再这种情况下，副方站将不接受来自原方的报文，直到它答复了来自原方站的复位指令 (RESET) 为止。

复位指令还使原方与副方站的 FCB 位整步。副方站在完成了对 RESET 的答复后，它期待下一次来自原方站的报文的 FCB=1 (凡 FCB 为有效者)。原方站，在完成 RESET 通信后，在发送给副方站的下帧报文内将使其 FCB 位置 1 (条件是 FCB 位有效)。

### 3. 5 原方的通信事务

按：此流程图系自原文归纳而成，CC 较醒目，但有所补充以求完整。



### 3. 6 副方的通信事务

在启动 (start-up) 以后或通信回合的失败后;

等待收获  $FCV=0$ ,  $FCB$ =任意,  $PRM=1$ ,  $DIR$ =任意的 RESET 令。

收到 RESET 令, 副方响应以确认帧 ACK (其  $DFC$ =任意,  $PRM=0$ ,  $DIR$ =任意)。副方站内的  $FCB$  状态位应置 1, (即期待, 下一次收到的  $FCV=1$  的报文帧, 其  $FCB=1$ )。同时, 副方也有个肯定的信号 IND 送给数据链路的用户。

在正常运行中, 若收到了一条 RESET 令 (其  $FCV=0$ ,  $FCB$ =任意,  $PRM=1$  和  $DIR$ =任意), 则当前的通信回合 (若有) 可以被异常废止 (可能还要将否定的 IND 送给数据链路的用户)。

在这种情况下, 要用一条确认帧 ACK ( $DFC$ =任意,  $PRM=0$ ,  $DIR=X$  去响应。副方的  $FCB$  状态标志应置 1 (期望下次收到的  $FCV$  有效的报文帧, 其  $FCB$  位=1)。可以将一个肯定信号 IND 送给数据链路用户。

### 3. 7 对用户过程的 RESET

该功能码系用以使数据链路的用户过程复位。一个副方站收到此令后, 应将一个信号 IND 送给数据链路的用户。数据链路的用户可以用这个信号去使它的内部状态复位。如果数据链路用户接受了, 就要送一条确认帧 ACK 作为回答, 否则就送一条否定帧 NACK 作答。

### 3. 8 TEST (测试)

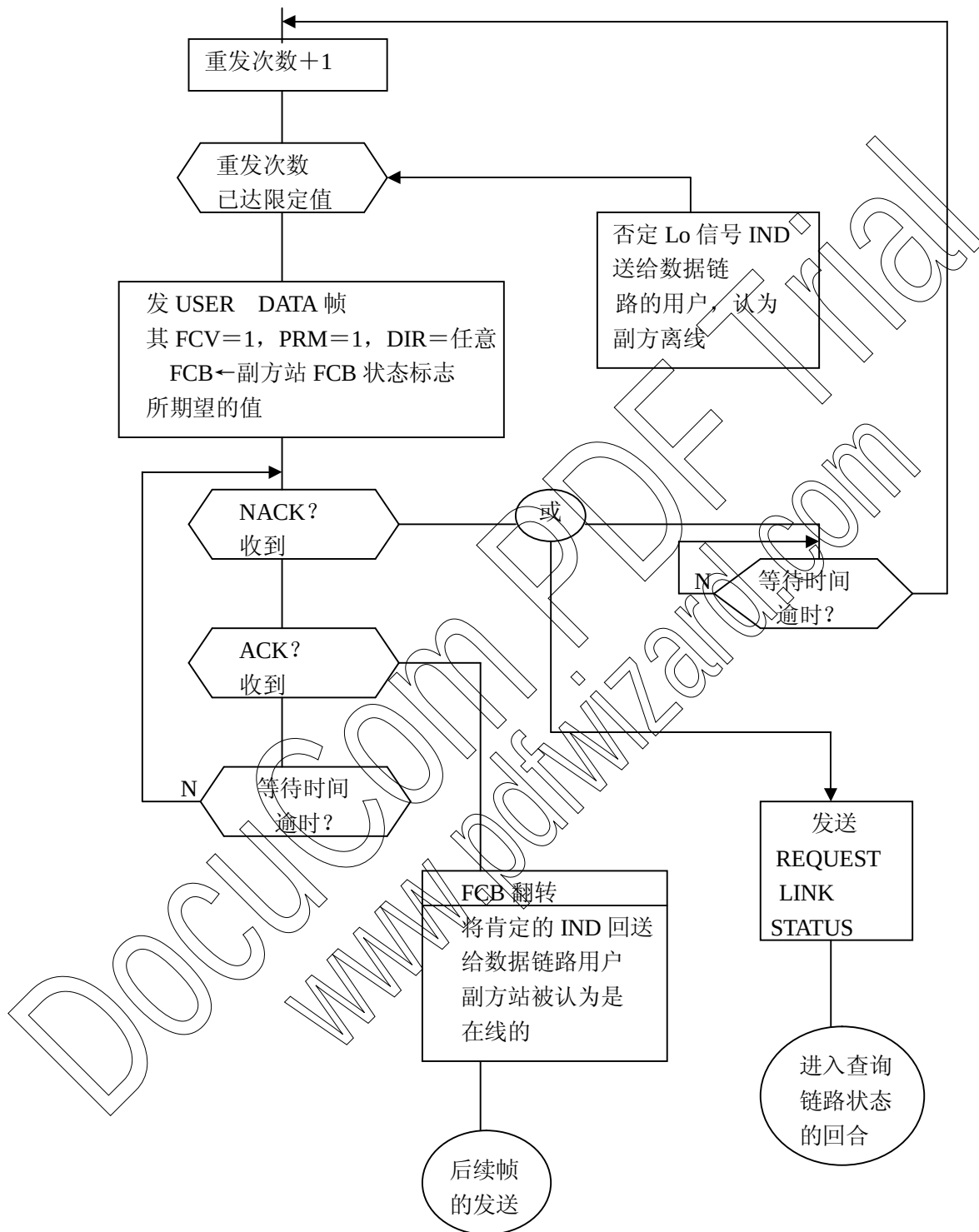
该 TEST 命令系用以测试副方数据链路的状态。一个副方站收到此令后, 它便将原方报文内的  $FCB$  位对副方为此原方站所设置的  $FCB$  状态标志 (即所期望的  $FCB$  值) 进行校核。若两者不符, 则副方站应送出上次的副方 confirm 帧。否则, 就送出确认帧 ACK 作答, 并将所期待的  $FCB$  状态标志翻转一次。

副方站在响应中可按规定设置  $DFC$  位。

### 3. 9 USER DATA (用户数据)

用户数据功能被用以对一个副方站发送被确定了的数据。在通信可以开始之前, 副方站必须按上述规则 (见 RESET) 已被 RESET 了。所发送的帧包含有从原方数据链路的用户要传递给副方站数据链路的用户, 之用户数据。传送的步骤如下:

原方事务:



### 副方事务:

收到 USER DATA 帧 (FCV=1, PRM=1, DIR=任意, 以及 FCB 位的状态与副方的 FCB 状态标志位 (对 FCB 位的期望值) 相符)

如果数据链路用户已可以接受这个 USER DATA, 则用 ACK 帧去作出响应 (其 DFC=任意, PRM=0, DIR=任意) 否则就用一个 NAC 帧去响应 (控制字的标志同 ACK) 并退出此循环。

### 3. 10 无需确认的用户数据 (unconfirmed User Data) .

本功能系用以将用户数据发送到副方站而无需予以确认。如果用户的数据是低优先级的, 用这个方法可以充分利用此系统之带帧内包含原方数据链路的用户要传送给副方站数据链路用户的用户数据。传送的步骤如下:

#### • 原方

发送非确认的用户数据帧, 其 PRM=1, DIR=任意, FCV=0, FCB=任意。  
向数据链路的用户宣告 IND 信号是肯定的。

#### • 副方

接收非确认的用户数据帧并向数据链路的用户送一个肯定的 IND。

### 查询链路的状态 (Request link status)

该命令系用以查询副方站数据链路的状态。副方站对口查询, 将用一个链路状态 (LINK STATUS) 的响应帧来答复:

若数据链路正忙或数据链路用户不能再接受任何数据, 则使 DFC 位置。

若数据链路不忙且数据链路用户可以再接受数据, 则使 DFC 位置, 该传输过程与 TEST 相似, 所不同者原方站仅当在 USER DATA 的传送回合中收到了一帧 NACK 时才使用这条命令。

### 3. 11 传输过程

已定义的帧型, 其用法示例如下:

#### 对副方链路之复位

原方站发送一帧 SEND-CONFIRM RESET (按: 功能码 # 0) 给一个副方站。副方站收到这个报文, 用一帧 A C K (按: 功能码#0) 作响应。

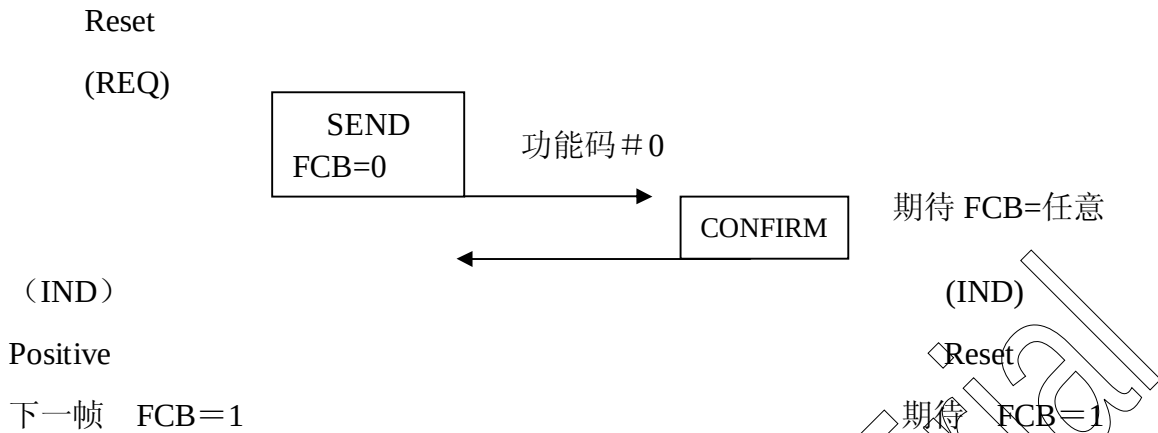


图 3-7 使副方链路复位

### 对用户过程之复位

原方站发送一帧 SEND—CONFIRM Reset User Process (按功能码 #1) 给一个副方站。副方站收到此报文，用一帧 ACK (功能码 #0) 去响应。

Reset 复位

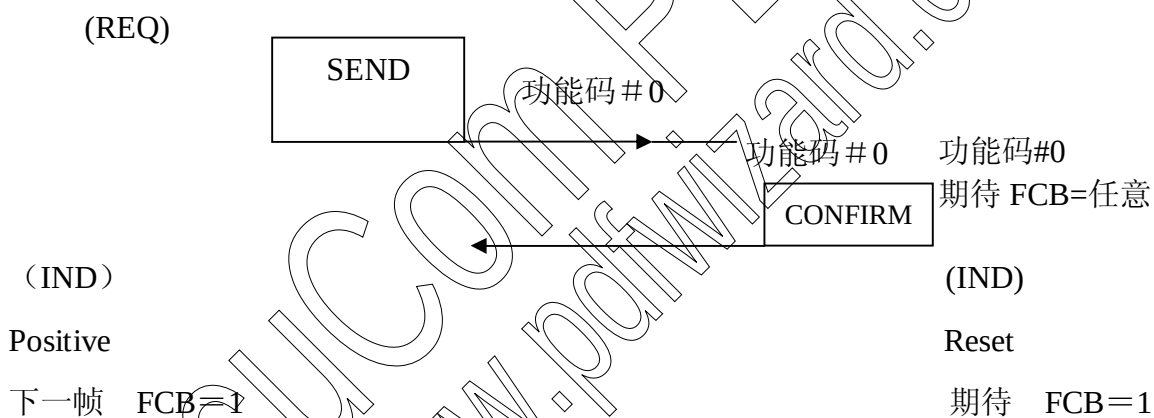


图 3-8 使用户过程复位

### 3. 12 发送/确认用户数据 (Send/confirm User Data)

在本例中，所指定的主站 (master) 作为一个原方站，发送一条 SEND-CONFIRM (要确认的发送) 帧给一个作为副方站的非主站。这是在使副方链路复位后，FCV 有效条件下所发的第一帧，所以该帧的 FCB=1。副方站所期望的 FCB=1，因为这是它的链路被复位后所收到的 (FCV 有效的) 第一帧。它随即回送 CONFIRM 帧。主站收到这个 CONFIRM 后，就认为报文已被正确地接收到并向主站数据链路的用户发出表示成功的信号 IND。

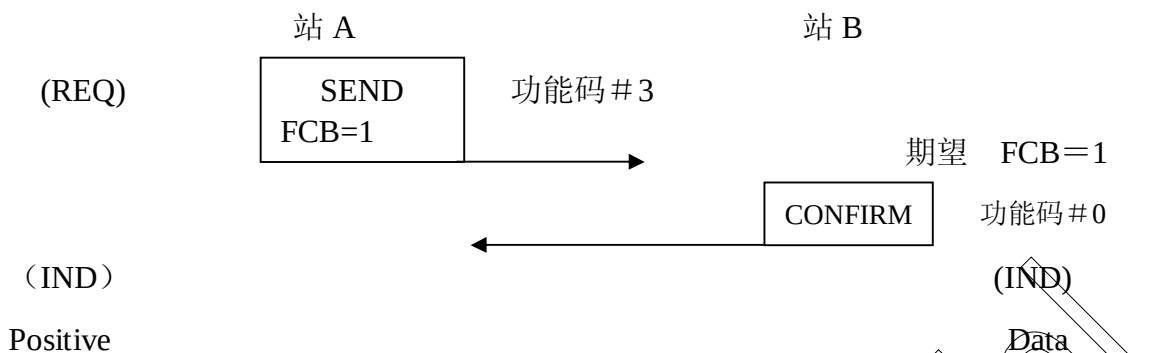


图 3—9 自 A 站发送 / 自 B 站确认

在下面，图 3—10 中，由一个非主站作为一个原方站发送一个 SEND—CONFIRM 帧给一个作为副方站的主站。由于这是在副方链路已被复位后的第二个发送帧，在此 SEND 内 FCB=0。此时副方也期待所收到的帧，其 FCB=0。副方收到后，便以一个 CONFIRM 帧作为响应。非主站方收到此 CONFIRM（按：功能码#0），就向非主站数据链用户给出成功的信号 IND。

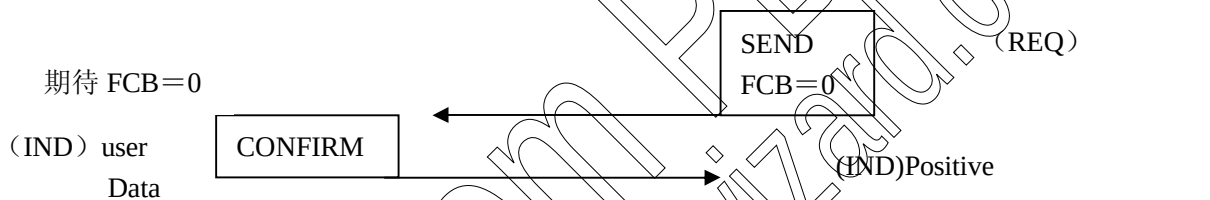


图 3—10 自 B 站发送 (SEND) / 自 A 站发确认

在图 3—11 中，所指定的主站发三个连续帧至同一个非主站。

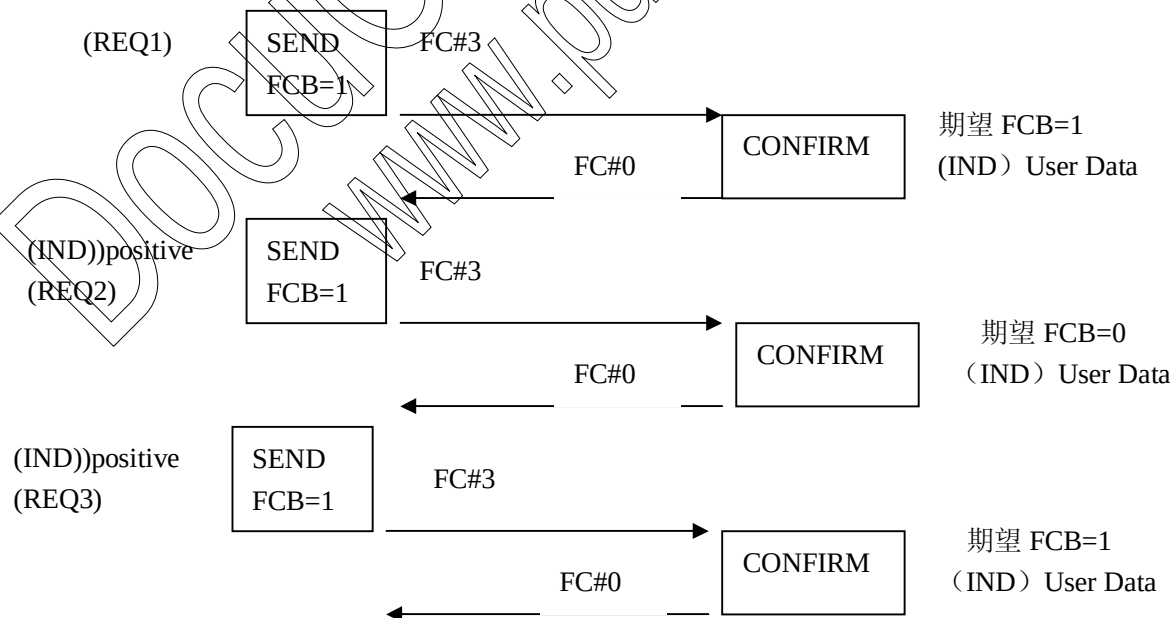


图 3—11 自 A 站 SEND 多帧/自 B 站发 CONFIRMS

在图 3—12 中，所指定的主站作为原方向非主站的副方发送一帧报文。该例说明，来自副方站的 CONFIRM 被失落时，会发生什么事情。

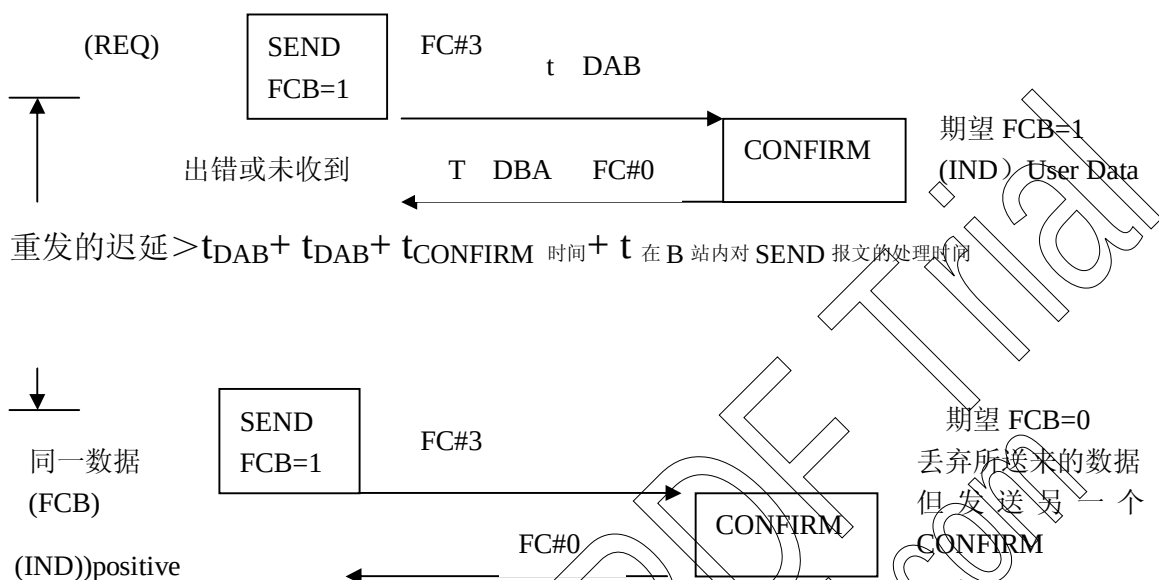


图 3—12 帧计数位的操作

在图 3—13 中，所指定的主站作为原方向非主站的副方发送两帧报文。本例说明，发 (IND) 自主站的 SEND 帧被失落时，会发生什么事。

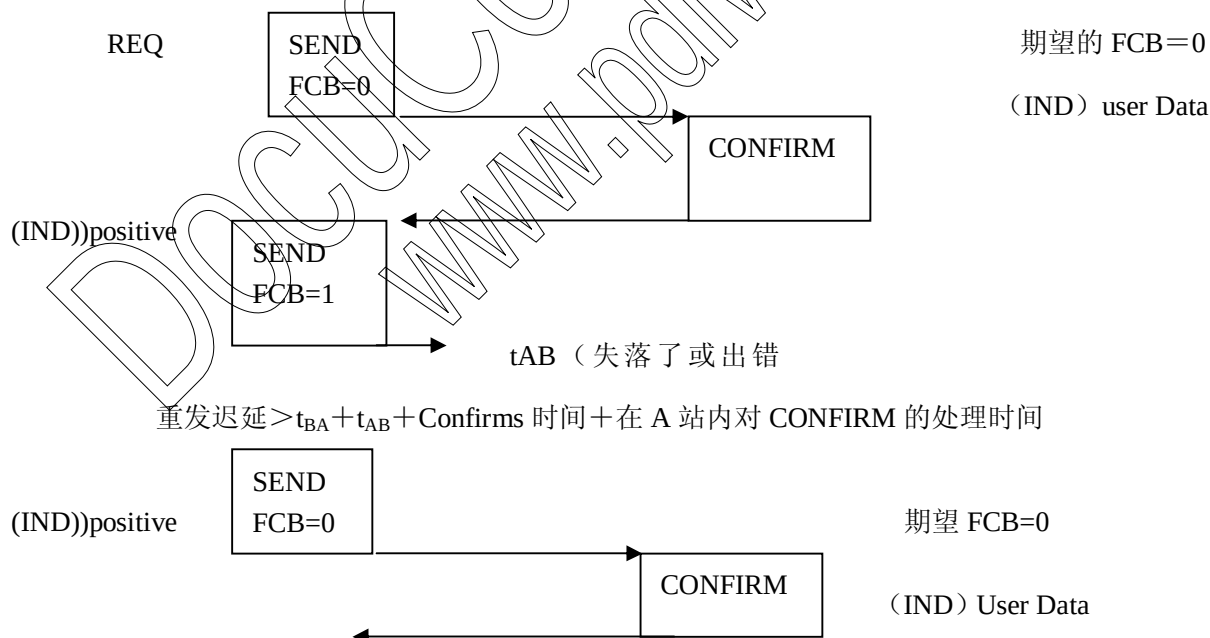


图 3—13 帧计数位的操作

注：主站和非主站作为原方时，都会重发 SEND 帧。

### 3. 13 Send/No Reply Expected(发送而不期待回信)

在图 3—14 中，主站非主站的原方站向主站或非主站的副方站发送三帧报文。在成功地发送了 SEND 帧之时，原方站的信号 IND 向数据链路的用户表示成功。副方站在收到一个有效帧时，便以信号 IND 向数据链路用户表示有数据可用。

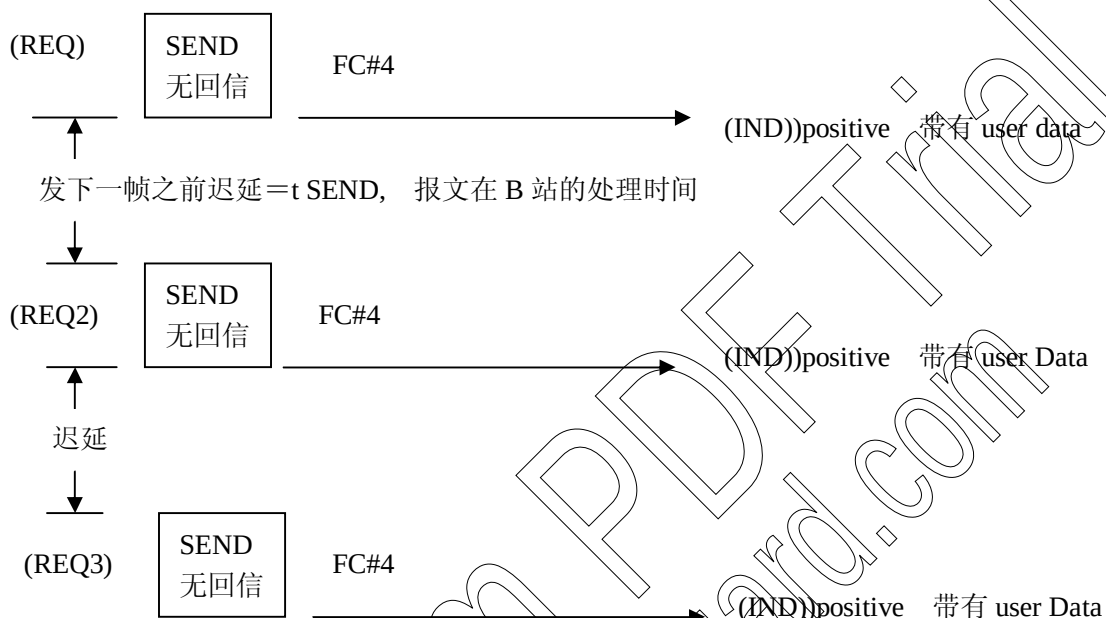


图 3—14 自 A 站发送而不期待回信号

#### Send/NACK

图 3—15 中，非主站的原方站发送一帧报文给作为副方的主站。在收到第一个 CONFIRM 时，原方向数据链的用户指示成功的信号。原方发送第二帧给副方。作为副方的主站判定它此时不能再接受任何帧时，它就发回一帧 NACK。原方收到 NACK 后，将停止传送的回合并向数据链路的用户送一个否定的信号 IND。

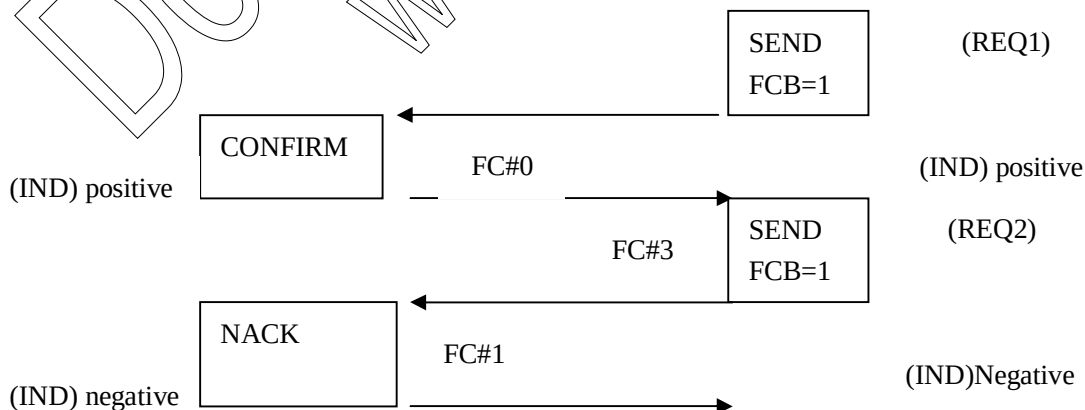


图 3—15 自 B 站发 SEND / 自 A 站发 NACK

### 3. 14 Request/Respond (查询 / 响应)

图 3-16 中，一个原方站送 (SEND) 连续帧至一副方站，当副方站不能再接收帧时，其 CONFIRM 报文所含 DFC 就置位，原方站收到这个 CONFIRM 时，当停止对该副方站，发送数据帧并代之以周期性地发送 REQUEST-RESPOND 帧去查询副方站的状态，副方站对查询以其当前状态的 DFC 作为响应。副方站若已准备好再接收数据帧，则回复的 DFC 为 0，否则所回复的 DFC 为 1。当原方站识别出 RESPOND 帧的 DFC=0 时，就继续发送 SEND 帧。

(REQ 1)

(IND)Positive

收到 DCF=0 的 CONFIRM 帧，这是继续发送下一个 User Data 帧的条件。

(IND ) Positive

(REQ=3)

收到 DCF=0 的 CONFIRM 帧，这是发送下一个 User Data 帧的条件。

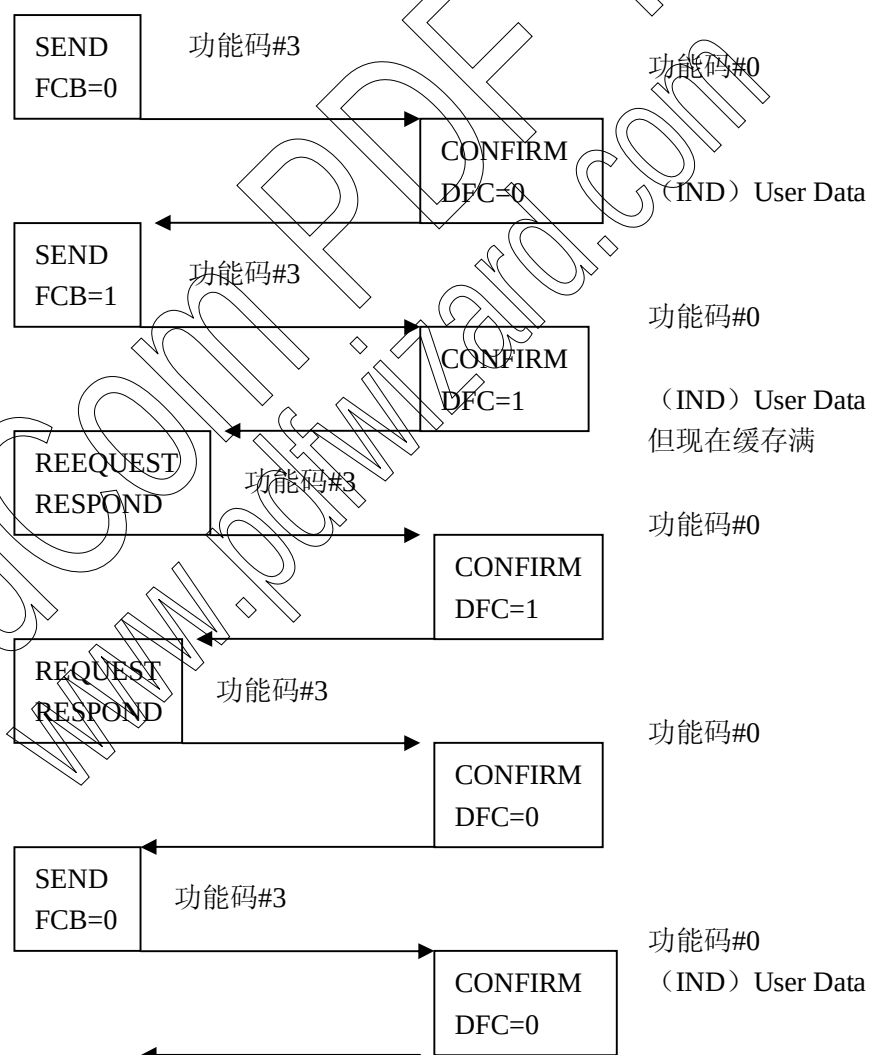


图 3-16 REQUEST/RESPOND 帧及 DFC 位的用途

## 第四章：数据链路的服务与职责

---

### 4.1 数据链路的功能

本节说明由数据链路所作的服务及其功能。数据链路层原语（**primitive** 译作原语，它类似一条扩充了的机器指令。它与一般广义指令的区别在于它可看作是**不可中断的**）。会满足网络层和伪传输层的通信要求。

数据链路负责实现以下的功能：

- 实现报文的重发
- 对控制字内 FCB 位的整步与管理
- 根据缓存的可用空间对 DFC 位置位和清除。
- 在一个拨号的环境中，当用户请求一个所指向的服务时，根据目的地址参数自动建立一个连接。

- 在一个拨号环境中断开连接
- 将用户数据包装入规定的帧格式并将数据传输至物理层
- 从来自物理层的帧中解出用户数据。
- 控制物理层的所有方面
- 实现防碰撞/检测过程以保证通过物理链路可靠地传输数据
- 对于自物理链路接收到的全部有效帧（功能码）做出响应，

数据链路负责提供以下的服务：

- 在对等的 DNP 数据链路之间交换 SDUS
- 对数据链路的用户提供出错信号
- 对 SDUS 的排序
- 按优先次序进行 SDUS 投递
- SDUS 性质的投递

SDUS 这种服务数据单元，仅在对等的 DNP 数据链路间交换。

优先级投递可以是 **EXPEDITED**（加速）或 **NORMAL**（常规），用以表示请求优先级的高或低。

性质投递可以是 **SEND-NO-REPLY** 或 **SEND-CONFIRM**，用以表示是否要求对报文的

确认。

当对一个请求的响应未被收到时，应有出错信号送给数据链的用户。

#### 4. 2 接口说明

用一个虚构的程序作为数据链路原语的示例，用以表示在一个实际实施中的要求和作为，并非以此当作接口的确切定义。

当数据链路已被系统起动或组态后在任何时候可使用数据链路请求 (REQ) 的服务。

Confirm = request-data-link-service

(SERVICE,  
TIME-SERVICE  
Destination,  
Source,  
Send-data-buffer,  
Send-count,  
Retry-flag,  
Time-of-transmission

输入:

- SERVICE-要执行的服务
- TIME-SERVICE-要完成服务的保证时间
- Source-用于发报文的源地址
- Destination-用于发报文的地址
- Send-data-buffer-在报文中发送的数据
- Send-count-在报文中的字节数
- Retry-flag-指示数据链层，是否要重发未被确认的帧
- Time-of-transmission-报文中第一字节第一位的发出时间

输出:

- time-of-transmission-time 报文中第一字节第一位已发出后的时间。
- Confirm = 0—所请求的服务已成功。  
1—所请求的服务已失败。  
2—所请求的 SEND 数据服务被当前的原方站所终止(自副方站收到了一个 NACK

帧)

3—服务程序未完成。

4—所请求的服务现时不能进行,由于数据链路正忙或是因为有先前所请求的通信回合,或是有当前非被请求的通信回合或是等待有物理层可用。

Service = 0—使用 SEND-CONFIRM 帧发一帧在参数中所指定的报文,

如果数据链路忙,就失败了。

1—使用 SEND-NO-CONFIRM 帧发一帧在参数中所指定的报文,如果数据链路忙就失败了。

2—使用 SEND-CONFIRM 帧,加急发送一帧在参数内所规定的报文。如果使用一个半双工系统,可以迫使当前的副方通信回合撤消(即在下一个副方发送的通信中强迫发一个 NACK 帧以替代原来的 CONFIRM 帧)。这种操作仅当原方站使用 SEND-CONFIRM 帧时方可行。

3—使用 SEND-NO-REPLY 所要用的帧加急发送一份,在参数中所指定报文。在半双工系统中,这可以意味着取消当前的副方通信回合(如以上所述)。

4—返回链路状态。若数据链路不忙,则返回成功。

Time-service=0—在 time-of-transmission 所规定的时间发送报文。这种服务应用最高优先权。

1—在任何时候按规定的优先级发送报文。数据链路信号(IND)可以在任何时候为服务用户所查询,但应该尽可能经常地去检查以便得到所收到的数据。

indications=request-data-link-indication (

source-address,

destination-address,

received-data-buffer,

received-data-count,

time-of-reception)

输出:

- source-address—所收到报文之源地址
- destination-address—接收点之目的址
- received-data-buffer—所接收的报文

- received-data-count—报文中的字节数
- time-of-reception—收到报文第一字节的第一位之时间
- indications=0—无信号可报告

1—数据链路已收到一个有效的报文，该报文已被放在 received-data-buffer 中，所收到的字节数已被放在 received-data-count 内。所收到报文之源地址已被放在 source-address 中，若数据链路被组态为一主站，则报文第一字节第一位被收到的时间已被放入 time-of-reception 如果数据链路被组态为外站，则 time-of-reception 仍被保留，但这服务的用户必须要当心，在外站被时间整步之前，收到的时间有可能不正确。

2—数据链路已检查出一次通信回合的失败。

## 第五章：物理层的接口

本章节说明 DNP 第 3 版数据链路对物理层的接口。该接口描述了任何物理层所必须提供的服务以便接纳 DNPV3.00 的 Datalink。

### 5.1 物理层说明

向数据链路所推荐的物理层是一种面向位串行的异步物理层，支持着 8 位数据，1 个起始位，1 个停止位，无奇偶校验以及 RS-232C 的电压等级和控制信号。CCITT V24 标准说明用以对 DCE（数据通信设备）通信的 DTE（数据终端设备），通常是一个移频键控的调制解调器（FSK）。这种类型的电路可以连接至一个公用交换网（PSN）或连接至一条专用的租用线。在每种情况下，都必须采用合适的调制解调器，并且必须最低限度地遵守 V24 标准的 DCE 定义。

物理层必须提供五种基本服务：发送、接收、连接、拆接和状态。发送服务将数据字节转换成位串数据供 DTE 与 DCE 之间通信使用。它必须提供正确的控制信号以便对指定的 DCE 通信。接收服务必须能接收到来自 DCE 的数据，因而对 DCE 提供正确的信令以便接收数据而不是噪音。连接与拆接服务提供对 PSN（若用 PSN）的连接与拆接。状态服务必须能回送物理介质的状态，作为最低限度的服务必须能指示出介质是否在忙。

物理链路服务的原语用虚构程序例示其在实际执行中的要求与作为，并不意图当作接口的准确定义。

在物理层已被起动且已用所有有关的参数组态之后，物理层的请求可在任何时候发送。

```
Confirm=request(SERVICE,  
                Data-buffer,  
                Data-count,  
                Modem-string,  
                Time-of-transmission)
```

输入：

- data-buffer:所要发送的数据
- data-count:所要发送的字节数

- modem-string: 对 ICE 的命令串

输出:

- time-of-transmission: 报文第一字节第一位的发送时间
- confirm=0—所请求的服务已成功

1—所请求的服务已失败

2—服务程序未被执行

3—所请求的服务现时不能进行, 因为物理链路正忙或是忙于先前所请求的通信回合, 或是忙于当前非请求的通信回合或是等待 DCE 可用。

• Service=0—发送一份在 data-buffer 中所指定的报文, 其长度规模由 data-count 内所指定。

1—使用在 modem-string 内规定的命令串去初始化 DCE。

2—使用在 modem-string 内所规定的命令串去连接 PSN

3—自 PSN 上拆接。

4—查询物理链路的状态, 若忙则返回至 0, 若不忙则置 1。

物理层的信号 (IND) 可以由用户服务在任何时候去查询, 但应尽可能地经常去检查以便得到所接收到的数据。

Indications=indicate(received-data-buffer,  
Received-data-count,  
Time-of-reception)

输出:

- received-data-buffer—所收到的报文
- received-data-count—报文中字节数
- time-of-reception—报文中第一字节第一位的收到时间
- indications=0—无信号可以报告

1—物理层已收到一份报文, 它已被放入 received-data-duffer 内, 所收到的字节数已被放入 received-data-count 内。

2—DCE 已连接于 PSN (打进来的呼叫)。

3—DCE 已自 PSI 上拆接 (挂断)。

4—物理层已检查出问题, 用这个链路或 DCE 去通信是不可取的或可能的, 要等

到过些时候再说。可能要求对 DCE 重新初始化。

DocuCom PDF Trial  
www.pdfwizard.com

## 第六章：物理层特性

### 6.1 线路配置

不论用什么物理层，用以构成一个 SCADA 通信网络有两种物理的拓扑结构。这就是直接的和串行总线的拓扑结构。

直接的拓扑结构有两个物理节点，在两个物理节点间接相连。这种拓扑常被称之为点对点式，它可以是点对点间直接的物理电缆，也可以是在两点之间用无线电或调制解调器网络或是通过公用交换网（PSN）的拨号连接。

串行总线的拓扑可拥有多于两个物理节点，每个节点连接在同一个信道或通信线上作为串行总线网络上的节点。这种拓扑常称之为多点(multi-)配置并且通常是用很多台输入/输出端并接的 Bell202 调制解调器所构成。在这种配置中，有一个节点被认为由它控制该物理网络。该点常称之为 SCADA 的主站。该点向多点发送又自多点接收。其余的点接收主站节点的报文并向主站发送。

DNP 数据链路支持多主站，多从站和对等(peer-to-peer)通信。在对等通信中，所有的设备扮演从属的数据链路(slave data link)，以采用防碰撞措施，没有一个设备拥有更高的优先权，并且能作自发地发送。

在一个多主站的配置中，那些主站比从站拥有较高的优先权。然而，优先权该在主站间分配的。

#### 传输的模式

由 DNP 支持的物理层必须在串行模式中收/发数据。通常所传输的数据单元为 8 比特长。传输可以是异步的、同步的或准同步的(isochro)用同步式调制解调器可以得到较高的吞吐率。所采用的实际机构对由此链路的操作无影响。

#### 本地环路

以通信节点上（即 NOT 在调制解调器上）数据通信电路的终端可以用一组线或四线电路去实现（即 TX/RX 对或独立的 TX 和 RX 对）。

DNP 数据链路可以用一组二线电路作半双工工作，并可用一组 4 线电路作全双工或半双工工作。

DNP 数据链路在本地环路上既能支持全双工也能支持半双工工作。然而，在两种情况

下将采用很不相同的处理。

DocuCom PDF Trial  
www.pdfwizard.com

## 第七章：物理层的规程

### 7.1 基本概念

对于物理层接口，数据链路的目的是允许数据链路向或自另一个数据链路发或收一幅报文。为了达到此目的在数据传输在进行时数据链路必须能够在物理通信电路上检出数据的存在并利用控制线的信令控制物理电路。此外，主站（或最高优先级的设备）需要能控制通信电路并封锁其它站的发送。

在一个直接连接型的拓扑中，原方站（始发站）仅能对一个站通信。若此电路为四线的，则将使用全双工规程。然而，若为二线电路则将采用半双工规程。在这种情况下，若两个站同时想传输数据，就会发生一次碰撞。一个对拨号 PSN 的直接连接就是典型的二线制，但是从站到调制解调器却是一个四线的全双工电路并且必应用于全双工样式。拨号的调制解调器必须用 CTS 拖住发信器直到 RTS 建立之后。

在一个多点的拓扑中(multi-drop)被指定的主站可以作为许多付方站之原方站。在这种情况下，在二线制或四线制电路中都有发生碰撞的机会。

在一个二线制电路中所指定的主站其报文可以和任何它站的报文碰撞，同时从站的报文也会任何时候相互碰撞。

在一个四线制电路中，主站的报文不会与从站的报文碰撞，但从站的报文会相互碰撞。

### 7.2 半双工规程

在一个二线制系统采用半双工规程时，有几种方法可以在通信电路中防止碰撞或自碰撞中恢复过来。不管用什么物理层，所有的物理层都应能回送一个数据载波检出信号(DCD)以表示该电路上有没有通信业务。

在二线制系统中，当主站或从站正在电路上传输时，就出现有效的 DCD 信号。当出现此信号时，定有一个站正在此电路上传输。在此期间，没有其它站想在此电路上传送。当此信号消失时，这个电路就开放给某个站使用。现在的问题是允许哪个站在此电路上发送。

在点对点配置中，主站或从站都可以发送。在多点 (multi-drop)配置中，主站或任何一个从站都能发送 DNP 链路规约并不对主站或从站的报文分配优先权，但是在 SCADA 系统内通常能接受应由主站控制通信电路的概念，因而该由它发送报文（若有报文要发送）。

此时任何从站，若被允许发送报文，它可能会造成碰撞，所以从站在检测出数据载波已消失之后，必须等待一些时间再准备发送。在发送前，再检查一下那个信号，如果电路仍闲着则可进行发送。如果电路正忙则该站必须再等待直到忙的信号消失再实现那个发送过程。在数据载波消失后插入一段时延再允许主站取得对电路的控制（如果需要的话）并且关闭其它的站（因为主站的传输使载波信号 CDC 生效）。

### 点对点

在点对点的配置中所需的时延为主站检出数据载波消失以及开始送报（加上系统内的任何传布延迟）所需的时间（master-miu time）。

### 多点

在一个多点配置中，这种迟延时间对每个从站都是不同的，有一个可能是将每个从站配置成等待一个时间的稳定增长量（没有重复的时间但都大于 master-min time），从而对这些站分配了不同的优先级。按此办理，系统内重要的站可以得到较高的优先权，而且很少遇到碰撞（仅在设备定时不佳或系统配置不良时才会发生）。然而，若高优先权无报可送时，就会浪费了很多的时间（从而也将损失带宽）。

另一个模式对每个从站配置一个等待时间，这个时间是在 master-min 及 max 之间的一个随机数。Max 值是系统内从站数量之函数。在这种办法中，每个站都可以按相同的方法组态，所浪费时间的平均值约为  $(\text{max}-\text{min})/2$ 。然而，碰撞还是可能发生的。Max 值愈小，则发生碰撞的机会越多。

## 7.3 全双工规程

当全双工规程用于四线制直接连接的电路时，就没有发生碰撞的机会，因为有两个独立的通道分别供收信及发信之用。在这种情况下，主站与从站双方都能在任何时候按照需要传送数据。主站仍能对电路有控制权，因为只对一个站讲话，因而无需封锁其它的站。

当全双工规程用于四线多点(multidrop)系统中时，防碰撞的问题就增加了复杂性。其理由系基于这样的事实，具有两个独立通道的物理通信电路通常只能在接收方向上侦检通信业务。在双线制系统中，在接收与发送两个方向的通信业务都可以被检出，因为两者都在同一电路上但在四线制系统中，发送与接收却在不同的电路上进行的。

### 点对点

在一个点对点，全双工系统内主站与从站双方都可以同时发送而不发生碰撞，所以无需碰撞检出/防止的措施或入口控制机构。

### 多点 (multi-point)

在一个全双工的多点系统内，主站在任何时候可以发送报文而不会发生碰撞，但有可能不能立即收到数据链路的确认，由于另外一个站（作为原方的从站）已在原副方站之前控制了主站的接收电路，或者说发生了一次碰撞。

从站的报文将随机地碰撞，因为一个站无法知道主站的接收电路是否已被另一个站所控制。解决的办法是利用控制电路（在 RS-232 的情况下就是 RTS）对从站发出信号告诉它另一个从站已控制了主站的接收电路。这个信号必须是对从站的一个输入信号，它表明取得主站接收电路控制权的请求。

一种简单的解决办法是允许从站的报文碰撞。在这种情况下，主站仍能发送高优先级的报文，但可能有碰撞，这种碰撞将使一个副方站超时。（按：可能是超时收不到回文）。

### 拨号的调制解调器

一个拨号的调制解调器采用四线制全双工电路，它典型地要求用几个控制信号（在 DCD 以外的信号）去操作它。拨号电路是一个点对点的电路，然而，它的载波数据信号和直接连接的电路之载波信号具有不同的意义。这里数据载波（DCD）表示这个 modem 已与另一个 modem 通过 PSN 作了电气连接。它并不必需表示数据已在传送了。CTS 线表示数据链路已可安全发送了。DMP 数据链路在发送每帧之前先要保持 RTS 有效，并等待 CTS 线有效再发送数据。于是再释放 RTS，如 DCD 信号失效，则可认为失掉了一次连接，如果需要就该重新拨号。

## 第二部分

***DNP V3.00***

## 传输层

# 目 录

---

|                  |    |
|------------------|----|
| 绪言.....          | 37 |
| 本规范的目的.....      | 37 |
| 谁该使用此规范.....     | 37 |
| 帮助与附加的文件.....    | 37 |
| 本规范是如何组织的.....   | 37 |
| 使用本规范的惯例.....    | 37 |
| 第一章 概貌.....      | 39 |
| 第二章 传输功能.....    | 40 |
| 2.1 传输标题.....    | 40 |
| 2.2 传输标题段定义..... | 42 |
| 2.3 帧组装.....     | 42 |
| 2.4 报文的传送.....   | 42 |
| 第三章 传输服务与责任..... | 44 |
| 3.1 传输功能.....    | 44 |
| 3.2 接口说明.....    | 44 |

# 绪 言

---

## 本规范的目的

本文本规定了“分布网络规约”(DNP) V3.00 的“传输功能”，传输步骤和“传输规约的数据单元”(TPDU)。

## 谁应该使用此规范

本规范是为有兴趣了解 DNPV3.00“传输功能”之功能与报文格式的通信工程师和程序员准备的。这里包括程序员对 DNPV3.00“传输功能”软 / 硬件之实行与设计以及质量保证人员对 DNPV3.00“传输功能”之测试与验证的手。应用系统的程序员会发现在判别定如何接口于 DNPV3.00“传输功能”以及如何利用这些功能时，本规范是很有用的。熟悉 ISO-OSI 7 层规约，IEC 3 层 EPA 和 IEC TC-57 标准对此是有所助益的。

## 帮助与附加的文件

以下的文件可能有助益的。

IEC870-S-1 和 IEC870-5-2 标准(草案)-由技术委员会 N0.57 专为远动系统的数传部分写的。

DNP V3.00 DaTa Object Library

(P00g-OBL)

DNP V3.00Application layer (P00g-OPD.APP)

DNP V3.00 Datalink Layer(P00g-OPD.DL)

## 本规范是如何组织的

第 1 章：概貌-传输功能之概貌

第 2 章：传输功能-分组 (packet) 格式和传输步骤的详述

第 3 章：传输服务与责任-由一个接口提供给传输功能的服务

## 缩写词与首字母缩写词的清单

## 使用本规范的惯例

在本文中 octet 与 8 毕特的字节同意义。字节中低位为 0，高位为 7。

不规则的大写字母以表示技术名词，例如：Data link “indications”通常可用 IND 或 INDication 来表示。

DocuCom PDF Trial  
www.pdfwizard.com

## 第一章：概 貌

---

这个文本定义“分布网络规约” DNP V3.00“传输功能”，“传输规约数据单元(TPDU)以及传输服务与传输步骤，主站、分主站与外站或智能电子设备(IED)都能使用这些传输功能，在原发方(primary or originating) 站和原收方(secondary or receiving) 站之间传递报文。本规约中主站与主站和外站都既是原方(原发方) 站又是副方(原接收) 站。

本规约所支持的 ISO 之 OSI 模型只规定了物理层，数据链路层和应用层。这被名之为“增强了规约的体系结构”(EPA)。然而，为了支持先进的 RTU 功能和报文长度大于 IEC 870-5-1 所规定的最大帧长之报文，DNP V3.00 的“数据链路准备使用这种伪传输层以实现报文的组装与拆卸。

该伪传输层实际上是一个超数据的链路规约，它正常是某些 OSI 数据链路之一部份。然而，由于 IEC 数据链路(DNP V3.00 “数据链路层”)不支持数据链路内的这些功能,所以有必要将它们移出数据链路以保持遵从性。

## 第二章：传输功能

本节描述“传输层功能”，它对 DNP 链路层讲是一个伪传输层。伪传输层是只对那种长报文规定的，其长度大于原方与副方站间的一个链路规约数据单元(LPDUs)者。这个伪传输层的作用就像在一个规约堆栈内活动的 DNP 数据链路用户。该堆栈仅包含有“DNP 数据链路和“DNP 应用层。该功能性可以允许伪传输层将一个“传输服务数据单元”(TSDU)拆卸成多个(多于 1 个)TPDUs (或帧)以及将多个(多于 1 个)的 TPDUs 组装成一个 STDUs。这种过程的工作如下：

“伪传输”层取一个 TSDU(用户数据)并将它分裂成若干个顺序的 TPDUs(每个都带有传输规约控制信息”(TPCI))。每个 TPDUs 都被送到数据链路层当“链路服务数据单元”(LSDUs)去传输。

它也会工作于相反的方式。“伪传输层自数据链路收到多个 TPDUs 并将它们组装成一个 TSDU。

LSDUs 都是用户数据分段,它们都足够地短能装入已定义了的 FT3 帧格式, 当一个原主站发一份报文给副方站时, 传输功能将报文分裂成多个 LSDUs。这些功能加一个“传输层标题”(TH)字节在用户数据分段的起始处, 它包含有供副方站重新组建完整报文所需的信息, 所有伪传输层的报文都有一个 TH。

副方站收到每个 LSDUs 都检查它的 TH 字节, 为了要知道正确的顺序并为上层构建一份 TSDU 报文。

TH 包含的信息可以识别第一帧, 最后一帧以及用一个六位的顺序号去识别每一个帧。这种信息是重组报文所需的, 也防止上层收到误导的或不完整的报文。

### 2. 1 传输标题

在数据链路收到一个完整的帧之后, 呈现于传输功能的数据其格式如下, 在此帧与其它共属同一报文的帧组装起来之前, TH 字段就被剥去了。

图 2-1 展示 TPDU 的结构

|    |           |
|----|-----------|
| TH | USER DATA |
|----|-----------|

TH—传输控制字节, 单字节长

USER DATA——1 至 249 个字节的用户数据

图 2-1 传输层报文之排列

当一个应用请求传输一份长报文时, 报文被拆成几个字段, 每段都足够地短可以安置进单个 DNP V3.00 Data link 的帧. 每个分段的最大规模为 249 个字节的用户数据. TH 被加在分段的头上, 整帧的最大字节数为 250 个.

|             |         |
|-------------|---------|
| 数据链路的最大字节计数 | +255 字节 |
| 数据链路报头字节计数  | -5 字节   |
| 传输标题        | -1 字节   |
| 应用系统用户数据    | =249 字节 |

## 2. 2 传输标题段的定义

|     |     |   |     |    |     |   |   |
|-----|-----|---|-----|----|-----|---|---|
| 7   | 6   | 5 | 4   | 3  | 2   | 1 | 0 |
| FIN | FIR |   | SEQ | UE | NCE |   |   |

图 2-2 TH 的位定义

FIN: 终了位表示用户数据的这一帧在整个用户报文中的顺序里它是最后的一帧。

FIN=0 表示后面还有其它的帧。

FIN=1 表示这是一个顺序的最后一帧。

FIR: 首位表示本帧为构成完整报文的帧顺序内之第一帧。当副方站收到一帧其 FIR=1 时, 所有以前收到的未终了的帧顺序就全部被丢弃掉。一个顺序的第一帧, 其序号可以是 0 至 63 之间的任意一号。

若收到的一帧其 FIR 位未置位, 而且当前的报文顺序号也没有递增, 则此帧就被不计。

如果整个用户报文只有一帧之长, 则 FIR 与 FIN 两个都被置位。

FIR=1 表示该帧是一个顺序中的第一帧

FIR=0 表示该帧不是顺序中的第一帧。

顺序 (SEQ UE NCE): 帧的顺序号被用来检查正在接收的每一帧是否依序。它防止帧的丢失或重复。所有的用户报文在一顺序的开始就要指定第一帧的序号, 第一帧的 FIR 位置位 (每份报文的序号可从 0 至 63 间的任一号码起始)。顺序号 63 之后的下一个顺序号即为 0。

顺序号为同属一个报文的每一帧发向同一地址或收自同一地址而递增, 并在新报文的开始时复位。顺序号不必在报文的交界处递增, 即 FIR 置位时, 任何顺序号都是有效的。

## 2. 3 帧组装

图 2—3 例举一个三帧报文的组装。报文的第一帧以 TH 段内 FIR 之置位作它的标识。

它的最后一帧以下 TH 段内 FIN 之置位作标识。

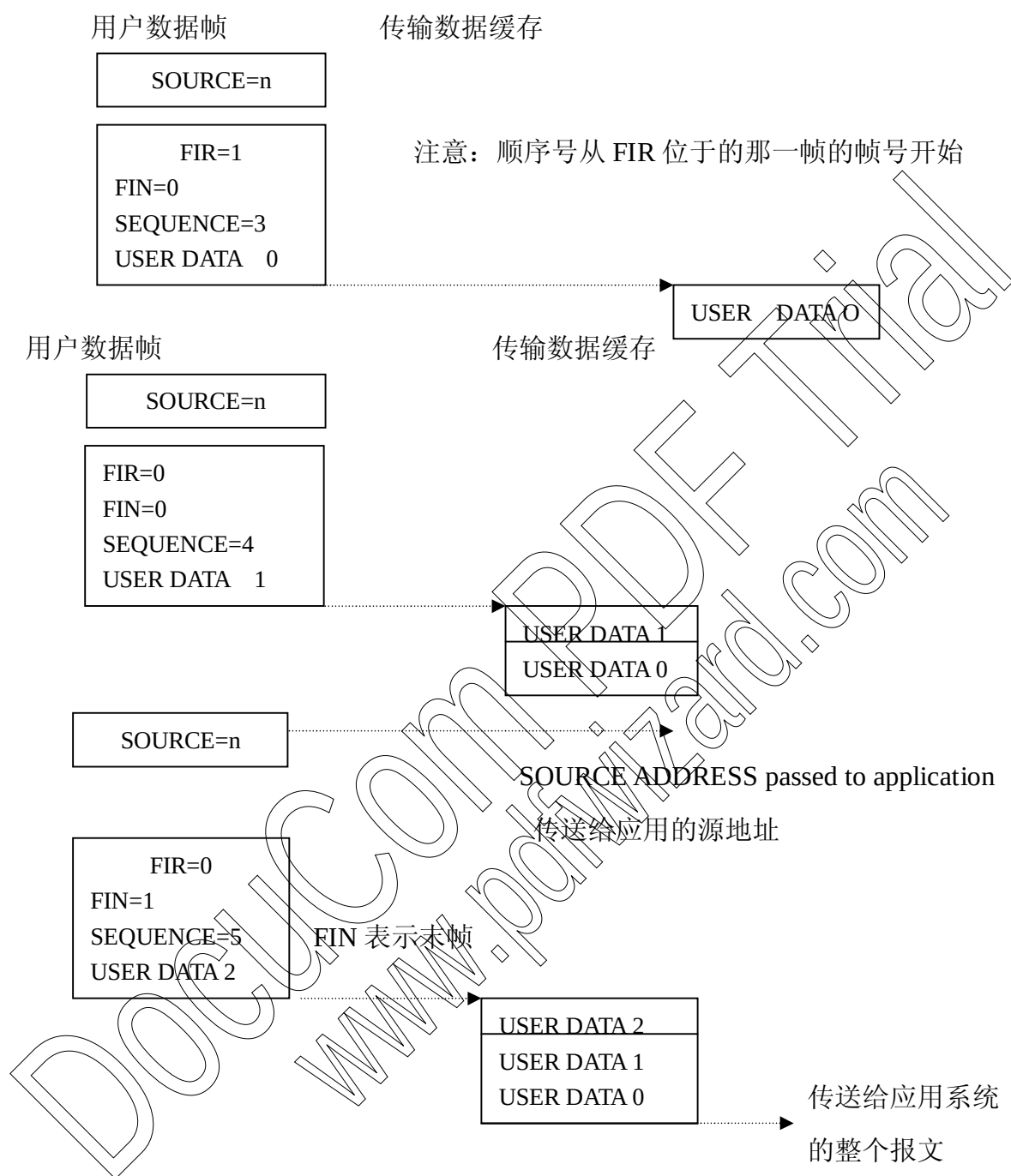


图 2—3 从 3 个数据帧组装数据

## 2. 4 报文的传送

图 2-4 例举使用“发送一确认”(SEND-CONFIRM)帧服务去传送一个单帧的报文。

图 2-5 例举使用 SEND-CONFIRM 的帧服务去传送多帧的报文。

自数据链路发送帧确认所收到的帧

自应用系统来的完整报文

DESTINATION 来自应用的参数

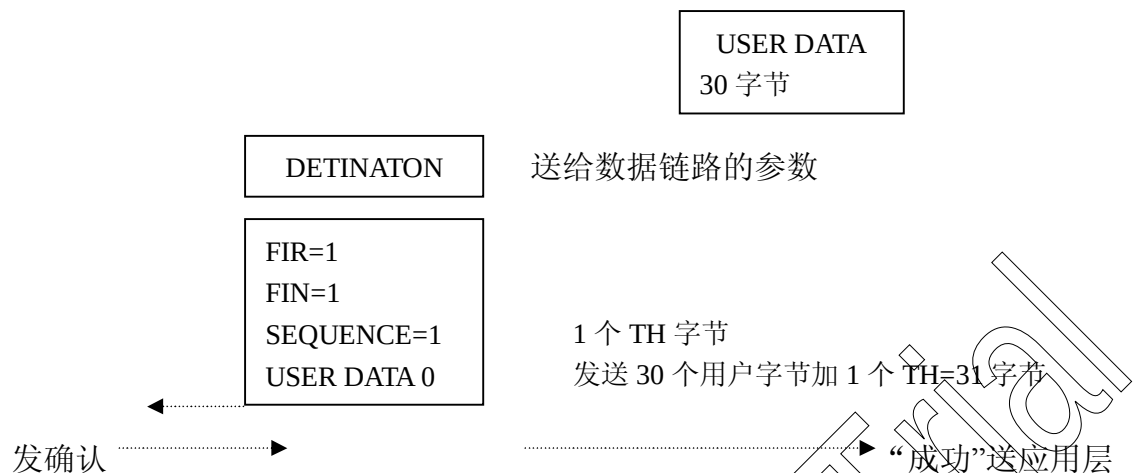


图 2-4 一个单帧报文的发送

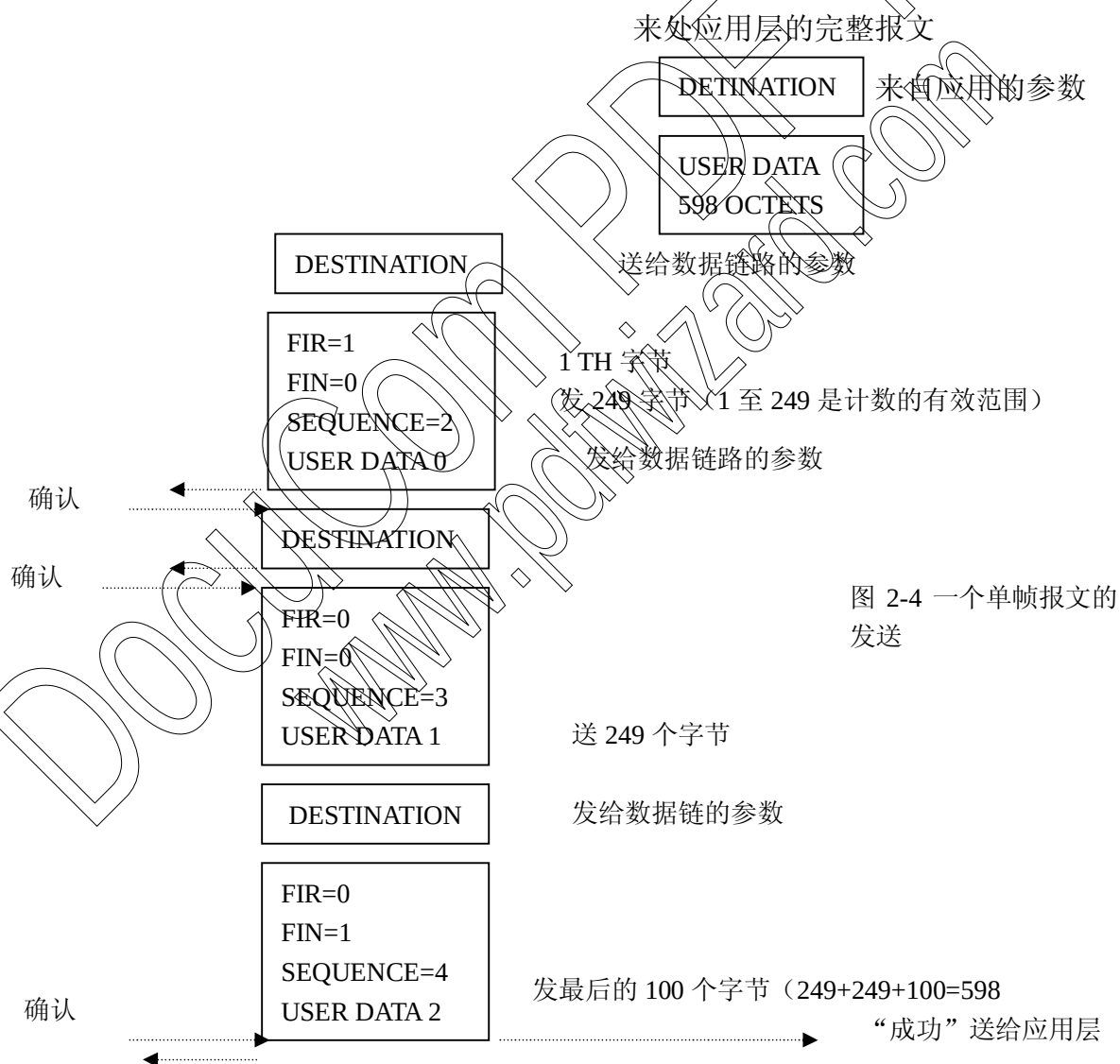


图 2-4 一个单帧报文的发送

## 第三章：传输服务与责任

---

### 3.1 传输功能

本节说明由伪传输层所提供的服务及其功能。伪传输层的服务原语满足网络层与应用层的通信要求。

伪传输层对实现以下的功能负责：

- 将用户数据分组打包在一个或多个按 DNP V3.00 DataLink 帧格式所规定的帧并使用 DNP V3.00 Data Link 的数传服务。

- 接收数据链路的多帧拆包编入用户数据。
- 除数据链路的配置以外，控制数据链路的各个方面。

传输层为提供以下的服务负责：

- DNP V3.00 伪传输层的对等层间交换 SDUs。
- 将出错通知传输的用户。
- 对 SDUs 的编序。
- 按优先级进行 SDU 交付
- SDU 的质量交付

SDU 将只能在对等的 DNP V3.00 伪传输层之间交换。

优先的交付可以设置“加快”(EXPEDITED)或平常“(NORMAL)以指明高的或低的优先级请求。

质量交付可以设置“发不要回”(SEND-NO-REPLY)或“发要确认”(SEND-CONFIRM)以表明是否要报文的确认。

出错通知是当一个请求的响应未被收到时，发给传输用户的。

### 3.2 接口说明

在伪码中例举的伪传输层之服务原语系用以举例说明在一次实际执行中的要求与行为，并非打算作为一种严格的接口定义。

在传输功能已被系统初始化和组态之后，就可以在任何时候使用传输请求服务(REQ)。

Confirm=request transport service

SERVICE

TIME SERVICE

Destination

Source

Send-data-buffer

Send-count

retry -flag

Time\_\_of\_\_transmission

输入:

SERVICE-要执行的服务

TIME-SERVICE-完成服务的保证时间

Source-发方报文中用的源地址

Destination-发方报文中用的目的址

Send-data-buffer-data 报文中发送的数据

Send-count-报文中的字节数

Retry-flag-指令数据链路层对未响应的帧要重发与否

Time-of-transmission 报文中第一个字节之第一位的发送时间

输出:

time-of-transmission-报文发送第一字节之第一位的时间

confirm= 0—所请求的服务是成功的

1—所请求的服务已失败

2—所请求 SEND 数据服务被当前的原方站截止（自副方站收到

一个 NACK 帧）

3—服务程序（code）未被执行

4—所请求的服务不能在此时进行因为数据链路忙，或是由于先前所请求的通信处理未完，或是有当前的非请求业务或是等待着物理层的可用性。

Service= 0—用 SEND-CONFIRM 帧发送一份在参数中所指定报文。数据链若忙则失败。

1—用 SEND-NO-REPLY 帧发送在参数中所指定的一份报文。数若忙则失败。

2—用 SEND-CONFIRM 帧加快发送在参数中所指定的一份报文。若使用一个半双工系统，可以迫使取消当前副方的业务（即强使数据链路在下个副方的业务（transaction）中发送一个 NACK 帧而不是 CONFIRM 帧。）这个动作只发生在原站使用 SEND-CONFIRM 帧的条件下。

3—采用 SEND-NO-REPLY 所期望的帧加快发送在参数中所指定的一份报文。在半双系统中，这可以意味着取消当前的副方业务（如上）。

4—回送链路状态，若数据链路不忙则回送成功。

Time-service 0—在 time-of-transmission 所规定的时间发送报文。这个服务应该有最高的优先级。

1—在优先级所指定的任何时间发送报文。

数据链路的指示信号（IND）可以由服务的用户在任何时候请求，但为了得到所接收的数据应该尽可能地时常去检查。

Indications = request-data-link-indication(  
Source-address,  
Destination-address,  
Received-data-buffer,  
Received-data-count,  
time-of-reception)

Output:

Source-address—所收到的报文源地址

Destination-address—所收到的报文之目的址

Received-data-buffer—所收到的报文

Received-data-count—报文内的字节数

Time-of-reception—收到报文第一字节之第一位的时间

Indications=0—无指示信号上报

1—数据链路已收到一个有效的报文，该报文已被放入 received-data-buffer 并且所收到的字节数已被放入 received-data-count。所收到的报文源地址已被放入 source-address。若数据链路被组态为主站，则报文第一字节第一位之收到时间已被放入为 Source-address。若数据链路被组态为主站，则报文第一字节第一位之收到时间以被放入 Time-of-reception。

若数据链路被组态为外站，则 **time-of-reception** 将仍被回送，但服务的用户该知道在外站被时间整步之前所收到的时间有不准确的可能性。

2—伪传输层已检出一通信业务（**transaction**）的失败。

DocuCom PDF Trial  
www.pdfwizard.com

# 第三部分

## *DNP V3.00*

### 应用层

文件版本: **0.03**

内部文件: **P009—OPD.AP**

# 目 录

---

|                             |    |
|-----------------------------|----|
| 导 言.....                    | 53 |
| 谁该使用本文件.....                | 53 |
| 求助的与附加的文件.....              | 53 |
| 用于本规范说明的常规约定.....           | 53 |
| <b>第一章 概述</b> .....         | 54 |
| 1.1 说明书与 IEC 的关系.....       | 54 |
| <b>第二章 报的关系文格式</b> .....    | 56 |
| 2.1 应用请求格式.....             | 57 |
| 2.2 应用响应格式.....             | 57 |
| <b>第三章 DNP 报文字段定义</b> ..... | 59 |
| 3.1 应用报头.....               | 59 |
| 3.1.1 请求报头.....             | 59 |
| 3.1.2 响应报头.....             | 59 |
| 3.1.3 应用控制.....             | 59 |
| 3.2 通信的流控制.....             | 60 |
| 3.3 主站请求响应与非请求响应之间的冲突.....  | 65 |
| 3.3.1 立即处理模式.....           | 66 |
| 3.3.2 确认后处理模式.....          | 67 |
| 3.4 出错恢复.....               | 68 |
| 3.5 功能码.....                | 68 |
| 3.6 内部信号.....               | 70 |
| 3.7 对象标题.....               | 72 |
| 3.7.1 对象段.....              | 72 |
| 3.7.2 限定词断.....             | 73 |
| 3.7.3 Range 段.....          | 76 |

|                        |     |
|------------------------|-----|
| 第四章    功能码的详细说明        | 83  |
| 4.1 确认（功能码 0）          | 83  |
| 4.2 读令（功能码 1）          | 84  |
| 4.2.1 读请求              | 84  |
| 4.2.2 读令响应             | 90  |
| 4.3 写令（功能码 2）          | 97  |
| 4.3.1 写请求              | 97  |
| 4.3.2 写令响应             | 98  |
| 4.4 选点（功能码 3）          | 99  |
| 4.4.1 模式控制             | 99  |
| 4.5 操作（功能码 4）          | 100 |
| 4.6 直接操作（功能码 5）        | 101 |
| 4.7 直接操作-无确认（功能码 6）    | 102 |
| 4.8 立即冻结（功能码 7）        | 102 |
| 4.9 立即冻结-无确认（功能码 8）    | 103 |
| 4.10 冻结并清除（功能码 9）      | 103 |
| 4.11 冻结并清除-无确认（功能码 10） | 103 |
| 4.12 按时间冻结（功能码 11）     | 103 |
| 4.13 按时间冻结-无确认（功能码 12） | 103 |
| 4.14 冷再起动（功能码 13）      | 104 |
| 4.15 热再起动（功能码 14）      | 105 |
| 4.16 初始化数据（功能码 15）     | 105 |
| 4.17 初始化应用程序（功能码 16）   | 106 |
| 4.18 起动应用程序（功能码 17）    | 106 |
| 4.19 停止应用程序（功能码 18）    | 107 |
| 4.20 保存组态（功能码 19）      | 107 |
| 4.21 允许自发的报文（功能码 20）   | 108 |
| 4.22 停用自发的报文（功能码 21）   | 108 |

|                           |     |
|---------------------------|-----|
| 4.23 级别分配（功能码 22） .....   | 108 |
| 4.24 延时测量 .....           | 109 |
| 第五章 等级 .....              | 110 |
| 第六章 时间同步 .....            | 111 |
| 第七章 带时间事件的二进制输入 .....     | 112 |
| 第八章 文件传输 .....            | 113 |
| 8. 1 执行写功能的文件识别对象 .....   | 113 |
| 8. 2 执行读功能 2 文件识别对象 ..... | 116 |

---

本文件规定了分布网络规约（DNP）应用层的服务与报文格式。本文件还规定了应用规约的数据单元（APDU），应用程序的流控和附属于 DNP 应用层服务的任何专用信息。

### 谁该使用本文件

本规范说明是为那些需要知道构成应用层报文消息各场段的结构和意义的。这里包括程序员对应用程序的执行与设计，和质量保证人员对应用层的测试与验证手段。

### 求助的与附加的文件

以下的文件是有帮助的：

DNP3.00 Data Object Library(P009-OBL)

Chapter 1 至 8

### 用于本规范说明的常规约定

用于本文件的 OCTET 这个词系指一个 8 比特的数据对象并且同义于字节这个词。OCTET 的低位被编号为（0）位，而高位则被编号为（7）位，在本文件中所讲的 8 位数（OCTET）之发送与接收均系左到右。

# 第一章： 概述

本文件定义了 Harris 公司的分布网络规约（DNP）应用层 APDU 的格式与服务。

ISO OSI（国际标准化组织开放系统互连）模型规定了七层。国际电工委员会（IEC）规定了一个简化了的模型只包含有物理、数据链路与应用层。它被名之为性能加强了的体系结构（EPA）。本文件定义该 EPA 的第三层或应用层。数据链路层被定义于：Distributed Network Protocol Version 3.00:Data Link Layer(p009-OPD.DL)

Harris Canada Inc 已开发了 DNP 既用于 SCADA 系统也应用于分布式的自动化（DA）系统。主要的焦点已放在这些领域在当前和今后的需要上面。DNP 适用于高可靠，中等速度，和中等吞吐量的应用。该规约高度灵活并且末端开放，不含任何目标的硬件的专用结构。

图 1-1 示出 EPA 的结构以及它如何适配于整个通信系统，如图所示，用户层对应用层只在一个地方有接口，它隐示用户除了对应用层的接口以外无需知道其它单元的通信系。用户层利用应用层向/自一个主站或外站发送/接收完整的 SCADA/DA 的报文消息。

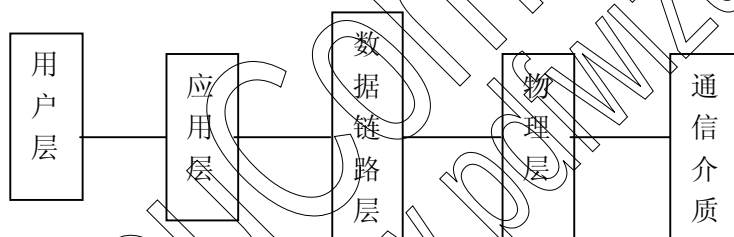


图 1—1EPA 的上下关系

## 1. 1 说明书与 IEC 的关系

DNP 应用层的 APDU 基于 TC57WG03 所拟定的 IEC870-5-3 及 870-5-4 草案文本的原则。在结构上，应用层的 PDU（规约数据单元）适合于 IEC 对 APDU 的描述。用户发送应用程序的用户数据给应用层，并在应用层将它转换为 ASDU（应用服务数据单元）。然而**在 DNP 中，应用程序的用户数据被转换成多个 ASDUS。**IEC870-5-3 规定每个 ASDU 被前置以 APCI（应用程序规约控制信息），然而打包成一个 APDU。在 DNP 内，则每个 APDU（它是多 APDU 的一个部分）被视作一个分段（fragment），且具有以下的限定条件，即每个分段仅包含完的数据对象，以及在同一报文或同一个多 APDU 内的每个分段的 APCI 之功能码是一致的，这就是说，在 APDU<sub>s</sub> 之内不必再信息对象的分割以及在报文中对每个对象都必须

请求同一种操作。这是为了保证每个分段本身就是可处理的，并且也隐示每个 ASDU 仅包含完整的数据对象。反过来谈，应用层收到一个 APDU (一次收一个)，并在那里卸了 APCI，就得到 ASDU，再将多个 ASDUS 组装入应用程序的用户数

DocuCom PDF Trial  
www.pdfwizard.com

## 第二章： 报文格式

本文定义应用层报文（APDU）的格式。APDU 这个词和分段（fragment）是可以互换的。在本规范说明书内主站被定义为发送请求报文的站而外站则为从属设备。被请求回送报文的 RTU 或智能终端（IED）是事先规定了的，在 DNP 内，只有被指定的主站能够发送应用层的请求报文而外站则只能发送应用层的响应报文。

图 2-1 示出了一个主站和一个外站之间应用层报文的顺序。

如图所示，主站将“应用层请求”发送给外站，外站回送“应用层响应”。外站可以决定用“应用层非请求的响应”自发地送发数据。

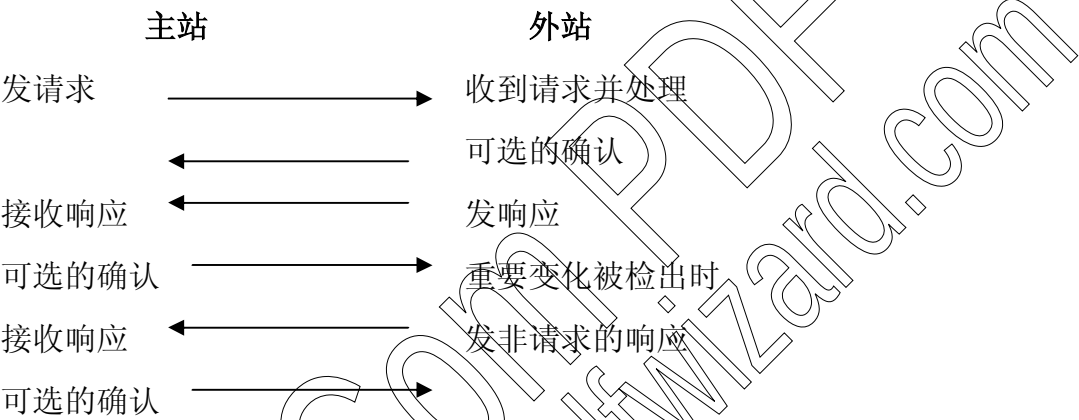


图 2-1 报文顺序

主站对一个外站必须在完成一个请求/响应回合之后才能对此外站发送其它的请求。在请求的回合正在进行之中，主站也可能收到非请求的响应。至于外站，也必需在完成一个请求/响应的回合之后才能接受任何其它请求或发送非请求的响应。非请求的响应只能在请求/响应回合之前或之后发送，而不是地进行时，一个外站若正处于非请求回合之中（即正在等待确认），它可以有条件地自主站接受一个请求令（详情请见 3.3 节—主站请求与非请求响应的冲突）。

此外，每个响应或请求都包含 1 个或更多的单个分段。然而每个分段都应是可领悟的（可解析的），因而是可执行的（因为功能码是属于每个分段的）。对于报文存贮能力有限的设备，建议应只送单个分段的请求报文而所期望的响应（它是全部分段的发送）则多于一个分段。这样是为了保证那些设备可以处理一个请求，并集结起来然而更重要的是在接收下一个请求之前发送一个响应。否则，多分段的报文将要分段的响应，这种响应所要求的报文存贮可能大于设备可用的容量。

## 2. 1 应用请求的格式

应用请求的报文格式示于图 2-2。APDU 由一个包含报文控制信息的 APCI 数据和包含要由接收站处理的信息之 ASDU 所组成。在一个应用请求报文中，APCI 常被称之为请求的(Request Header)。在 DNP 内，ASDU 是可选用的并且当报文的意思不能全部在请求报头中传递时才用得上它。请求报头所包含的信息是如何组装一个多分段报文的信息以及关于报文目的的信息。请求报头出现于所有应用层请求的 APDUS 中。如果请求报头隐含了请求所要求实现的全部信息，则 ASDU 就不存在了。

每个 ASDU 包含一个或多个数据单元的标识符 (DUI) 或对象标题和任选的与之相伴的信息对象 (IO) 或数据段，对象标题可以规定为 0 或多于接收站所回送的或多于报文内报头后所跟随的。

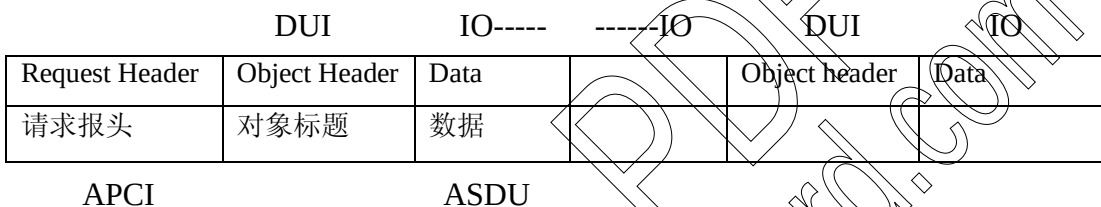


图 2-2 应用请求格式

请求报头：它标识报文的目的并且由 APCI（应用规约控制信息）所组成。

对象标题：它标识后随的数据对象。

数据：在对象标题内所指定类型的数据对象

## 2. 2 应用响应的格式

外站对（应用层的请求）APDU 之响应或发自外站的非请求响应所具的格式如图 2-3。此格式与请求者等同。在应用响应报文中，API 常被称为响应的报头（response header）。响应报头包含了与请求报头同样的信息，又加上一个包含外站内部信息的附加字段。响应报头永远是应用响应的一个部分。响应的 ASDU 具有与请求报文同样的格式，但又有显著的例外（在第三章内说明）。

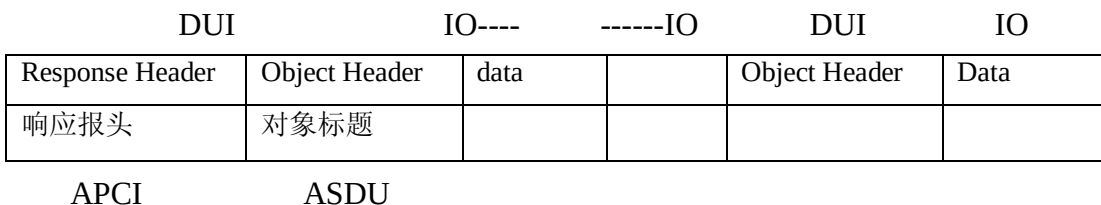


图 2-3 应用响应格式

响应报头：它标识报文的目的并且由 APCI（应用规约控制信息）所组成。

对象标题：它标识后随的数据对象。

数据：为对象标题内所指定类型的数据对象。

DocuCom PDF Trial  
www.pdfwizard.com

## 第三章：DNP 报文字段的定义

本章说明请求与响应的报头（APCIs），它们控制了主站与一个外站之间应用报文的顺序与流向，以及包含 DUI 或对象标题的 ASDU。报头则用以将分段（多 APDU）的报文组装进用户数据（AUD）。将对象标题用作信息对象（任选于报头之后者）的唯一标识。

### 3. 1 应用报头（Application Header）

#### 3. 1. 1 请求报头（Request Header）

请求报头（或 APCI）具有两个字段，每个字段为 8 位的字节，说明如下：

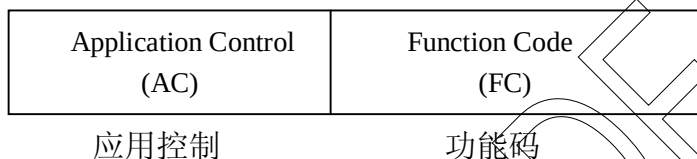


图 3-1 请求报头

#### 3. 1. 2 响应报头（Response Header）

响应报头有三个字段，说明如下：

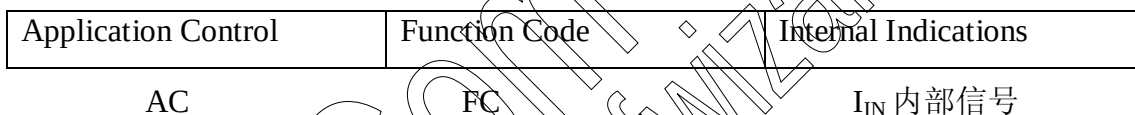


图 3-2 响应报头

#### 3. 1. 3 应用控制

应用控制字段的长度为一个 8 位字节，它提供构造多分段应用报文所需的信息。

应用的报文可以被打成若干包，每个段小到足以装配适于应用报文的缓存。对于分段缓存所推荐的规模为 2048 字节，以便保持与当前 DNP 设备的兼容性，每个分段有一个应用报头和适当的对象标题，以至每个分段都可以作为独立的报文去处理，处理过后应可丢弃掉让出地方来给下一个分段。

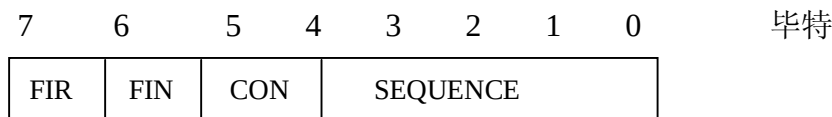


图 3-3 应用控制段

FIR：此位置“1”，表示本报文分段是整个应用报文的第一个分段。

FIN：此位若置“1”，表示本报文分段乃整个应用报文的最后一个分段。

CON：在所收到的报文中此位若置“1”，表示应用报文的发送方期待接收该分段的一

方在收方的应用报文中给予确认。在确认内用一个应用功能码（0）。

**SEQUENCE:** 表示分段的号码，分段号码 0 至 15 是为主站的请求和全部外站的响应保留的（不是非经请求的响应）。分段号码 16 至 31 是为外站来的非经请求的响应保留的。关于非经请求的响应，每个来自外站的连续分段都必需有一个升序的序号（这个号码自 31 溢出后转入 16）。关于对外站的请求和外站的响应（不是非经请求的响应），每个收自或发向同一外站的每个连续的分段也都必须有一个升序的序号（这个号码自 15 溢出到 0）。

注：一个未经请求的响应是外站发生的一个报文，通常它包含着事件数据，它会自动发送给主站，主站无需为这个数而询问外站。

注：建议，外站所报告的任何已变化了的数据在发送时均要求在响应中带有确认。

### 3. 2 通信的流控制

主站与外站之间，请求与响应的信息流由响应与请求的报头以及应用程序的定时器和参数控制着。控制报文流的字段，定时与参数是：

- 1) CON 位：此时的“置位/清除系启用/停用”对报文确认的响应（即 CONFIRM）。
  - 2) FIR 与 FIN 位。
  - 3) 序号字段，这个号码被用以组装多分段的报文以及识别哪一个响应匹配于哪个特定的请求。
  - 4) 主站与外站应用的响应超时，这些超时规定了一个应用对一个响应必需等待多长时间，或对 CONFIRM 响应要等待多长时间方才重发或异常地中断通信的回合。
  - 5) 主站与外站应用的重发计数。应用可以支持也可以不支持应用层的重发，重发计数规定，如果响应失败，请求可重发几次，或者收不到 CONFIRM 响应时可将响应重发多少次。
- 一个外站在开始处理第二个请求之前，必须将前一个请求完全处理好并对它作出响应，它不能同时处理多个请求。

主站发到外站的全部请求其序号为 0-15，发自外站的非经请求和响应其序号则包括在 16 至 31 以内。

以下的规则支配着序号是如何工作的：

序号自 15 滚到 0 或自 31 滚到 16，从 DNP 主站发出的每个连续的请求分段都有一个升序序号，至于请求报文的重发则例外，关于单元分段的请求在重发时其序号是不递增的，至于多分段请求的重发，所重发的请求中第一个分段的序号就是方才失败的请求之最后分段的序号。

对于单分段的请求其单分段响应具有与请求相同的序号。

对一个请求或一个响应的 **CONFIRM** 响应具有与请求或响应相同的序号。

对一个单元分段请求的多分段响应它的第一个分段，具有与请求相同的序号，至于多分段响应的后续分段，其序号是递增的。

对于多分段请求的多分段响应，其第一个分段的序号与多分段请求的最后分段之序号相同。

使用这种顺序号码的方案保证外站和主站对通信网络上报文的失落与延迟的各种情况都能够对付了，以下的规则在外站和主站都是要遵守的：

当一个响应在逾越时限之后没有被收到，如果系统使用的是应用层重发，则该请求将仍用同一序号重发。

如果所收到的两个报文具有相同的序号，通常意味着报文的响应未被对方站收到，在这种情况下，就重发响应（不必要重新处理这个报文）

如果所收到的两个 **CONFIRM** 响应具有相同的序号，则不理睬第二个响应。

以下的几幅图说明了报文传送回合中的几个案例以及如何用序号防止了出问题。在所举的例中，**SEQ** 是序号以及 **CON** 是报文中的请求确认（Confirmation Requested）位。在图中时间自左向右前进。垂直的箭头表示外站与主站间的报文流。

案例一说明典型的报文传送回合，主站发送一个请求，外站予以响应以及主站 **CONFIRM** 这个响应，以后外站发送一个非经请求的响应给主站，当外站在发送响应时，它起动一个监视 **CONFIRM** 响应的定时器，如果在收到 **CONFIRM** 之前已经逾时，则外站应重发这个响应。

案例二示出一个与案一相似的情况，所不同者主站的请求即 **CONFIRM** 响应又要求一个常规的响应。

案例一：

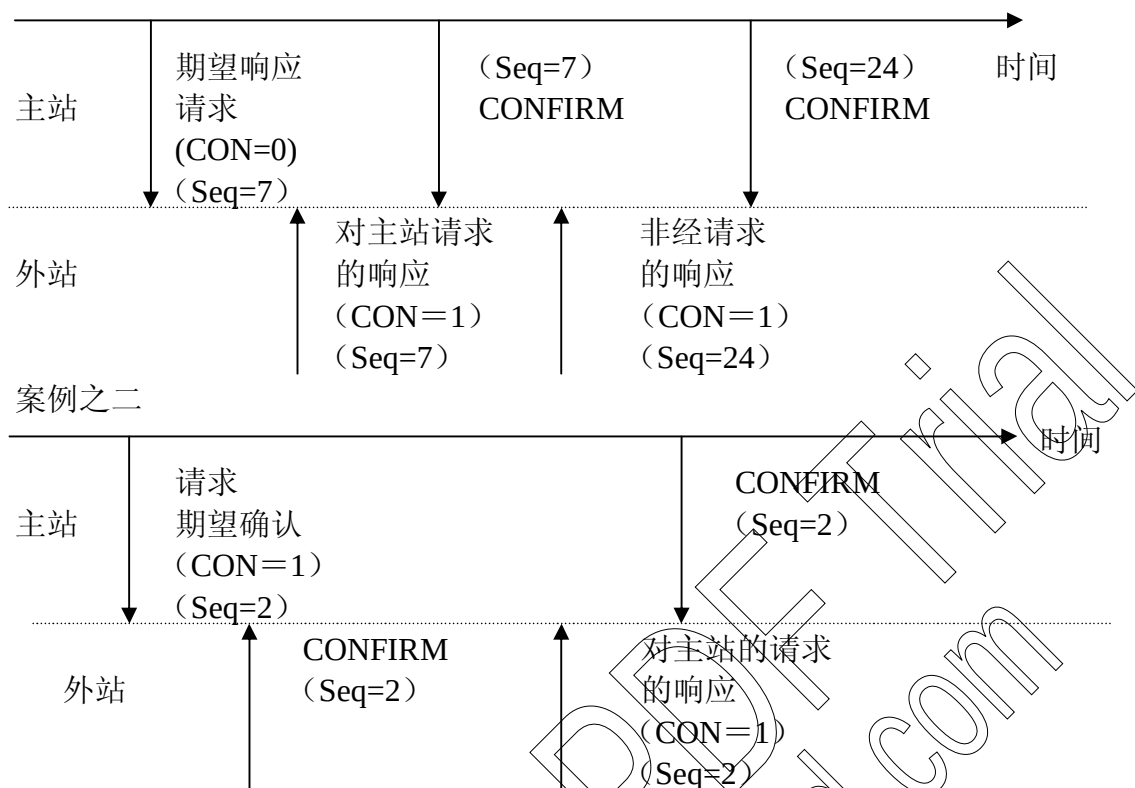


图 3-4 典型报文传送回送的流程图

**注:**

在图 3-4 中以下几个图中, CON 位在外站响应中置位, 在某些传送回合中 (例如在响应内不含事件数据 CON 位可能被清除。在这种情况下, 由于通信的原因丢失了数据往往并不严重。外站就只当响应已传送成功。

案例三说明来自外站的多分段响应, 在连续的分段中, 序号是递增的, 注意主站的下一个请求采用的序号为 4。

案例四外站的响应未被主站收到，外站等待 CONFIRM，当 CONFIRM 监视定时器超时它就重发响应。

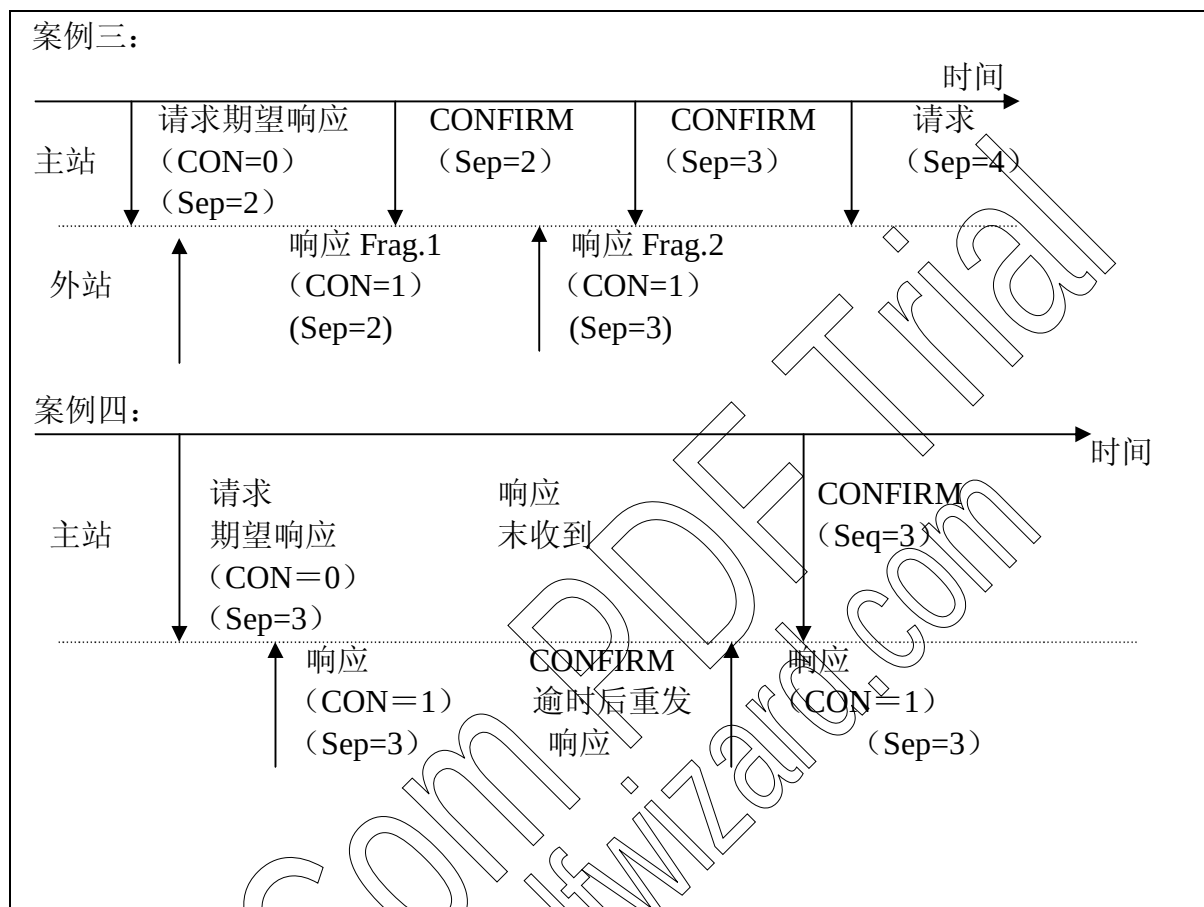
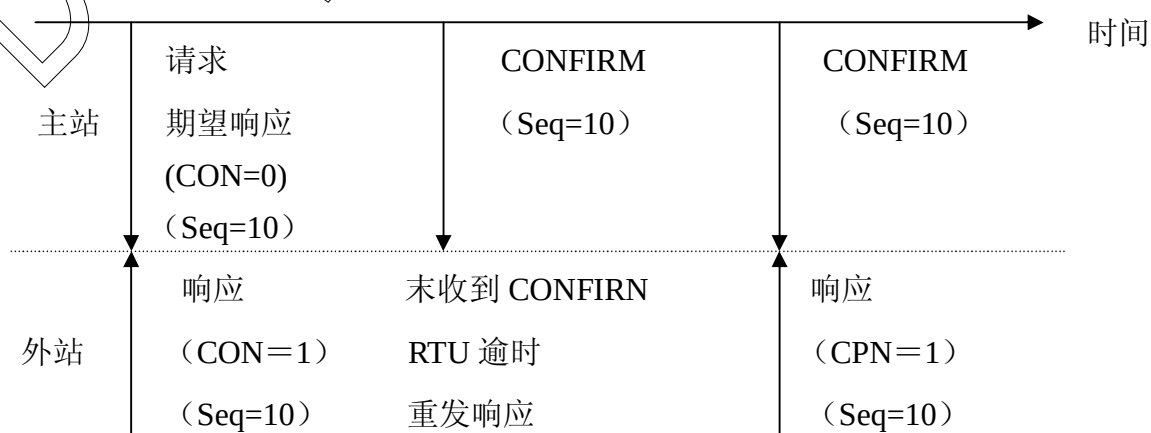


图 3-5 多分段的响应与 RTU 的 CONFIRM 超时

从外站这一边，案例 5 与案例 4 是相同的，在案例 5 中不同与案例之处是主站 CONFIRM 了外站的第一个响应。而外站没有收到此 CONFIRM。当外站重发响应时，主站将重发 CONFIRM。主站将不重新处理第二个响应。

案例五：



案例 6 是主站的请求被外站收，在响应延迟逾时后，主站重发请求。

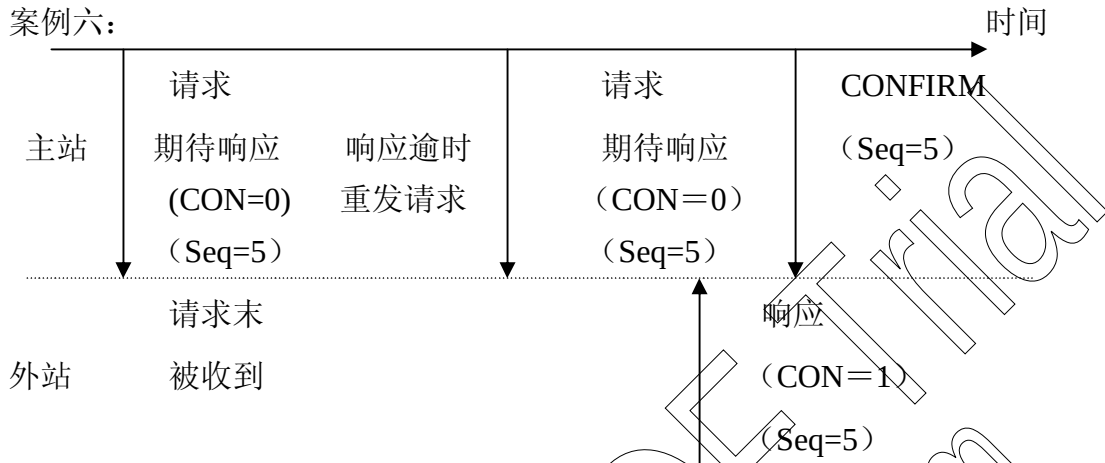
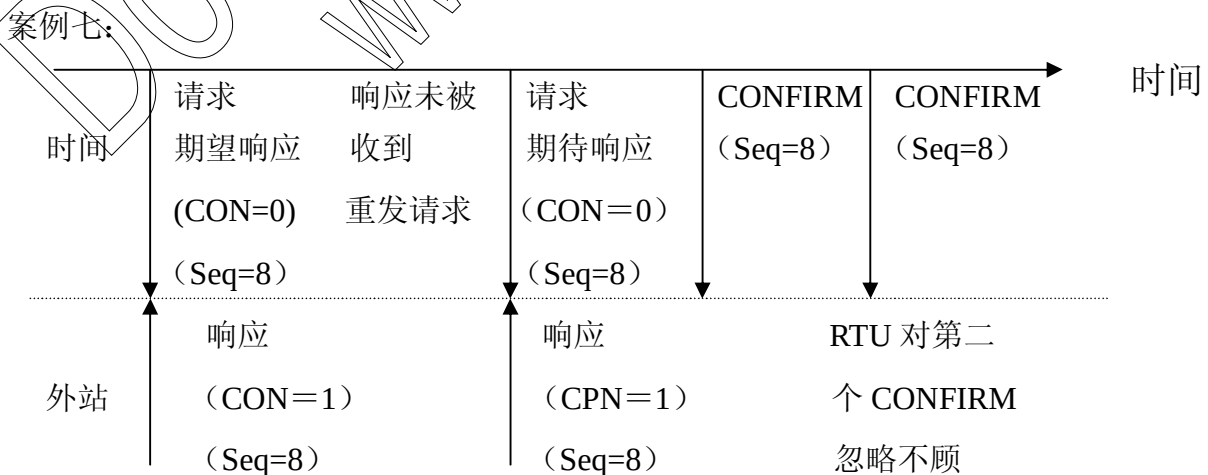


图 3—6 发生响应逾时的报文传送回合

案例 7 与案例 4 相仿，两者中，外站给主站的响应均未被收到，在案例 4 中，外站等待 CONFIRM 逾时并重发响应，然而在案例 7 中，主站先逾时而重发请求。外站自动停止对 CONFIRM 的等待并重发其先前的响应，案例 7 还说明加一种可能的情况。主站未收到的那个原始的响应系在通信网络中被延迟了，主站重发请求，外站回答以及主站以一个 CONFIRM 结束了该回合的顺序，于是外站的原始响应又到达了主站。主站以为第一个 CONFIRM 没有被外站所收到，所以它重发 CONFIRM，外站收到后，就不理会这第二个 CONFIRM。

案例 8 与案例 7 相似，在本案例中，外站的第一个 CONFIRM 在通信网络上被延迟了，当这个 CONFIRM 终于到达主站时，它就被丢弃了。



案例八：

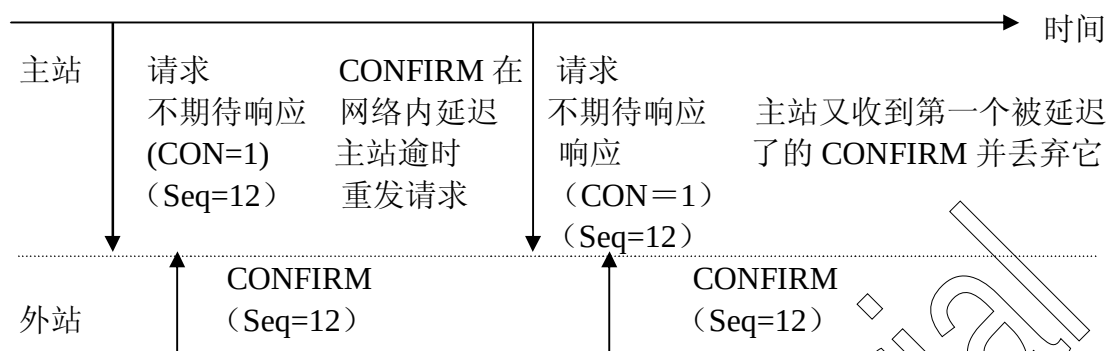


图 3-7 网络响应延迟的信息流程

案例九，外站等待 CONFIRM 超时，外站重发的非经请求的响应，主站最后收到了两次非经请求的响应。它对响应不作第二次处理，但仍如同第一次那样作出的响应。外站丢弃第二个 CONFIRM。这说明这样一种情况，网络延迟虽未丢失报文但会导致超时重发。

案例九：（Unsol.Rsep=非以请求的响应）



图 3-8 由于网络延迟重发非请求的响应

### 3.3 主站的请求与非请求的响应之冲突

当外站发生了非请求响应时，有一种可能是在外站发送非请求响应的同时，主站也发送请求。当外站期待收到对其非请求响应的 CONFIRM 时却收到了主站的请求，主站正期待收到其请求的响应时却收到了非请求的响应。

对上述情况以及类似情况的处理取决于主站所发出的请求之类型。

主站永远将立即处理非请求响应，即使主站正在期待先前所发请求之响应时也要立即处理非请求的响应。如果外站要求确认（即，CON 位置位），主站就要立即发出对非请求响应之 CONFIRM

通常外站将立即处理请求，即使它正期待着先前非请求响应之 CONFIRM。除了对系统数据（例如二进制输入数据，事件计数数据等等之外）的 READ 请求之外，所有的请求都按此处理。这种操作模式被称作立即处理（IMMEDIATE-PROCESS MODE）模式。另一种办法是外站若正在等待先前非请求响应之 CONFIRM 时，它就不处理主站来的 READ 请求。在处理请求之前，将先等待 CONFIRM，这种功能性的变异，其目的在于防止事件之丢失或重复，这种操作的模式被称之为 PROCES-AFTER-CONFIRM（确认后处理模式）。

### 3.3.1 （立即处理）模式

图 3—9 和图 3—10 说明当主站的请求与外站非请求响应被同时发送时，且外站处 IMMEDIATE\_PROCESS 模式（即所请求的不是一个 READ 请求）。时的功能性。

在案例 10 中，主站立即响应于非请求的响应。外站立即处理并对主站的请求作出响应。注意两个 CONFIRM 响应自主站发送的次序可以与案例 10 中所示者相反。这不会使外站感到困扰的。

案例 11 说明不要求一个 CONFIRM 的非请求响应之基本报文流。

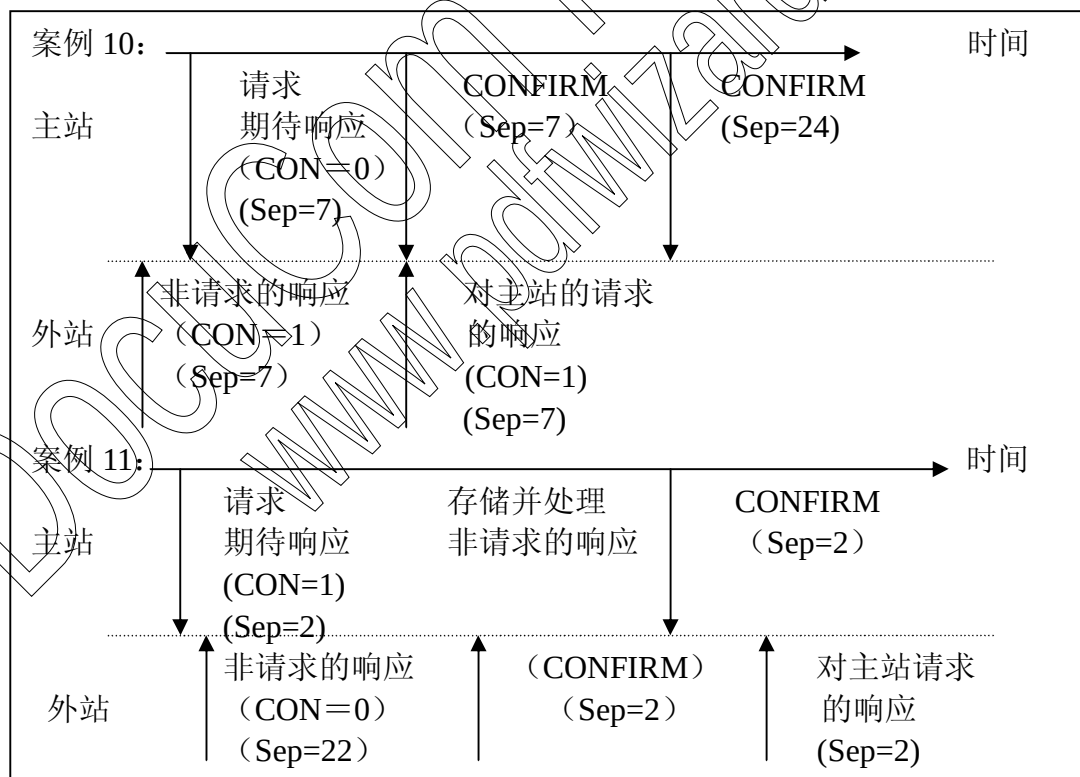
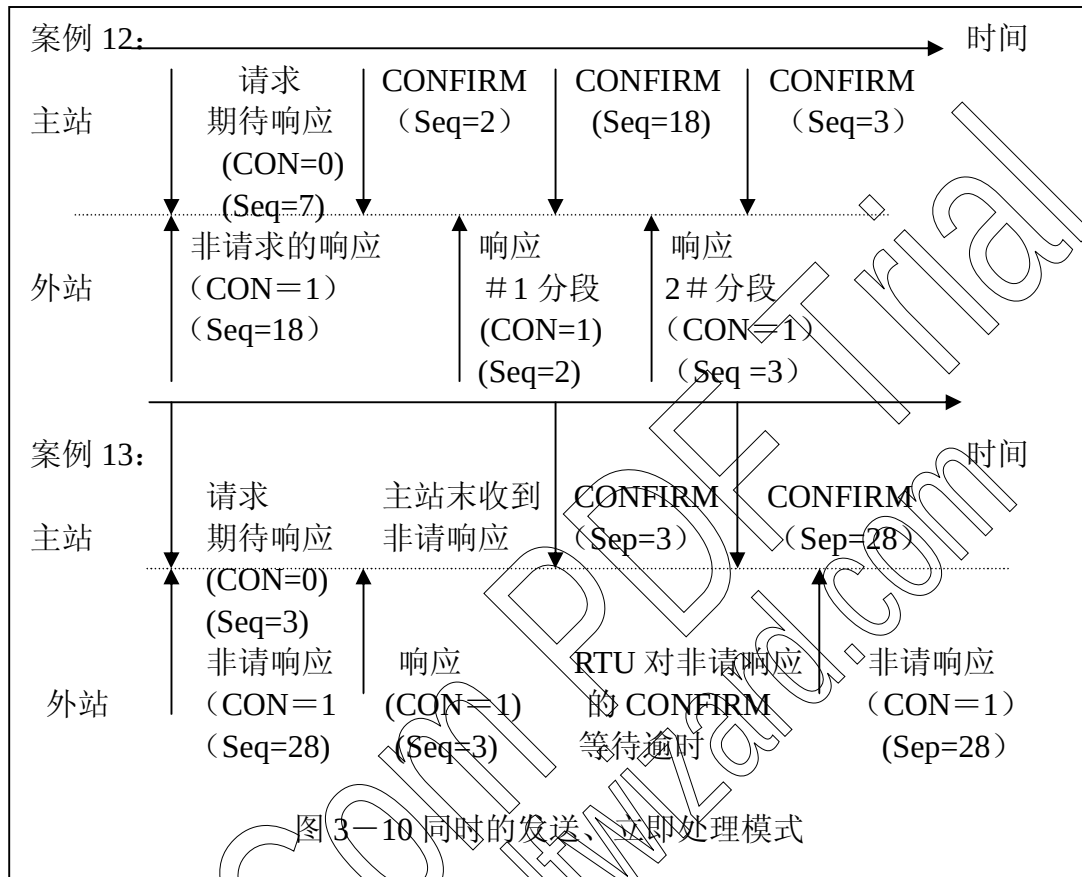


图 3—9 同时的发送、立即处理模式

案例 12 中，非请求响应的 CONFIRM 正处在多分段响应中对两个分段的 CONFIRM 之间。



案例 13 中说明：然后在非请求响应等待 CONFIRM 的定时器逾时后重发非请求响应。于是主站再 CONFIRM 此非请求响应。

注意，有可能第一个非请求响应会迟后到达主站（网络中的延迟），主站不会再处理此响应，但会再次回答，外站将丢弃此加答。

### 3. 3. 2 PROCESS-AFTER\_CONFIRM（确认后处理）模式

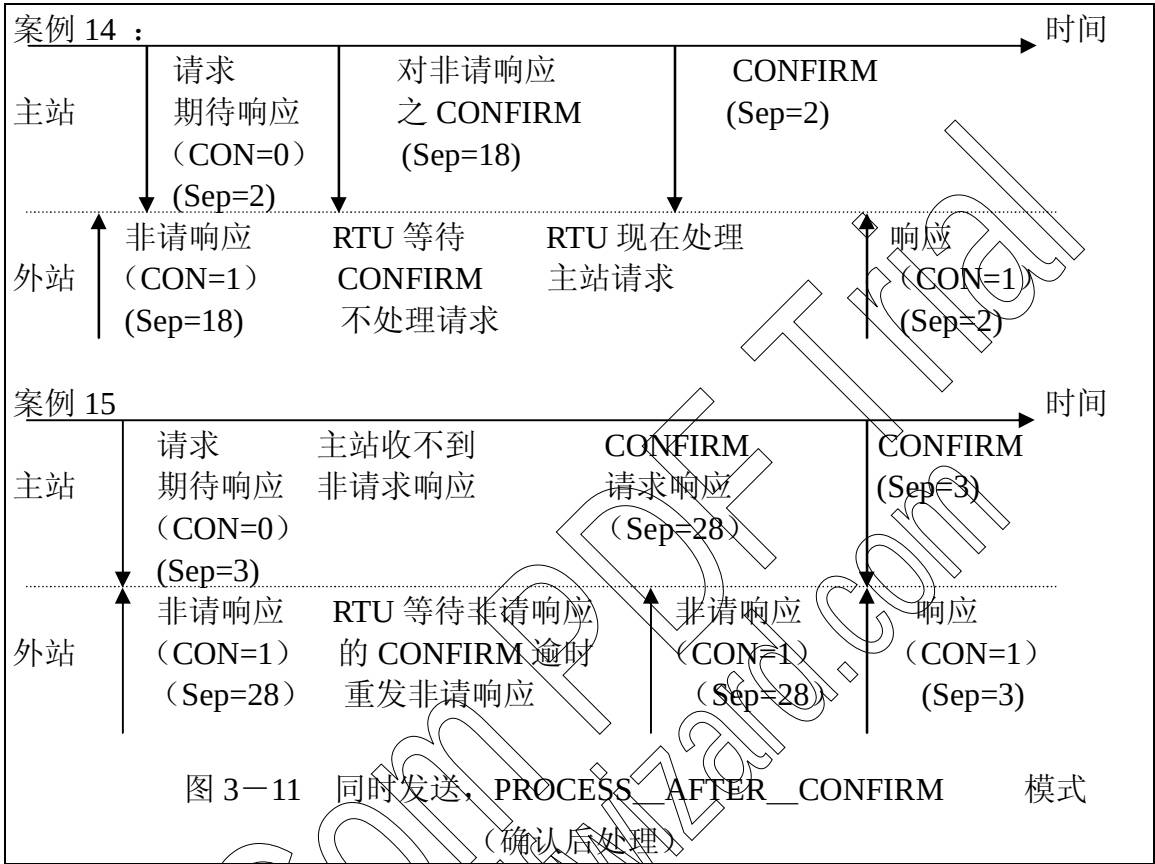
当外站收到一个对系统数据的 READ 请求时，并且先前一个非请求响应时，外站将不处理 READ 请求，直到非请求响应获得 CONFIRM 为止，如果外站立即处理了 READ 请求，就要承担失落数据或数据重复的危险，这是因为有可能 READ 请求所要的数据对象已经尚未 CONFIRM 的非请求响应之中了。

案例 14 说明 READ 请求已被收到，而外站正在等待 CONFIRM 之时的情况，外站不处理 READ 请求，直到收到 CONFIRM 之后。

案例 15 与 14 相似，但非请求响应未被 CONFIRM，外站必须重发非请求响应，直到被 CONFIRM 或者达到了所组态的重发次数极限为止，如果已达到此极限，则外站将在内部

重新缓存此数据于非请求响应，响应任何尚未解决的请求，然后再试送非请响应。

3. 4 出错后的恢复



DNP 应用层依靠数据链路层作所有报文的发送、接收与检错，应用层不负责通信问题的恢复。如果一个通信回合的故障被数据链路层报出，应用层中止应用层的通信回合并向用户报告出错。此外，主站应用层应指示出全部站响应内的内部信号值。用户层负责起动任何一种出错的恢复步骤。用户层特别应利用自任何站响应返送回来的内部信号 (LIN)。

3. 5 功能码

功能码标识着报文的目的，这个字段的长为一个 8 位字节，有两组功能码，一个用于请求，另一个用于响应。

| 代码 | 功能 | 说明 |
|----|----|----|
|----|----|----|

|           |            |                                                                  |
|-----------|------------|------------------------------------------------------------------|
| 传输 功能码    |            |                                                                  |
| 0         | 确认         | 用于请求与响应报文分段之确认，对此报文不要求响应                                         |
| 1         | 读          | 请外站送指定的对象；以所请的对象（可用的对象）作响应                                       |
| 2         | 写          | 向外站存入指定的对象；以操作的状态作响应                                             |
| 控制 功能码    |            |                                                                  |
| 3         | 选择         | 选择或打开输出点但不置或产生任何输出作用（控制，设置数值，模拟输出）；以所选控制点的状态作响应，“操作”功能码被用以激活这些输出 |
| 4         | 操作         | 对于由“选择”功能所已选定的点设置或产生输出动作；用控制点的状态作为响应。                            |
| 5         | 直接操作       | 选择并设置或操作指定的输出，用控制点的状态作响应                                         |
| 6         | 直接操作无确认    | 选择并设置或操作指定的输出，但不发送响应给请求方。                                        |
| 冻结 功能码    |            |                                                                  |
| 7         | 立即冻结       | 复制指定的对象于一个冻结缓存，并以操作的状态作响应。                                       |
| 8         | 立即冻结-无确认   | 复制指定的对象于一个冻结缓存，不用报文作响应                                           |
| 传输功能码     |            |                                                                  |
| 9         | 冻结与清除      | 复制指定的对象于一个冻结缓存，然后清除对象；用操作的状态作响应                                  |
| 10        | 冻结与清除无确认   | 复制指定的对象于一个冻结缓存，然后清除对象，不用报文去响应                                    |
| 11        | 有时间的冻结-无确认 | 在指定的时间和时间间隔上，复制指定的对象于一个冻结缓存，用操作的状态作响应。                           |
| 12        | 有时间的冻结-无确认 | 复制指定的对象于冻结缓存，在指定的时间与时间间隔上；不用报文去响应。                               |
| 应用 的控制功能码 |            |                                                                  |
| 13        | 冷再启动       | 实现所要求的复位顺序，以指示时间的时间对象作响应，直到外站可以工作为止。                             |
| 14        | 暖再启动       | 实现所要求的部分复位顺序；以批示时间的时间对象作响应，直到外站可以工作为止。                           |
| 15        | 将数据初始化到缺省值 | 将指定的数据初始化到上电的初始值；以操作的状态作响应。                                      |
| 16        | 初始化应用程序    | 使指定的应用程序之运行准备就绪；以操作的状态作响应。                                       |
| 17        | 启动应用程序     | 启动指定的应用程序之运行，以操作的状态作响应。                                          |
| 代码        | 功 能        | 说 明                                                              |

|               |        |                                                                        |
|---------------|--------|------------------------------------------------------------------------|
| 18            | 停止应用程序 | 停止指定的应用程序，以操作的状态作响应。                                                   |
| 19            | 保存组态   | 将指定的组态数据存入非易失的存储器，用一个批示时间的<br>时间对象作响应，直到外站可用为止。                        |
| 20            | 允许非请报文 | 允许指定的数据对象自发地报告，以操作的状态作响应。                                              |
| 传输 功能码        |        |                                                                        |
| 21            | 停用非请报文 | 停用指定数据对象的自发报告，以操作的状态作响应。                                               |
| 22            | 分配等级   | 将指定的数据对象分配给特定的等级（class）                                                |
| 时间 同步功能码      |        |                                                                        |
| 23            | 延迟测量   | 允许应用程序为一个特定的外站计算其通路的延迟（或传播<br>延迟）。由此功能码所算出的数值应被用以在设置外站时间<br>时，调节日期的时间。 |
| 保留的           |        |                                                                        |
| 自 24<br>至 120 |        | 保留供今后之用                                                                |
| 121 至<br>128  |        | 仅保留供测试之用                                                               |
| 响应功能码         |        |                                                                        |
| 0             | 确认     | 对报文分段的确认，既用于请求也用于响应。<br>该报文不求响应                                        |
| 129           | 响应     | 响应于一个请求的报文                                                             |
| 130           | 非请响应   | 非由请求提示的非请响应。                                                           |

表 3 — 1

### 3. 6 内部信号（Internal Indications）

内部信号（IIN）字段是一个有两个字节（16位）的字段，它在所有的响应中跟在功能码之后。当一个请求，由于格式出错或所请求的数据无效而不能处理时，IIN 总能以适当的置位作回报。

|      |      |   |   |   |   |   |   |   |   |
|------|------|---|---|---|---|---|---|---|---|
| 第一字节 | 第二字节 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|------|------|---|---|---|---|---|---|---|---|

第一字节的每一位置位表示下面所述的状态。

Bit 0 \*所有外站的报文已收到。

\*当收到带有全部站目的址（ffffH）的请求时，则置位。

\*在下一个响应后被清除（即使是要求对全局请求的响应）。

\*用以让主站知道本站收到了广播报文。

Bit 1 \*1 级（class）数据有效可用。

\*当已被组态为 1 级的数据正准备发送给主站时，则置位。

\*当此位在响应中置位时，主站应向外站请求此级数据。

Bit 2 \*2 级数据有效可用。

\*当已被组态为 2 级的数据正准备发送给主站时，则置位。

\*当此位在响应中置位时，主站应向外站请求此级数据。

Bit 3 \*3 级数据有效可用。

\*当已被组态为 3 级的数据准备向主站发送时，则置位。

\*当此位在响应中置位时，主站应向外站请求此级数据。

Bit 4 \*向主站要求作时间整步。主站以向外站写入“时间与日期”作时间的整步。

\*当时间被主站设定后，这位就被清除，当主站直率地对外站信号对象的这一位写入“0”时，它也被清除。

Bit 5 \*当某些或全部外站的数字输出点处于本地状态时，这位就置位。亦即，外站的控制点不能通过 DNP 规约访问了。

\*当外站处于远方态，它就被清除，亦即外站的控制输出可通过 DNP 规约访问了。

Bit 6 设备有毛病

\*外站有异常情况时，它被置位，对于一台给定的设备有一份简要说明讲述影响这一位的条件。

\*仅当不能使一个或几个其它的 IIN 位的组合来描述这个状态时，才该使用这位。

Bit 7 \*设备再启动。

\*当用户应用程序在外站再启动时它被置位。

\*当主站直率地向外站的内部信号对象的这一位写“0”时，此位即被清除。

第二字节

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

Bit 0 \*功能码未被完成。

Bit 1 \*被请的对象是未知的，外站没有这个被指定的对象或者没有对象被分配在所指定的级中。

\*该信号应该被用于查找故障之用并通常用以指示设备说明（Profile）或组态问题之失配。

- Bit 2 \*在限定词，变程或数据场内的参数失效或越出变程，这是对应用请求格式出借的一个总管信号。  
\*这个信号应该被用于查找故障之用并且通常指示组态的问题。
- Bit 3 \*事件缓存，或其它应用的缓存已溢出了。例如 COS/SOE 的缓存已溢出。  
\*主机应该努力去恢复尽可能多的数据并向用户指出有可能丢失了数据。  
用户应起动适当的出错恢复过程。
- Bit 4 \*请求已被理解，且所请求的操作已在执行。
- Bit 5 \*置位以表明当前外站内的组态已崩溃以及主站应用层应将此种异常通知用户，主站可将另一套组态数据下装至外站。注意，有时一个崩溃的组态会使一个外站不能工作，使它不能将这种情况以通信告知主站。
- Bit 6 保留按协议使用，当前始终置“0”回送。
- Bit 7 保留按协议使用，当前始终置“0”回送。

3. 7 对象标题（Object Header）

报文的对象标题指定包含在报文中的数据对象（或 I/O）或是被用以响应此报文的数据对象（或 I/O）。在请求与响应中对象标题的格式是相同的，但对标题的解释取决于它是请求还是响应以及与标题伴随的功能码。

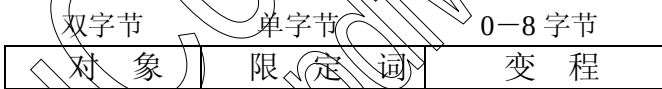


图 3-13 对象标题

对象：指定对象组以及跟在标题后面的对象之变化，如第 3 章所述。这是一个 2 字节段。  
对象段唯一地标识了对象的类（Class）与型（Type），它给定了数据对象的结构（因而也决定了其规模）。

限定词：规定了变程段的意义，如第 2 章所述，这是一个单字节段，这个限定词规定变程段该如何解释。

变程：表明对象的量，起点与终点的指标或所讨论对象的识别等，如第 3 章所述。本段唯一地标识出所关心的对象。

注：如果限定词段指出没有变程段，则变程段也可以不存在，这个字段的规则可自零变到八个字节。

3. 7. 1 对象段（Object Field）

对象段规定一个对象组内的对象变体（variation）。对象的组别与变体结合起来可以唯

一地规定报文所指定的对象。当前定义对象的结构如 DNPV3.00 Data Object Library(p009—OBL)所述。

对象可被分配于四级之一。当对象段批定一个对象级而不是一个指定的对象型时，对象段间接地指所有分配于此级的数据对象而不是指任何指定的对象的形式，见第 5 章：(CLASSES)。

对象段为两字节长。第一个八位字节指定数据的基本型式（例如模拟输入），第二个八位字节指定数据型式和变体（例如 6 位模拟输入或是 2 位模拟输入）。在请求方，若对象变体被指定为零，这表明所有的对象变体属于同一组内（即全部的或任何的模拟输入型）。然而在响应方，变体 0 不能被用来指定对象，必需指明特定的变体。

设想一个例子，请求报文在第一个字节中指定为计数对象以及其变体为 0，已知外站仅持 16 毕特的计数器，它将以一个对象标题去响应，标题变体段被用以表示该计数对象为 16 毕特的计数器。

用同样的请求送给另一个站，它回送的对象标题可以表明是一个 32 位的计数段。根据请求的数据所带的变体为 0。主站就不必要了解外站支持何种变体了。然而主站必须能解释对象标题并具有每个变体结构的一些知识。

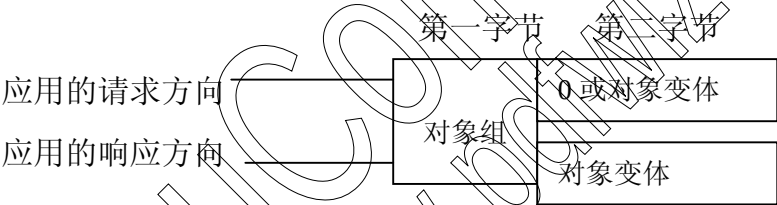


图 3—14 对象段

3. 7. 2 限定词段 (Qualifier Field)

限定词段规定了变程段的意义。

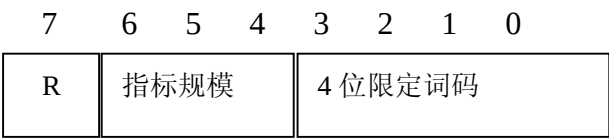


图 3—15 限定词段

R: -保留位始终为 0

变程段被用以索引数据或者作为一个识别符。变程段的使用与结构取决于在指标规模 (Index Size)段和限定词码 (Qualifier Code)段内的数值。当变程段被用以索引数据时，它常包

含有起始变程值和终点变程值。它们合起来为对象标题后随的数据定义一个对象的变程。

每个起始变程和停止变程的分段被称为指标 (**index**)。

## 1、Index Size (3—毕特): (指标规模)

\*当 Qualifier 码为 11 时, 在一个请求的对象标题内之(Index Size)

仅当 Qualifier 码为 11 时, Index Size 的右方才有效, 这些比特指明在 Range 段内每个条目之规模有几个字节。Range 段跟在 Qualifier 段之后。Range 段由一些指标 (去规定对象的一个 Range) 或由对象识别符的清单所组成 (见 Qualifier 码 11)。

0=对于 Qualifier 码 11 无效

1= (identifier) 标识码的规模为 1 字节

2= (identifier) 标识码的规模为 2 字节

3= (identifier) 标识码的规模为 3 字节

4 至 7=保留

\*在包含数据的请求或响应标题中, 三毕特的 Index Size 段规定前置于每个对象的指标规模或对象的规模。

0=对象们被压缩了, 它们未前置以 Index

1=对象们被前置以一个字节的 Index

2=对象们被前置以两个字节的 Index

3=对象被前置以 4 个字节的 Index

4=对象被前置以 1 个字节的对象规模 (object Size)

5=对象被前置以 2 个字节的对象规模 (object Size)

6=对象被前置以 4 个字节的对象规模 (object Size)

7=保留

## 2、限定词代码 (Qualifier Code) (4 毕特)

Qualifier 码被用以规定 Range 段的意义。

\*在 Range 段内的 Start 与 Stop 分段 (Sub—field)

跟在 Qualifier 段后面的 Range 段常包含着若干分段 (Start Range 和 Stop Range) 这些分段用数字指明自 Start Range (包括 Start Range 的数) 至 Stop Range (包括 Stop Range 的数) 的一个整数值的变程。

关于 Qualifier 码 0, 1 和 2, Start Range 和 Stop Range 被解释作为虚拟存贮地址。

在无数据的报文中, IndexSize 段应为 0, 以表明再没有 indexing 了。在有数据对象的

报文中 IndexSize 段可以是 4、5 或 6 以指明每个数据对象(由 Range 段规定 Indices 的 Qualifier 码, 不能使用 Index 规模值 1、2 与 3。

几个 Qualifier 码的定义为:

1=在变程段内有 16 毕特的 Start 与 Stop indices

2=在变程段内有 32 毕特的 Start 与 Stop indices

3=在变程段内有 8 毕特绝对址的识别符 (Identifier)

4=在变程段内有 16 毕特绝对址的识别符 (Identifier)

5=在变程段内有 32 毕特绝对址的识别符 (Identifier)

#### **\*给定对象形式之全部对象**

当 Qualifier 码为 6, 则 Range 段的长为 0 (即无变程段) 因为所指的是给定型式之全部数据对象。这种 Qualifier 可用于带有对象标题的报文中, 仅因为它不能唯一地标识报文中如若存在的数据对象。当使用该 Qualifier 码时, Index 的规模应置于 0。

Qualifier 码 6=无 Range 段 (隐示全部指定的对象)。

#### **\*单个段的数量 (Single field quantity)**

Qualifier 码之 7, 8 和 9 被用以说明 Range 段系由一个计数值所组成, 它指明所论及的数据对象之数目。

如果 Index 规模等于零, 则 Range 段的数值减 1 为止。

若 Index 规模为 1, 2 或 3, 则 Range 段规定了跟在 Range 段后面的 indices 与对象的数目。

Qualifier 码 7, 8 与可用于请求和响应的报文, 在一个报文中, 不论有无数据对象, Range 段的数值规定了所援引的数据对象之数目。其 Index Size 段则应设置 indices 的规模, 这种 indices 或是前置于每个数据对象 (关于有数据对象的报文) 或是构成一份 identifiers 的顺序清单。

Index size 段不必指明一个对象规模的标识码 (identifier), 因为这并不唯一地规定了正在谈论的数据对象, 并应将它置 0 如若其后并无 identifier 或 indices。Identifiers (以及可选用的数据对象) 之次序是任意的但不可由重复的 indices 所组成。

几个 Qualifier 码的定义为:

7=有单段为 8 毕特的数量值

8=有单段为 16 毕特的数量值

9=有单段为 32 毕特的数量值

### **\*自由格式的 Qualifier (Qualifier 码 11)**

当其它的 Qualifier 码都不适用时或不能提供足够的标识信息时，就用这个标识码去规定对象。

仅当 Range 段 (Index) 不能唯一地规定所谈论的数据对象时，才用 Qualifier 11。在这种情况下，Qualifier 码定义一个包含对象标识码的长度可变的字节数组 (串)。这种标识具有一种自由格式并且此时应用层已无论如何不能解释了。

Range 段总是一个单字节的值 (计数值)，它规定了对象标识码的数目。在 Range 段之后跟着一对一对的 Count 对象之规模/对象标识码。标识码的规模 (以字节计) 取决于前置在每个标识码前面的对象规模段。对象规模段的规模则取决于 Index size。要用 Index size 4, 5, 6 去规定对象规模段的规模。

### **\*保留的 Qualifier 码**

以下的 Qualifier Code 是保留的，不要去用它：

10=保留

12=保留

13=保留

14=保留

15=-保留

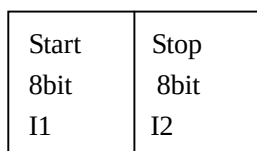
### **3. 7. 3 Range**

Range 段的意义由 Qualifier 段所规定的，对于 Qualifier 码 0 至 5，Range 段具有两个分段规定了一个起动一个停止的 Index 或地址。这些段的数值是全部包含的 (inclusice)。对于 Qualifier 码 6，Range 段是不出现的，关于 Qualifier 码 7, 8, 9 及 11，Range 段是规定一个数量的单个段。在以下，用 “Q—码” 这个词去指 4 毕特的 Qualifier 码段而以 “I—规模” 去指 Index 规模段。

以下图为不包含任何 IO 信息对象或数据对象的并且只不过规定所讨论的对象的请求或响应，定义全部有效的 Qualifier/Range/Index 之组合，所描述的字节出现在对象标题的 Qualifier 字节之后和下一个对象标题或报文尾的前面。

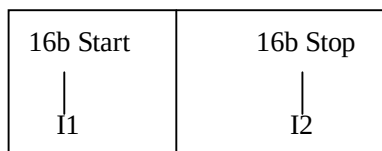
用 Indices 的 Range 规定已知诸点的请求 (Request)

- 用 Q 码之 0—5 描述与顺序有关的点，则其 Range 段为：



Q—码 0 和 3;      I—Size 必为 0  
Range 段的定义

LSB      MSB    LSB    MSB

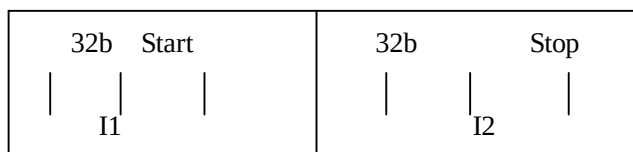


I1 至 I2 之间的点全部包括

0-码 1 和 4;      I-Size 必为 0

I1 和 I2 的点全部包括

LSb                                  MSB    LSB                                  MSB

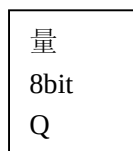


Q 码 2 和 5; I—Size 必为 0

I1 至 I2 的点全部包括

- 用 Q—码 6 以描述给定数据型式的全部点;  
(用这个 Q—码就没有 **range** 段或 **indices**。)

- 用 Q—码 7—9 以描述几个不相关的点:



Q—码 7; I—Size 必为 0

包含 0.....Q-1 点



Index

I1

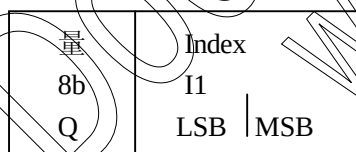
.....

Index

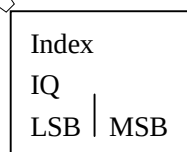
IQ

Q—码 7; I—Size 必为 1

包含 I1, I2, .....IQ 点

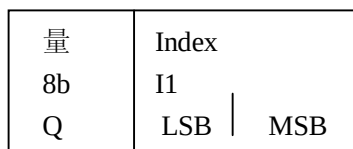


.....

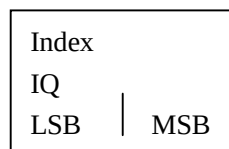


Q—码 7; I—Size 必为 2

包含 I1, I2, .....IQ 点



.....



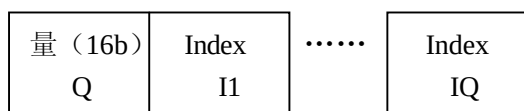
Q—码 7; I—Size 必为 3

包含 I1, I2, .....IQ 点

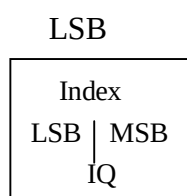
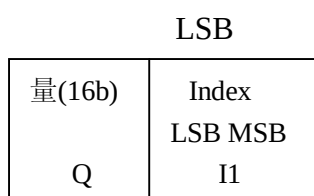
LSB    MSB



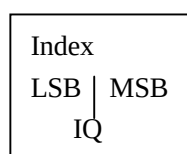
Q—码 8; I—Size 必为 0  
包含 Q 至 Q-1 点



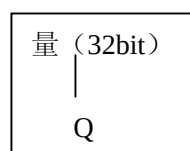
Q—码 8; I—Size 必为 1  
包含 I1, I2, .....IQ



Q—码 8; I—Size 必为 2  
包含 I1, I2, .....IQ 点

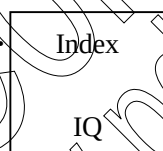
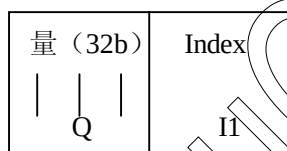


Q—码 8; I—Size 必为 3  
包含 I1, I2, .....IQ 点

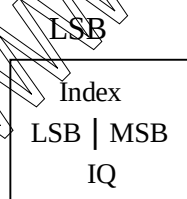
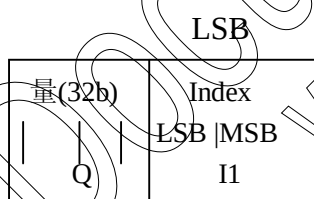


Q—码 9; Insize 必为 0  
包含 0.....(Q-1)点

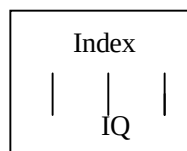
LSB MSB



Q—码 9; I—Size 必为 1  
包含 I1, I2, .....IQ 点

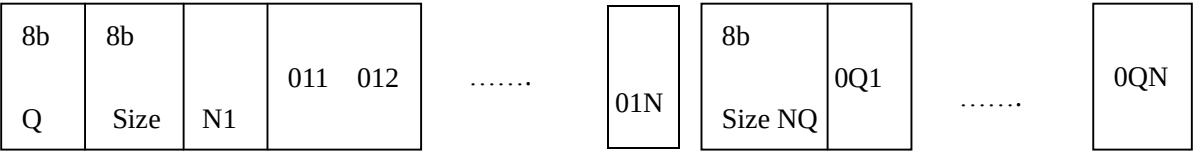


Q—码 9; I—Size 必为 2  
包含 I1, I2, .....IQ 点



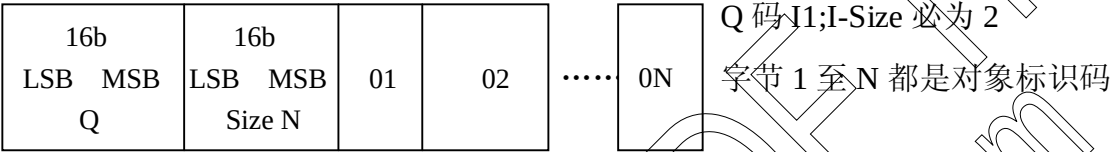
Q—码 9; I—Size 必为 3  
包含 I1, I2, IQ 点

当所描述的点需要用—个对象标识码作唯一的标识时，例如—个文件对象标识码或组态的报头，就要 Qralifier 11,标识码的型式隐含于对象型式。



Q-码 11; index Size 必为 1  
 字节 Oi1, .....OiN 形成对象 I 标识码, 其  $\Phi \leq I < Q$ .

应有  $O\Phi 1.....O\Phi N.....O(\Phi-1)1.....O(\Phi-1)N$



如同以前一例, 可以有许多的 identifiers, 每个跟在另一个的后面。



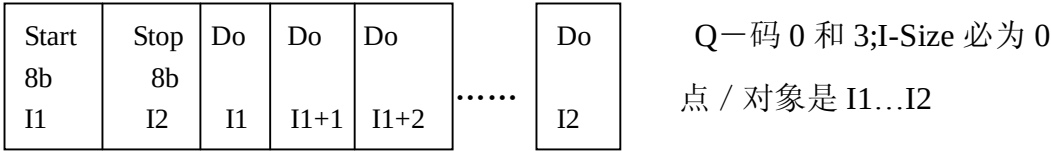
图 3—16 无数据对象的报文

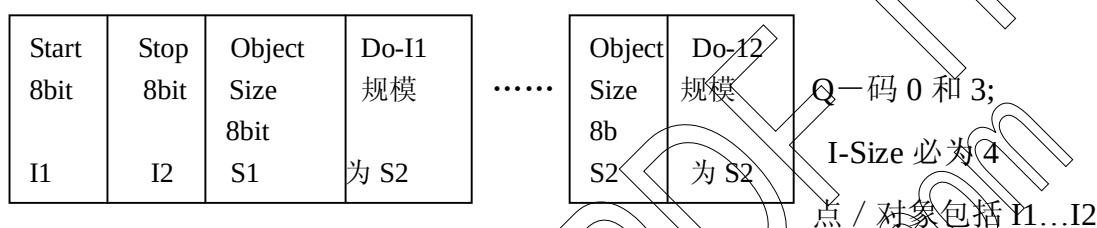
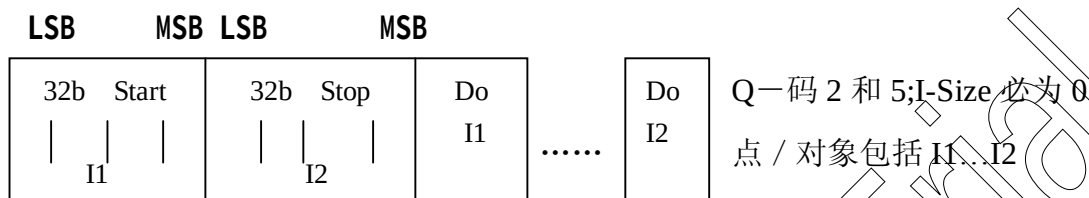
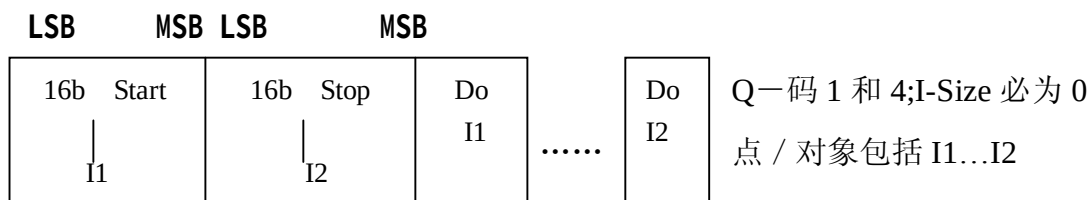
(按: 3.7.2 之自由格式 **Qualifier(Qualifier)** 一节中说的 **Range Field** 总是一字节的值(计数值)但这里的 **Q** 却为何是多字节的? 似应为一字节的 **Q**。

以下的图中为(包含 **IO (Information Object)** 或数据对象的)请求或响应定义了全部 **qualifier/range/index** 之组合, 并且规定了所述的对象, 这些对象或是跟在 **qualifier/range** 段(关于无前置 **indices** 的 **qualifier**)后面或是跟在每一个名别识别的指标 (**identifying index**)之后, 所描述的字节出现在对象标题 (**object header**)的 **qualifier** 字节之后和下一个对象标题之前或报文末尾之前。

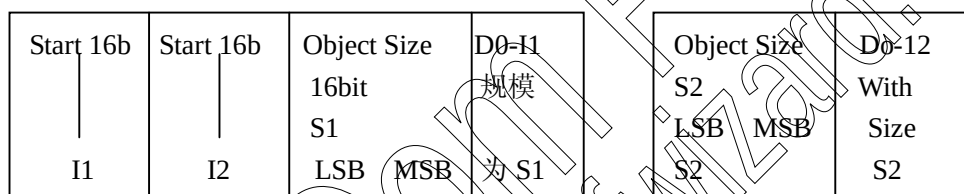
带有已知点(这些点是由指标变程 (**range of indices**) 所规定的)的请求 / 响应。

用 **Q—码 0—5** 描述与顺序有关的点: (**D0**=数据对象)





注意：采用 I-Size 和 6 也能用 16 毕特和 32 毕特的对象规模（Object Size）



Q 码 1 和 4; I-Size 必为 5 点 / 对象包含 I1...I2

注意：采用 I-Size 4 和 6 也能用 8 和 32 毕特的 Object Size.



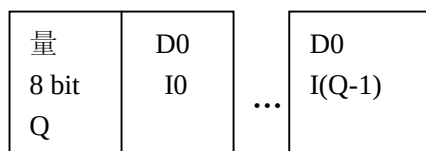
Q 码 2 和 5; I-Size 必为 6 点 / 对象包括 I1...I2

注意：采用 I-Size 4 和 5 也能用 8 和 16 毕特的 object size

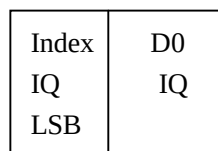
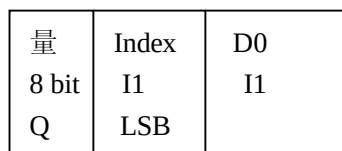
特别注意!!!

对于所包含的数据对象其确切点数尚不明白电文，不可用 Q-码 6 去描述此报文，因为报文的内容尚不能判知。

用 Q-码 7 和 9 描述一些不相关的点：

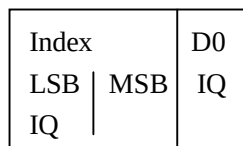
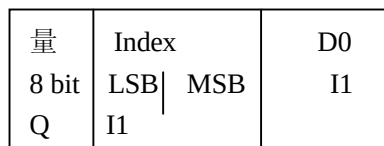


Q 码 7; I-Size 必为 0  
点 / 对象包括 I0.... (Q-1)



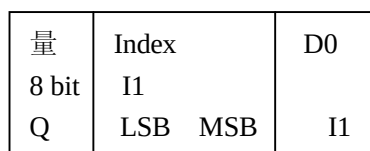
Q 码 7; I-Size 必为 1

点 / 对象包括 I1, I2....IQ

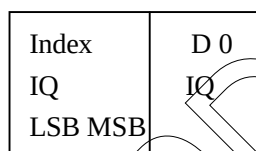


Q 码 7; I-Size 必为 2

点 / 对象包括 I1, I2....IQ



...



Q 码 7; I-Size 必为 3

点 / 对象包括 I1, I2....IQ

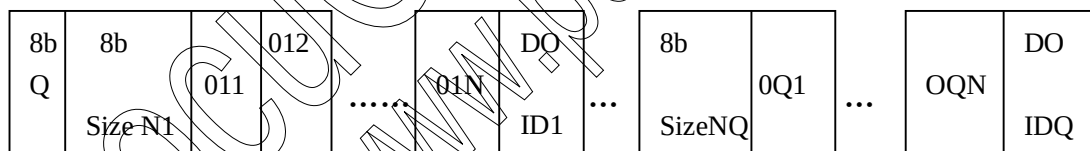
Q-码 8; I-Size 0, 1, 2, 3

与 Q-码 7; I-Size 0, 1, 2, 3 基本相同, 唯数量段 (Quantity) 变为 16bit。

Q-码 9; I-Size 0, 1, 2, 3

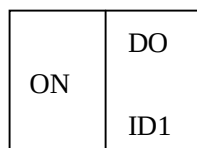
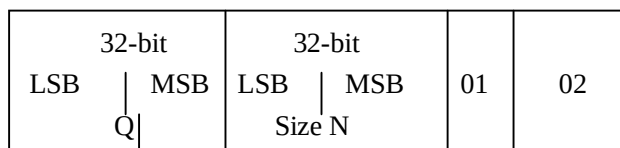
与 Q-码 7; I-Size 0, 1, 2, 3 的注解基本相同, 唯数量段变为 32bit。

当所描述的点需要用一个对象标识码 (例如一个文件对象的标识码或组态标题) 作为唯一的识别时, 用 Qualifier11。这种型式的标识码由对象的型式隐含。



Q 码为 11; I-Size 必为 1

字节 0i1...0iN 形成对象 1 之对象标识码, 其中  $0 \leq i < Q$  (Q 是数量) 在其后跟有被标识的对象。对象的规范包含在对象标识码内, 因而应用层必需能够解释对象标识标的某些段, 以便处理报文。



Q 码 11; I-Size 必为 2 字节 1 至 N 是对象标识码

至于前例，可以有很多标识码每一个跟在另一个的后面。

图 3-17 带有数据对象的报文

DocuCom PDF Trial  
www.pdfwizard.com

## 第四章：功能码的详细说明

本章以例题来解释应用层的功能码，包含有应用控制功能码与内部信号的应用标题已在大多数例子中省略了，但事实上它始终是存在的。请求与响应均被设想为各由一个分段（tuagment）所组成，多分段报文之一例在功能码 0 的说明中举出。

### 4. 1 CONFIRM（功能码 0）确认

在所收到的分段之应用控制段内当 CON 置位时，就要用确认功能去确认一个报文段之收到。注意在确认报文中，CON 位可置 0 或 1。这是可以选择本确认报文是否必须被确认。无论如何，任何一个站收到一个 CON 位置位的报文分段时，都要在发送任何其它应用层报文之前，先用一个确认报文去响应。此外，任何一个站发送一个 CON 位置位的报文分段后，就必须等到确认报文到达之后，才能继续传送报文分段或继续另一个传输回合。

|                                                  |                        |
|--------------------------------------------------|------------------------|
| 应用控制<br>Application Control<br>FIR=1,FIN=1,CON=? | 功能码<br>Function Code=0 |
|--------------------------------------------------|------------------------|

图 4-1 确认报文

对于一个单元段报文的确认响应示于图 4-1

#### Request Header

|                                |                   |
|--------------------------------|-------------------|
| 请求的标题<br>FLR=1, FIN=0<br>CON=1 | 读请求<br>请求一个多分段的响应 |
|--------------------------------|-------------------|

读请求，CON=0 表示该请求并不期望应用的确认。

#### Request Header

|                                |                   |
|--------------------------------|-------------------|
| 响应的标题<br>FLR=1, FIN=0<br>CON=1 | 响应的第一分段<br>之 ASDU |
|--------------------------------|-------------------|

响应的第一分段。CON=1 表示外站正期待“收到该分段”之应用确认。

|                                |           |
|--------------------------------|-----------|
| 请求的标题<br>FIR=1, FIN= 1 , CON=0 | 功能码<br>=0 |
|--------------------------------|-----------|

请求站发送的确认。

|                                |                   |
|--------------------------------|-------------------|
| 响应的标题<br>FIR=0, FIN= 0 , CON=1 | 响应的第二分段<br>之 ASDU |
|--------------------------------|-------------------|

响应的第二分段。

|                                |           |
|--------------------------------|-----------|
| 请求的标题<br>FIR=1, FIN= 1 , CON=0 | 功能码<br>=0 |
|--------------------------------|-----------|

对第二分段之确认。

|                              |                     |
|------------------------------|---------------------|
| 响应的标题<br>FIR=0, FIN=1, CON=0 | 响应的最后一个分段<br>之 ASDU |
|------------------------------|---------------------|

响应的最后分段

|                              |           |
|------------------------------|-----------|
| 请求的标题<br>FIR=1, FIN=1, CON=0 | 功能码<br>=0 |
|------------------------------|-----------|

对最后一个分段确认。

#### 4. 2 READ (功能码 1) 读令

读功能码是一种基本码，用以自外站请求数据对象，Object,Qualifier 和 Range 段的编码方法使它们的规模是可计算的，因而可以请求在一份报文里发送多个对象或变程。在一份单独的报文中所允许的多请求之数目由 DNP 内每一个设备的设备文件所定义。

对单个 range 对象这读请求示于图 4—2。图 4—3 之例是对两种对象型式的请求或者可能是对同一型式对象两种 range 之请求。

| 请求的标题 | 对象   | Qualifier | Range |
|-------|------|-----------|-------|
| AC    | FC=1 |           |       |

图 4—2 单个对象的请求

| 请求的标题 |     | 第一个读请求 |           |       | 第二个读请求 |          |       |
|-------|-----|--------|-----------|-------|--------|----------|-------|
| AC    | FC1 | object | qualifier | range | object | qulifier | range |

Object Header

Object Header

图 4—3 多对象或多 ranges 之请求

##### 4. 2. 1 读请求

以下诸例列举用于读功能码的几种 qualifier 与 range 之合法组合。

| 1  | 2    | 3         | 4         | 5                            | 字节                   |
|----|------|-----------|-----------|------------------------------|----------------------|
| AC | FC=1 | 对象<br>g=n | gv<br>V=0 | Qualifer<br>0000 0110<br>0 6 | I-Size=Φ<br>Q-Code=6 |

- 读对象组 **n**。
- 限定码 (**qualifier code**) • 指定外站所规定的组合之全部被定义的对象。  
该限定码也指明 **Range** 段不存在。
- 指标 (**index**) 的规模无意义故设于 **0**。
- 外站可以用所定义的组之任何对象或全部对象去响应。这种方法对于读事件数据之请求是有用的，因为请求方的站事先不知道哪些点发生了事件。

| octet1 | 2    | 3         | 4         | 5                             | 6                          | 7 | 字节制                  |
|--------|------|-----------|-----------|-------------------------------|----------------------------|---|----------------------|
| AC     | FC=1 | 对象<br>g=n | gv<br>v=x | qualifier<br>0000 0000<br>0 0 | range<br>start stop<br>7 9 |   | I-Size=Φ<br>Q-Code=Φ |

- 读第 **n** 组对象之变体 **X**。在本例中，**n** 与 **X** 该规定一个类属的字节对象。
- 限定码用一个两字节 **start** 与 **stop** 分段来规定一个 **range** 段。
- 该 **range** 段指定了一个起始的虚拟址 **7ΦΦ** 和虚拟的尾址 **7Φ2**。于对象包括自 **index7** 至 **index 9**。
- 指标规模 (**Index Size**) 没有意义并被设置 **0**。
- 外站可以在指定的范围(**range**)内响应，这种方法适用于请求当前（在请求时）有效的数据。

| 1  | 2    | 3                | 4   | 5                             | 6                              | 7 | 8 | 9 | 字节                   |
|----|------|------------------|-----|-------------------------------|--------------------------------|---|---|---|----------------------|
| AC | FC=1 | Object GV<br>G=N | V=X | Qualifier<br>0000 0001<br>0 1 | Range<br>start stop<br>700 702 |   |   |   | I-Size=0<br>Q-Code=1 |

- 读第 **n** 对象组的变体 (**variation**) **X**。
- 限定码用一个两字节的 **start** 与分段规定了一个 **range** 段。
- 该 **range** 段指定了 **3** 个对象，包括自 **index700** 至 **index702**。
- **Index** 的规模没有意义并被置 **0**。
- 外站可以在所指定的范围内响应，这种方法也适用于请求当前（在请求时）的有效

数据。

| 1  | 2    | 3             | 4         | 5                            | 6                          | 7 | 字节                   |
|----|------|---------------|-----------|------------------------------|----------------------------|---|----------------------|
| AC | FC=1 | Object<br>G=N | Gv<br>V=X | Qualifer<br>0000 0011<br>0 3 | Range<br>Start Stop<br>7 9 |   | I-Size=0<br>Q-Code=3 |

- 读第 **n** 对象组的变体 **X**。在本例中，**n** 与 **X** 该规定一个类属的字节对象。
- 限定码用一个单字节的 **Start** 与 **Stop** 分段规定了一个 **range** 段。
- **Range** 段指定了一个起始的虚拟址 **7** 和一个虚拟的尾址 **9**。
- **Index Size** 段无意义，并被设  $\Phi$ 。
- 外站可以在所指定的范围内响。这个方法适用于自某些远方应用之贮存器请求指定的字节。

**注：**虚拟地址通常被用于诊断的或制造的测试，因为必需熟悉外站的知识才能解释其响应

| 1  | 2   | 3             | 4         | 5                            | 6                              | 7 | 字节                         |
|----|-----|---------------|-----------|------------------------------|--------------------------------|---|----------------------------|
| AC | FC1 | Object<br>G=N | Gv<br>V=X | Qualifer<br>0000 0011<br>0 4 | Range<br>Start Stop<br>700 702 |   | I-Size= $\Phi$<br>Q-Code=4 |

- 读第 **n** 对象组的变体 **X**。在本例中，**n** 与 **X** 该规定一个类属的字节对象。
- 限定码用一个两字节的 **Start** 与 **Stop** 分段规定了一个 **range** 段。
- **Range** 段指定了一个起始的虚拟址 **7 $\Phi\Phi$**  和虚拟的尾址 **7 $\Phi$ 2**。
- **Index** 规模段无意义，并被设于  $\Phi$ 。
- 外站可以在所指定的范围内响。这个方法适用于自某些远方应用的贮存器请求指定的字节。

**注：**虚拟地址通常只用于诊断的或制造的或制造的测试，因为解释响应必需熟悉外站的知识。

| 1  | 2    | 3                    | 4 | 5                            | 6                                  | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|------|----------------------|---|------------------------------|------------------------------------|---|---|---|----|----|----|----|
| AC | FC=1 | Object GN<br>G=N V=X |   | Qualifer<br>0000 0101<br>0 5 | Range<br>Start Stop<br>70000 70002 |   |   |   |    |    |    |    |

I-Size=Φ      Q-Code=5

- 读第 **n** 对象组的变体 **X**。在本例中 **n** 与 **X** 该指定一个类属的字节对象。
- 限定码用一个 **4** 字节的 **Start** 与 **Stop** 分段规定了一个 **range** 段。
- **Range** 段指定了起始的虚拟址 **7ΦΦΦΦ** 和虚拟的尾址 **7ΦΦΦ2**。
- **Index** 规模段无意义，并被设于 **Φ**。
- 外站可以在所指定的范围内响应。这个方法适用于自某些远方应用的贮存器请求指定的字节。

注：同前。

| 1  | 2    | 3                    | 4                            | 5                      | 6 |                      |
|----|------|----------------------|------------------------------|------------------------|---|----------------------|
| AC | FC=1 | Object GV<br>G=N V=P | Qualifer<br>0000 0111<br>0 7 | Range<br>Quantity<br>3 |   | I-Size=0<br>Q-Code=7 |

- 读对象组 **n** 变体 **P** 组。
- 限定码指定一个 **1** 字节的量。
- **Range** 段指定了 **3** 个对象。
- **Index** 规模无意义并被设于 **0**。
- 这个方法适用于请求特定变体（例如模拟量的变化事件）的有限个数据，因为接收站可能不掌握外站的整个数据库。

| 1  | 2    | 3                    | 4                            | 5 | 6                          | 7 | 8 | 9 | 字节                   |
|----|------|----------------------|------------------------------|---|----------------------------|---|---|---|----------------------|
| AC | FC=1 | Object GV<br>G=N V=P | Qualifer<br>0000 0111<br>0 9 |   | Range<br>Quantity<br>70000 |   |   |   | I-Size=0<br>Q-Code=9 |

- 读对象组 **n** 变体 **P**。
- **Q**—码规定 **index** 段内的一份对象清单。
- **Range** 段规定了包含有 **3** 个条目的清单。
- **Index** 规范指定在清单内每个条目都是 **1** 字节的 **index**。
- 这个方法适用于请求一个特定的变体的有限数据，由于接收站不能掌握外站的整个数据库。

| 1  | 2    | 3                    | 4                            | 5                      | 6                        | 7 | 8 | 9 |
|----|------|----------------------|------------------------------|------------------------|--------------------------|---|---|---|
| AC | FC=1 | Object GV<br>G=N V=X | Qualifer<br>0001 0111<br>1 7 | Range<br>Quantity<br>3 | Indices<br>11   22   108 |   |   |   |

- 读对象组 **n** 变体 **X**。
- **Q**-码规定 **index** 段内的一份对象清单。
- **Range** 段指定了包含 **3** 个条目的清单。
- **Index** 规范指定在清单内每个条目都是 **1** 字节的 **index**。
- 这个方法适用于自远方设备请求某些指定的点。例如，可以请求某些紧要的状态点供分析或报告之用。

注：**Range** 段的规模总是与 **Index** 清单中的每一条目的规模相同，凡 **Indices** 之值在 **0** 与 **255** 之间者均可正常使用此格式。

| 1  | 2    | 3                    | 4                     | 5                 | 6                          | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|------|----------------------|-----------------------|-------------------|----------------------------|---|---|---|----|----|----|----|
| AC | FC=1 | Object GV<br>G=N V=X | Qualifer<br>0010 0111 | Range<br>Quantity | Indices<br>311   422   108 |   |   |   |    |    |    |    |

字节 **I-Size=2** **Q-Code=7**

- 读对象组 **n** 之变体。
- 限定码 (**Q-Code**) 规定了一份对象清单于 **Index** 段内。
- **Range** 段规定了清单内有 **3** 个条目。
- **I-Size** 段规定清单内每个条目为一个 **3** 字节的 **Index**。
- 这个方法适用于自远方设备请求某些指定的点。例如，为了分析或报告的目的值大于 **1** 个字节时，通常用此格式。

| 1  | 2    | 3                    | 4                            | 5                      | 6                                 | 7 | 8 | 9 | 18 | 19 | 20 | 21 |
|----|------|----------------------|------------------------------|------------------------|-----------------------------------|---|---|---|----|----|----|----|
| AC | FC=1 | Object GV<br>G=N V=X | Qualifer<br>0011 0111<br>3 7 | Range<br>Quantity<br>3 | Indices<br>70000   7600 0   90000 |   |   |   |    |    |    |    |

- 读对象组 **n** 变体 **X**。
- **Q-Code** (**7**) 规定一份对象清单一 **Index** 段。
- **Range** 段规定该清单内有 **3** 个条目。

- **I-Size** 段规定清单内每个条目为 **4** 字节的 **index**。
- 这个方法适用于自远方设备请求某些指定的点。例如，为了分析或报告的目的而请求某些紧要的状态点。

注：**Range** 段的规模总是与 **Index** 清单内每条条目的规模总是相同的。

| 1  | 2    | 3                    | 4                     | 5                 | 6                                   | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|------|----------------------|-----------------------|-------------------|-------------------------------------|---|---|---|----|----|----|----|
| AC | FC=1 | Object GV<br>G=N V=V | Qualifer<br>0010 0111 | Range<br>Quantity | Idetifier<br>Size Object Identifier |   |   |   |    |    |    |    |

Size Octet  
规模的字节数=Size

- 读对象组 **n** 之变体 **V**[在此情况下组 (**group**) 与变体 (**variation**) 就是数据的标识标。]

• **Q-Code(11)** 规定在段内是一份对象标识码清单和 **Range** 段必为 **8** 毕特的量。**Size** 段也必为一个 **8** 毕特的量，它规定着对象标识 (**Object identifier**) 的长度为 '**Size**' 个字节

• **Range** 段规定清单内只有一个条目，**I-Size** 段 (**1**) 规定 **Quantity** 段和 **Size** 段的长度均为 **8** 毕特。

• 这种方法租用于读取远方设备的组态参数 (若使用了文件对象的识别码)。这个方法还是请求一数据对象，其规模大于一个 **Fragment** 者的唯一方法。

| 1  | 2    | 3                    | 4                            | 5                 | 6                                  | 7 | 8 | 9 | 10 | 11 | 12 | 13 |
|----|------|----------------------|------------------------------|-------------------|------------------------------------|---|---|---|----|----|----|----|
| AC | FC=1 | Object GV<br>G=N V=X | Qualifer<br>0000 0010<br>0 2 | Range<br>Quantity | Range<br>Sart   Stop70002<br>70000 |   |   |   |    |    |    |    |

I-Size=0

Q-Code=2

- 读对象组 **n**, 变体 **X**。
- **Q-Code** 为 **2**，即指定 **S** 与 **S** 各为 **4** 字节。
- **Range** 段规定三个对象，自 **70000** 至 **70002**。
- **I-Size** 段无意义，故设于  $\Phi$ 。
- 外站可在规定的内作出响应，这个方法适用于请求，在请求时已知有效的数据。

#### 4. 2. 2 对读令之响应

本节对读请求的响应内，前置于对象的对象标题（**Object Header**）规定几种合法的形式。

| 1           | 2        | 3    | 4     | 5    | 字节 |
|-------------|----------|------|-------|------|----|
| Object GV   | Qualifer |      | Range |      |    |
| Group=n     | 0000     | 0000 | Start | Stop |    |
| Variation=V | 0        | 0    | 7     | 9    |    |

- 在此标题后面跟随着第 **n** 组中的变体 **V**。
- **Q—Code** 段规定 **Range** 段具有一字节的起停分段。
- **Range** 段规定在标题之后有 **3** 个对象，其 **Index** 为 **7** 至 **9**。
- **I—Size** (**0**) 表示 **Index** 的规模无意义，因为起停段指明的是每个对象的 **Index**。

| 1           | 2        | 3    | 4     | 5 | 6    | 7 | 字节 |
|-------------|----------|------|-------|---|------|---|----|
| Object GV   | Qualifer |      | Range |   |      |   |    |
| Group=n     | 0000     | 0000 | Start |   | Stop |   |    |
| Variation=V | 0        | 1    | 300   |   | 302  |   |    |

- 在此标题后面跟随着第 **n** 组中的变体 **V**。
- **Q—Code** (**1**) 规定 **Range** 段内具有两字节的起和停的分段。
- **Range** 段规定有 **3** 个对象，其 **Index** 为 **300** 至 **302**，跟在标题之后。
- **I—Size** 无意义被设置于  $\Phi$ ，因为起停段说明的是每个对象的 **Index**。

| 1           | 2        | 3    | 4     | 5 | 6 | 7 | 8 | 9     | 10 | 11 | 字节 |
|-------------|----------|------|-------|---|---|---|---|-------|----|----|----|
| Object GV   | Qualifer |      | Range |   |   |   |   |       |    |    |    |
| Group=n     | 0000     | 001D | Start |   |   |   |   | Stop  |    |    |    |
| Variation=V | 0        | 2    | 70000 |   |   |   |   | 70002 |    |    |    |

- 此标题之后跟随着第 **n** 组中的变体 **V**。
- **Q—Code** (**2**) 规定 **Range** 段内拥有四字节的起停分段。
- **Range** 段规定有 **3** 个对象跟在标题之后，其为 **Index** 自 **70000** 至 **70002**。
- **I—Size** 在此无意义，被设置于 **0**，因为起停段说明的是每个对象之 **Index**。

| 1           | 2        | 3    | 4     | 5    | 字节 |
|-------------|----------|------|-------|------|----|
| Object GV   | Qualifer |      | Range |      |    |
| Group=n     | 0000     | 0011 | Start | Stop |    |
| Variation=V | 0        | 3    | 7     | 9    |    |

- 此标题之后跟随着第 **n** 组中的变体 **V**。
- **Q—Code (3)** 规定 **range** 段内带有一字节的起停分段 (**Sub-field**)。
- **Range** 段规定虚拟地址 **7** 至 **9** 的内容跟随在标题有之后。
- **I—Size** 无意义，被设置于 **0**，因为起停段指明的是每个对象之 **Index**。

| 1           | 2        | 3    | 4     | 5    | 6 | 7 | 字节 |
|-------------|----------|------|-------|------|---|---|----|
| Object GV   | Qualifer |      | Range |      |   |   |    |
| Group=n     | 0000     | 0100 | Start | Stop |   |   |    |
| Variation=V | 0        | 4    | 300   | 302  |   |   |    |

- 此标题之后跟随着第 **n** 组中的变体 **V**。
- **Q—Code (4)** 规定 **Range** 段内为 **2** 字节的起停分段。
- **Range** 段指定虚拟地址 **300** 至 **302** 的内容跟随在此标题有之后。
- **I—Size** 段无意义并被设置于 **0**，因为起停地址指出了每个对象的 **Index**。

| 1           | 2        | 3    | 4     | 5 | 6 | 7 | 8     | 9 | 10 | 11 | 字节 |
|-------------|----------|------|-------|---|---|---|-------|---|----|----|----|
| Object GV   | Qualifer |      | Range |   |   |   |       |   |    |    |    |
| Group=n     | 0000     | 0101 | Start |   |   |   | Stop  |   |    |    |    |
| Variation=V | 0        | 5    | 70000 |   |   |   | 70002 |   |    |    |    |

- 此标题之后跟随着第 **n** 组中的变体 **V**。
- **Q—Code** 规定 **Range** 段内有 **4** 字节的 **Start** 与 **Stop** 分段。
- **Range** 段指定虚拟地址 **70000** 至 **70002** 的内容跟随于本标题有之后。
- **I—Size** 无意义并被设置于 **0**，因为 **Start** 与 **Stop** 段地址指明了每个对象的 **Index**。

| 1           | 2        | 3   | 4     |
|-------------|----------|-----|-------|
| Object GV   | Qualifer |     | Range |
| Group=n     | 0000     | 011 | 3     |
| Variation=V | 0        | 7   |       |

- 在此标题后面跟随着以第 **n** 组中的变体 **V**。
- **Q—Code** (7) 段规定 **Range** 段为 **1** 字节的量。
- **Range** 段规定了数量为 **3** 个对象。
- **I—Size** 规定数据是被压缩了的, 没有 **Indices** 前置于对象。

本例中隐含着在报文的第一个对象有一个 **0** 的 **Index** 和在末尾那个对象将有一个 **2** 的 **Index**。

| 1           | 2        | 3    | 4     | 字节 |
|-------------|----------|------|-------|----|
| Object GV   | Qualifer |      | Range |    |
| Group=n     | 0010     | 0111 |       |    |
| Variation=V | 2        | 7    | 3     |    |

- 跟随于本标题之后者是第 **n** 组中的变体 **V**。
- **Q—Code** 规定段为一字节的量 (**Q**)。
- **Range** 段规定有 **3** 个对象
- **I—Size** 规定对象们都前置有 **2** 字节的 **Index** 标题码 (**iden-fifiers**)。

| 1           | 2        | 3    | 4     | 字节 |
|-------------|----------|------|-------|----|
| Object GV   | Qualifer |      | Range |    |
| Group=n     |          |      |       |    |
| Variation=V | 0011     | 0111 |       |    |
|             | 3        | 7    | 3     |    |

- 跟随于本标题之后者是第 **n** 组中的变体 **V**。
- **Q—Code** (7) 规定 **Range** 段有一个字节的量 (**Q**)。
- **Range** 段数据段规定了有 **3** 个对象。
- **I—Size** 规定对象们均前置 **4** 个字节的 **Index identifiers**。

| 1           | 2        | 3    | 4     |  |
|-------------|----------|------|-------|--|
| Object GV   | Qualifer |      | Range |  |
| Group=n     |          |      |       |  |
| Variation=V | 0100     | 0111 |       |  |
|             | 4        | 7    | 1     |  |

- 跟随于本标题之后者是第 **n** 组中的变体 **V**。
- **Q—Code** (7) 规定 **Range** 段为一个 **8** 毕特的量 (**Q**)。
- **Range** 段规定了数据段对象数目为 **1**。

- **I—Size (4)** 规定了对象们都前置以 **1** 个字节的对象规模 (**object size**)。

• 注意，由于没有指定的数据 **indices** (以及响应请求也不一定配适，数据对象本身就要包含唯一的标识 (例如时间印记))。

| 1           | 2        | 3    | 4     | 5 | 字节 |
|-------------|----------|------|-------|---|----|
| Object GV   | Qualifer |      | Range |   |    |
| Group=n     | 0000     | 1000 |       |   |    |
| Variation=V | 0        | 8    | 3     |   |    |

- 跟随在本标题后者的是第 **n** 组中的变体 **V**。
- **Q—Code** 规定 **Range** 段有 **2** 个字节的量 (**Q**)。
- **Range** 段规定了有 **3** 个对象。
- **I—Size** 规定对象是被压缩了的没有前置于对象。

这里隐含报文中的第一对象有 **Index 0** 以及最后一个对象有一个 **Index 2**。

• 注意没有指定对象的 **index** (以及响应与请求不一定配适，数据对象本身必需包含某些唯一的 **identification** (诸如一个时间印记))。

| 1           | 2        | 3    | 4     | 5 | 字节 |
|-------------|----------|------|-------|---|----|
| Object GV   | Qualifer |      | Range |   |    |
| Group=n     | 0001     | 1000 |       |   |    |
| Variation=V | 1        | 8    | 3     |   |    |

- 跟在标题后面的是第 **n** 组中的变体 **V**。
- **Q—Code** 规定一个 **Range** 段有一个 **2** 字节的量 (**Q**)。
- **Range** 段规定了有 **3** 个对象。
- **I—Size (1)** 规定对象们都前置有一个 **1** 字节的 **Index identifier**。

| 1           | 2        | 3    | 4     | 5 | 字节 |
|-------------|----------|------|-------|---|----|
| Object GV   | Qualifer |      | Range |   |    |
| Group=n     | 0010     | 1000 |       |   |    |
| Variation=V | 2        | 8    | 300   |   |    |

- 跟在本标题后面的对象是第 **n** 组中的变体 **V**。
- **Q—Code (8)** 规定 **Range** 段有一个 **2** 字节的量 (**Q**)。
- **Range** 段规定了有 **300** 个对象。

- **I—Size** 规定对象们都被前置一个 **2** 字节的 **Index identifier**。

1      2                      3      4      5                      字节

| Object GV   | Qualifer |      | Range |
|-------------|----------|------|-------|
| Group=n     | 0011     | 1000 |       |
| Variation=V | 3        | 8    | 300   |

- 跟在标题后面的对象是第 **n** 组中的变体 **V**。
- **Q—Size** 规定段有一个 **2** 字节的量 (**Q**)。
- **Range** 段规定了有 **300** 个对象。
- **I—Size(1)** 规定对象们均前置了一个四个字节的 **Index identifier**。

1      2                      3      4      5                      字节

| Object GV   | Qualifer |      | Range    |
|-------------|----------|------|----------|
| Group=n     | 010      | 1000 | Quantity |
| Variation=V | 5        | 8    | Q        |

- 跟随在此标题后面的对象是第 **n** 组中的变体 **V**。
- **Q—Code (8)** 规定 **Range** 段是一个 **16** 个毕特的量 (**Q**)。
- **Range** 段规定了在数据段内的对象数目。
- **I—Size (5)** 规定对象们均前置有 **2** 字节的对象规模。

• 注意，由于对象的 **indices** 未被指定（以及响应与请求不一定配适），数据对象就必需包含某些唯一的标识（例如一个时间印记）。

1      2                      3      4      5      6      7                      字节

| Object GV   | Qualifer |      | Range |
|-------------|----------|------|-------|
| Group=n     | 0000     | 1010 |       |
| Variation=V | 0        | 9    | 3     |

- 跟随在此标题后面的对象是第 **n** 组中的变体 **V**。
- **Q—Code (9)** 规定 **Range** 段是一个 **4** 字节的量 (**Q**)。
- **Range** 段规定了有 **3** 个对象。
- **I—Size** 规定对象是被压缩了的，没有 **indices** 前置于每个对象之前，这隐示在报文中的第一个对象有一个 **index0**，在本例中末尾一个对象有一个 **index2**。

| 1           | 2        | 3    | 4     | 5 | 6 | 7 | 字节 |
|-------------|----------|------|-------|---|---|---|----|
| Object GV   | Qualifer |      | Range |   |   |   |    |
| Group=n     | 0001     | 1001 |       |   |   |   |    |
| Variation=V | 1        | 9    | 3     |   |   |   |    |

- 跟随在此标题之后的对象是第 **n** 组中的变体 **V**。
- **Q—Code (9)** 规定 **Range** 段是一个 **4** 字节的量 (**Q**)。
- **Range** 段规定了有 **3** 个对象。
- **I—Size** 规定对象们均前置有 **1** 个字节的 **Index identifier**。

| 1           | 2        | 3    | 4     | 5 | 6 | 7 | 字节 |
|-------------|----------|------|-------|---|---|---|----|
| Object GV   | Qualifer |      | Range |   |   |   |    |
| Group=n     | 0010     | 1001 |       |   |   |   |    |
| Variation=V | 2        | 9    | 3     |   |   |   |    |

- 跟随在此标题后面都是第 **n** 组中的变体 **V**。
- **Q—Code (9)** 规定 **Range** 段是一个 **4** 字节的量 (**Q**)。
- **Range** 段规定了有 **3** 个对象。
- **I—Size** 规定对象们均前置有 **2** 个字节的 **Index identifier**。

| 1           | 2        | 3    | 4     | 5 | 6 | 7 | 字节 |
|-------------|----------|------|-------|---|---|---|----|
| Object GV   | Qualifer |      | Range |   |   |   |    |
| Group=n     | 0100     | 1001 |       |   |   |   |    |
| Variation=V | 3        | 9    | 3     |   |   |   |    |

- 跟在此标题之后的对象是第 **n** 组中的变体 **V**。
- **Q—Code (9)** 规定 **Range** 段内是一个 **4** 个字节的量 (**Q**)。
- **Range** 段的 **Q** 规定有 **3** 个对象。
- **I—Size** 规定对象们均都前置有 **4** 个字节的对象规模 **idex identifier**。

| 1           | 2        | 3    | 4        | 5 | 6 | 7 | 字节 |
|-------------|----------|------|----------|---|---|---|----|
| Object gv   | Qualifer |      | Range    |   |   |   |    |
| Group=n     | 0100     | 1001 | quantity |   |   |   |    |
| Variation=V | 4        | 9    |          |   |   |   |    |

- 跟在此标题之后的对象是第 **n** 组中的变体 **V**。

- **Q—Code (9)** 规定 **Range** 段内是一个 4 个字节的量 (**Q**)。
- **Range** 段的 **Q** 规定有 **Q** 个对象在数据段内。
- **I—Size** 规定对象们均都前置有一个字节的对象规模 **index identifier**。
- 注意，由于对象的未被指定（以及响应与请求不一定配适），数据对象本身必需包含某些唯一的标识（例如时间印记）。

读令的响应也可以由多段 (**multi-fragment**) 报文所组成用每一个对象识别码 (**object identifier**) 去规定全部对象的不同部份 (页面):

#1 段 (**fragment**): **FC=129**    **CON=?**    **FIR=1**    **FIN=0**

字节    1    2    3    4    6    7.....

| Object GV |     | Qualifer |      | Range    | Identifier                  |  | Data    |
|-----------|-----|----------|------|----------|-----------------------------|--|---------|
| G=N       | V=V | 0001     | 1011 | Quantity | Object Identifier for piece |  | For     |
|           |     | 1        | 11   | 1        |                             |  | Piece 1 |

→ m1 字节 ←

#2 段: **FC=129**    **CON=?**    **FIR=0**    **FIN=0**

字节    1    2    3    4    5    6.....

| Object GV |     | Qualifer |      | Range    | Identifier                  |  | Data    |
|-----------|-----|----------|------|----------|-----------------------------|--|---------|
| G=N       | V=V | 0001     | 1011 | Quantity | Object Identifier for piece |  | For     |
|           |     | 1        | 11   | 1        |                             |  | Piece 2 |

→ m2 字节 ←

#3 段 0: **FC=129**    **CON=?**    **FIR=0**    **FIN=1**

字节    1    2    3    5    6.....

| Object GV |     | Qualifer |      | Range    | Identifier                  |  | Data    |
|-----------|-----|----------|------|----------|-----------------------------|--|---------|
| G=N       | V=V | 0001     | 1011 | Quantity | Object Identifier for piece |  | For     |
|           |     | 1        | 11   | 1        |                             |  | Piece 3 |

→ m3 字节 ←

- 读对象第 **n** 组中的变体 **V**（在本例中组别与变体类别作为对象识别码）。
- **Q—Code (11)** 规定在 **Identifier** 段内是一份 **Object identifiers** 的清单并规定段内是 8 毕特的量 (**Q**)。 **Size** 段也是一个 8 毕特的量 (**m**)，它规定对象识别码 (**Object Identifier**) 加上其后的数据之字节长。

- **Range** 段规定了此清单只有一个条目，**I-Size (1)** 也规定 **Range** 段和 **Size** 段都是 8 毕特长。

- 这个方法适用于远方终端读取组态参数（如果用“文件标识码对象）。这个方法又是请求其规模长于一个段（**fragment**）的数据对象之唯一方法。

- 注意每个 **fragment** 都是请求站所能领悟的，因为每一个 **object identifier** 规定了对象的唯一部份。

#### 4. 3 写令（功能码 2）

写功能码采用以将对象自一个主站移送至一个外站，图 4—4 举出写单个对象型式或的格式，对象标题的定义就和写的请求一样。图 4—6 是对一个外站写入组态参数之列。来自外站的响应总是一个功能码，后面跟着内部信号 **IIN**。写功能的典型用途系用以下载组态参数或文件至一个外站以及向外站置定时间。



图 4—4 单个对象或多 **Ranges** 的写令

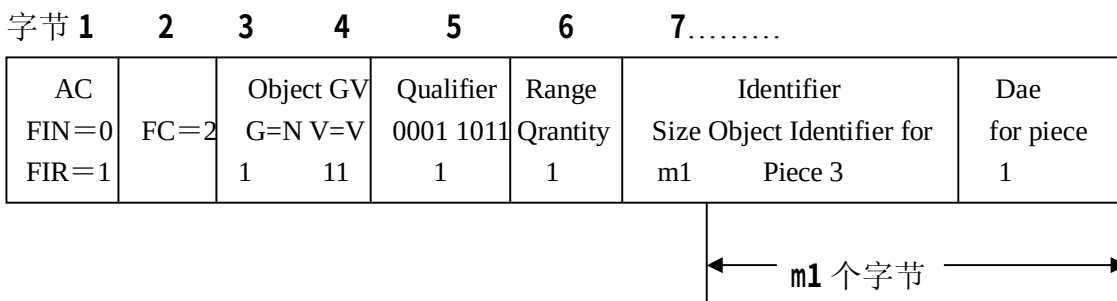


图 4—5 多个对象或多 **Ranges** 的写令

##### 4. 3. 1 写请求（Write Requests）

写请求将一个多段（**multi-fragment**）的对象传输至远方设备，写请求典型地用以写对象，诸如内部信号（**IIN**）对象，文件识别（**File Identifier**）对象以及时间对象和日期对象。

每个段（**fragment**）包含整个对象每个部份的标识信息。



字节 1      2      3      4      5      6      7.....

|                      |      |                      |                                   |                        |                                                             |                        |
|----------------------|------|----------------------|-----------------------------------|------------------------|-------------------------------------------------------------|------------------------|
| AC<br>FIN=0<br>FIR=0 | FC=2 | Object GV<br>G=N V=V | Qualifier<br>0001 1011<br>1    11 | Range<br>Quantity<br>1 | Identifier<br>Size Object Identifier for<br>m2      Piece 2 | Date<br>for piece<br>2 |
|                      |      |                      |                                   |                        | ← m2 个字节 →                                                  |                        |

字节 1      2      3      4      5      6      7.....

|                      |      |                      |                                   |                        |                                                            |                        |
|----------------------|------|----------------------|-----------------------------------|------------------------|------------------------------------------------------------|------------------------|
| AC<br>FIN=1<br>FIR=0 | FC=2 | Object GV<br>G=N V=V | Qualifier<br>0001 1011<br>1    11 | Range<br>Quantity<br>1 | Identifier<br>Size Object Identifier for<br>m3      Piece3 | Date<br>for piece<br>3 |
|                      |      |                      |                                   |                        | ← m3 个字节 →                                                 |                        |

- 写第 **n** 组中的变体 **V** 对象（在此例中组别与变体类别就成了对象标识码）。
- **Q-Code (n)** 规定了在 **identifier** 段内是一份标识码的清单，并在 **Range** 段内是一 **8** 毕特的量。在 **Size** 段内也是一个 **8** 毕特的量用以规定对象标识码加上数字的字节的长度。
- 该 **Range** 段规定该清单只有一个条目（**entry**）。
- 数据段则包含属于标识对象的数据（其规模定于标识段）。
- 这种方法适用于向远方设备下装组态内容（如果使用了文件对象标识码）因为这种数据的内容应用层是不予解释的。若要发送一个规模大于一节（**fragment**）的数据对象，用这条还是唯一的办法。

下面的一条写请求的是对外站设置时间和 **E** 掌握的

1      2      3      4      5      6      7.....

|                      |      |                      |                                   |                        |             |
|----------------------|------|----------------------|-----------------------------------|------------------------|-------------|
| AC<br>FIN=1<br>FIR=1 | FC=2 | Object gv<br>G=V V=V | Qualifier<br>0000 0000<br>0    00 | Range<br>Quantity<br>1 | Time Object |
|----------------------|------|----------------------|-----------------------------------|------------------------|-------------|

- **Q-Code** 规定 **Range** 段内是一个字节的量，**Quantity** 段规定只有 **1** 个对象以及 **Object** 段内标明对象是时间与日期的对象。

#### 4. 3. 2 写响应

对于一个写的请求，它的响应可以仅包含有表明写操作成功了的内部信号（**IIN**），所以在一个写的响应中没有 **ASDU** 部份，这响应总是使用 **Response** 的功能码。

4. 4 选点（功能码 3）

选点功能码（**Select**）用以在外站选定一个或多个控制点。这些点可能是控制继电器输出、模拟量输出或模式（**pattern**）控制组，这种功能并不输出或激活新的状态或数值，而是使它们准备好（处于待发态）并报告它们的状态。必需发送附加的操作令报文至此外站去激活这个请求，该操作命令的报文所控制的对象相符，选择令报文使外站起动一个定时器，为了产生激活操作，执行令必需在定时器的时间之前送发。

有两种方式去控制报文，第一种方式是用于固定规模的控制块而第二种方式是用于规模可变的控制块（**block**）。可变规模的控制块用于模式（**pattern**）输出。

图 4—6 例举一条选点报文（固定规模的控制块）自主站发往外站。

外站以准备好指定的点（在控制块前加上 **index** 来指定去响应一个选择报文，并在响应中回送请求如图 4—7 所示（固定规模的控制块）。除了包含 **IIN** 和被修改的状态字节（控制块对象的一部份）以外，响应等同于请求，状态的字节陈述“选择”请求之成功或失败。

|    |      |                                     |                                |                        |       |                              |       |                             |
|----|------|-------------------------------------|--------------------------------|------------------------|-------|------------------------------|-------|-----------------------------|
| AC | FC=3 | Object GV<br>Group=N<br>Variation=V | Qualifier<br>00001 0111<br>1 7 | Range<br>Quantity<br>2 | Index | Control;<br>Block<br>Object1 | Index | Control<br>Block<br>Object2 |
|----|------|-------------------------------------|--------------------------------|------------------------|-------|------------------------------|-------|-----------------------------|

图 4—6 主站选择 2 个控制或模拟输出

- 跟在报头后面的 **Object** 是第 **n** 组（必需是一个控制块对象），变体 **V**。
- 限定码（**Q**—码）用控制块的一个字节的量类规定一个范围字段（**Range Field**）。（按：规定 **Range Field** 为 1 字节。）
- 指标规模规定每个控制块前置以 1 个字节的 **indices**。

|    |    |     |                                     |                               |                   |       |                              |       |                             |
|----|----|-----|-------------------------------------|-------------------------------|-------------------|-------|------------------------------|-------|-----------------------------|
| AC | FC | FIN | Object GV<br>Group=N<br>Variation=V | Qualifier<br>0001 0111<br>1 7 | Range<br>Quantity | Index | Control;<br>Block<br>Object1 | Index | Control<br>Block<br>Object2 |
|----|----|-----|-------------------------------------|-------------------------------|-------------------|-------|------------------------------|-------|-----------------------------|

图 4—7 外站的响应

外站响应中的控制对象都必须与请求中的对象相匹配，否则将不发送操作信息。

4. 4. 1 模式控制（**Pattern Control**）

图 4—8 举一个主站为一个模式而发的选择的报文，和图 4—9 一个外站的响应。

|    |      |                                     |                               |                   |                            |
|----|------|-------------------------------------|-------------------------------|-------------------|----------------------------|
| AC | FC=3 | Object GV<br>Group=a<br>Variation=b | Qualifier<br>0000 0111<br>0 7 | Quantity<br><br>1 | Pttern<br>Control<br>block |
|----|------|-------------------------------------|-------------------------------|-------------------|----------------------------|

(continued)

|                                   |                               |                             |                            |                            |                           |                            |
|-----------------------------------|-------------------------------|-----------------------------|----------------------------|----------------------------|---------------------------|----------------------------|
| Object GV<br>Guoup-c<br>Vauiation | Qualifier<br>0000 0000<br>0 7 | Range<br>start stoop<br>5 8 | Pattern<br>Mask<br>object5 | Pattern<br>Mask<br>object6 | Pttern<br>Mask<br>object7 | Pattern<br>Mask<br>object8 |
|-----------------------------------|-------------------------------|-----------------------------|----------------------------|----------------------------|---------------------------|----------------------------|

图 4—8 一个输出模式的主站选择报文

• 所指定的第一对象，对象，变体，是一个“模式控制块”，它描述“模式屏蔽对象”(Pattern Mask Object)们所规定的控制点间所共有的全部参数。在这个对象之后是一些模式屏蔽对象，对象，变体，它描述点是要被激活。

- 第一个 Q 码规定一个 8 位的变程（数量）段，它规定了 1 个“模式屏蔽块”。
- 第二个码规定了要用一个字节的起停分字段作为范围（range）段。这个 range 指定了被作为模式控制的模式屏蔽对象。
- 以上的报文构成一个模式控制请求。

多个请求（“模式控制块后随以多个模式屏蔽对象”）可以在一帧报文中发送，只要请求的总规模塞进一个应用层分段就行。

|    |        |                                     |                               |                   |                            |
|----|--------|-------------------------------------|-------------------------------|-------------------|----------------------------|
| AC | FC=128 | Object GV<br>Group=a<br>Variation=b | Qualifier<br>0000 0111<br>0 7 | Quantity<br><br>1 | Pttern<br>Control<br>block |
|----|--------|-------------------------------------|-------------------------------|-------------------|----------------------------|

(continued)

|                                     |                               |                             |                            |                            |                           |                            |
|-------------------------------------|-------------------------------|-----------------------------|----------------------------|----------------------------|---------------------------|----------------------------|
| Object gv<br>Guoup-c<br>Vauiation=d | Qualifier<br>0000 0000<br>0 7 | Range<br>start stoop<br>5 8 | Pattern<br>Mask<br>object5 | Pattern<br>Mask<br>object6 | Pttern<br>Mask<br>object7 | Pattern<br>Mask<br>object8 |
|-------------------------------------|-------------------------------|-----------------------------|----------------------------|----------------------------|---------------------------|----------------------------|

图 4—9 外站对模式选择报文之响应

- 注意响应的格式等同于请求报文（应用响应报头除外），所回送的对象将被主站用以验证选择操作是否成功。

#### 4. 5 操作功能

被用以先收到一个与之相匹配的选择功能码的报文，并且在执行激活之前还得有一个开动的定时人仍在活动之中，除了功能为 4 之外，其余与选择报文相同。

|    |    |                                     |                               |                           |                   |                   |
|----|----|-------------------------------------|-------------------------------|---------------------------|-------------------|-------------------|
| AC | FC | Object GV<br>Group=a<br>Variation=b | Qualifier<br>0000 0111<br>0 7 | Quantity<br>Quantity<br>2 | Control<br>block1 | Control<br>block2 |
|----|----|-------------------------------------|-------------------------------|---------------------------|-------------------|-------------------|

图 4—10 2 个输出或 2 个置定值的主站操作令报文

- 在报头后面的对象是 **group2**（必须是与输出或设置点有关一个控制块），**Variation V**。
- **Q**—码指定的段带有一个字节的控制块数量。

|    |    |     |                                     |                               |                           |                       |                       |
|----|----|-----|-------------------------------------|-------------------------------|---------------------------|-----------------------|-----------------------|
| AC | FC | IIN | Object GV<br>Group=n<br>Variation=V | Qualifier<br>0000 0111<br>0 7 | Quantity<br>Quantity<br>2 | Control<br>block<br>1 | Control<br>block<br>2 |
|----|----|-----|-------------------------------------|-------------------------------|---------------------------|-----------------------|-----------------------|

图 4—11 外站的响应

- 跟在报头后面的对象是 **group n**（必需是有关输出或设置点有关一个控制块对象），**Variation V**。
  - **Q**—码规定控制块的数量为 **1** 字节。
  - 操作成功或失败的信号在“输出状态”字节中回送，它们被内置于每个控制对象之内。
- 若对象是“模式屏蔽”对象，其状态则为前置于屏蔽（**mask**）的“模式控制块”对象一部份。

#### 4. 6 直接操作（功能码 5）

直接操作码被用以在外站激活一个或多个输出，它不要求前置以选择令报文除了功能码为 **5** 以外，此报文格式和一个选择令或操作的报文相同

|    |      |                                   |                               |                        |                   |                   |
|----|------|-----------------------------------|-------------------------------|------------------------|-------------------|-------------------|
| AC | FC=5 | Object GV<br>Group=n<br>Variation | Qualifier<br>0000 0111<br>0 7 | Range<br>Quantity<br>2 | Control<br>block1 | Control<br>block2 |
|----|------|-----------------------------------|-------------------------------|------------------------|-------------------|-------------------|

图 4—12 2 个输出或设置点的主站选择令

- 在报头后面的对象是 **group n**（必需是有关输出或设置点的控制块对象），**Variation V**。
- **Q**—码指定 **range** 段为一字节的控制块数量。

|    |    |     |                                     |                               |                           |                   |                   |
|----|----|-----|-------------------------------------|-------------------------------|---------------------------|-------------------|-------------------|
| AC | FC | IIN | Object GV<br>Group=n<br>Variation=V | Qualifier<br>0000 0111<br>0 7 | Quantity<br>Quantity<br>2 | Control<br>block1 | Control<br>block2 |
|----|----|-----|-------------------------------------|-------------------------------|---------------------------|-------------------|-------------------|

- 在报头后面的对象是 **group n**（必需是有关输出或设置点的控制块对象），**Variation V**。

- **Q**—码指定 **range** 段为一字节的控制块数量。

#### 4.7 直接操作一无确认（功能码 6）

直接操作一无确认的功能码被用以激活一个站内一个或多个输出或设置点，无需前置以选择令报文，本报文格式与选择令或操作令报文相似，所不同者功能码为 **6** 以及外站对主站不发响应报文。

|    |      |                                     |                               |                        |                   |                   |
|----|------|-------------------------------------|-------------------------------|------------------------|-------------------|-------------------|
| AC | FC=6 | Object GV<br>Group=n<br>Variation=V | Qualifier<br>0000 0111<br>0 7 | Range<br>Quantity<br>2 | Control<br>block1 | Control<br>block2 |
|----|------|-------------------------------------|-------------------------------|------------------------|-------------------|-------------------|

图 4—14 2 个输出或设置点的主站选择令

- 在报头后面的对象是 **group n**（必需是有关输出或设置点的控制块对象），**Variation V**。

- **Q**—码指定 **range** 段为一字节的控制块数量。

#### 4.8 立即冻结（功能码 7）

本功能码用以将指定的数据对象复制入一个冻结缓存。收到此报文后，应将指定的数据对象之前值复制至它们适当的冻结缓存。被冻结的对象可以被请求（在另一个请求报文中）索取，冻结的对象。

|    |      |               |       |               |
|----|------|---------------|-------|---------------|
| AC | FC=7 | Object Header | ..... | Object Header |
|----|------|---------------|-------|---------------|

图 4—15 主站立即冻结的控制报文

在请求中可以包含多个对象的报头用以指定冻结许多对象，然而典型的应用只是冻结计数对象。

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—16 外站对立即冻结令之响应

外站也可以指出（在 **IIN** 内）对所请求的对象不知道。

#### 4.9 立即冻结一无响应（功能码 8）

该功能码与前一条功能码（即冻结）完全相同，底是外站无需响应，此功能典型地用以对同属一个主站的全体外站执行一种全面冻结。在这种情况下，数据链路层的无回信发送服务就必需在某种环境下使用了。

|    |      |               |       |               |
|----|------|---------------|-------|---------------|
| AC | FC=8 | Object Header | ..... | Object Header |
|----|------|---------------|-------|---------------|

图 4-17 主站立即冻结令之响应

#### 4. 10 冻结并清除（功能码 9）

本功能码用将指定的数据复制到一个冻结缓存器中，就和立即冻结功能码一样，但这里将随即清除掉那个指定的数据对象，此功能码，典型地用以冻结计数器或累加器并随即将它复位至 0。

|    |      |               |       |               |
|----|------|---------------|-------|---------------|
| AC | FC=9 | Object Header | ..... | Object Header |
|----|------|---------------|-------|---------------|

图 4-18 主站冻结并清除的控制报文

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4-19 外站对冻结并清除的控制报文

#### 4. 11 冻结并清除一无确认（功能码 10）

此功能码工作起来和前一功能码（冻结并清除）相同，仅仅无需外站的确认，此功能典型地用于对同属一个主站的全体外站执行一次全面的冻结并清除。

|    |       |               |       |               |
|----|-------|---------------|-------|---------------|
| AC | FC=10 | Object Header | ..... | Object Header |
|----|-------|---------------|-------|---------------|

图 4-20 主站的冻结并清除，无确认的报文

#### 4.12 按时间地冻结（功能码 11）

本功能码对一个指定的数据对象起周期性地冻结，在要冻结的对象之前发送时间、日期和时间间隔被说明以下的表格内，如表所示，对象有两个组成部份：一个是时间段（绝对的），另一个间隔段。这个字段的值取决于外站冻结的行为。

| 时间 | 间隔 | 说 明                                     |
|----|----|-----------------------------------------|
| 无零 | 零  | 在指定的时间冻结一次                              |
| 零  | 零  | 立即冻结                                    |
| 零  | 无零 | 冻结同步于当前小时的始点，第一次冻结将发生在等于或大于当前时间的最小递增倍数。 |
| 无零 | 无零 | 冻结起始于指定的时间并在以后每个递增的间隔上重复。               |

|    |       |                                  |                                       |                                                |
|----|-------|----------------------------------|---------------------------------------|------------------------------------------------|
| AC | FC=11 | Object Header<br>For time object | Time Object<br>Time and Date Interval | Object Header<br>for data objects<br>to freeze |
|----|-------|----------------------------------|---------------------------------------|------------------------------------------------|

|       |          |          |
|-------|----------|----------|
| 时间对象的 | 时间对象     | 要冻结的数据对象 |
| 对象标题  | 时间和日期 间隔 | 之对象标题    |

图 4-21 带时间的主站冻结令报文

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4-22 外站对带时间的冻结令之响应

时间对象必需包含时间与间隔，这些对象定义于（P009-OBL）。

例：一个时间对象规定了当天的时间 2: 32pm 而以后的冻结每 10 分钟发生一次，始自 2: 42pm.

#### 4. 13 带时间的冻结一无确认（功能码 12）

此功能码的工作与前一条功能码（按时间地冻结）完全相同，是无需外站响应。这种功能码典型地用以对属于一个主站的全体外站执行全面性的冻结在这种情况下，“数据链路层的 SEND-NO RELAY（发送一无回信）的服务就要用在某种环境中。

|    |        |                                  |                                       |                                                |
|----|--------|----------------------------------|---------------------------------------|------------------------------------------------|
| AC | FC=112 | Object Header<br>for time object | Time Object<br>Time and Date Interval | Object Header<br>for data objects<br>to freeze |
|----|--------|----------------------------------|---------------------------------------|------------------------------------------------|

图 4-23 主站带时间的冻结令一无确认之报文

#### 4. 14 冷再起动（功能码 13）

冷再起动功能码使外站执行一次用户应用层的全部再起动，就象新接能电源那样。

|    |       |
|----|-------|
| AC | FC=13 |
|----|-------|

图 4-24 主站冷再起动的控制报文

|    |    |     |                              |                      |
|----|----|-----|------------------------------|----------------------|
| AC | FC | IIN | Object<br>Header<br>for time | Time Delay<br>Object |
|----|----|-----|------------------------------|----------------------|

图 4-25 外站对冷再起动的响应

外站收到了冷再起动的请求后，将用一个“延时对象”Time Delay Object（Time Delay Fine or Time Delay Course）去响应，它指定一个时间间隔，直到这段间隔之后外站才能准备好再通信。主站在这段时间里不打算对此外站通信。这段时间使外站能执行一次再起动顺序并能再进行 DNP 通信，这个响应被发送完以后，（且通信回合是成功的）外站应执行再起动程序。“精延时对象”(Time Delay Fine)被定义于 DNP V3.00Data Object Library(P009-OBL)。

4. 15 热再起

此再起功能码指定外站执行一次再起但不必运行复位的整个顺序（只有某些应用需要再起）。DNP 的应用程序可以使它自己复位而无需使其它的子系统或过程复位。这个功能典型地用以使一个外站的应用程序初始化它的组态并清除掉存于其本地缓存的事件。

|    |       |
|----|-------|
| AC | FC=14 |
|----|-------|

图 4-26 主站热再起动的控制报文

|    |    |     |                        |                   |
|----|----|-----|------------------------|-------------------|
| AC | FC | IIN | Object Header for time | Time Delay Object |
|----|----|-----|------------------------|-------------------|

图 4-27 外站对热再起动的响应

外站的响应等同于对冷再起功能码之响应，并应按同样方式解释。“时间延时对象”(Time Delay Object)实际上是一个“精时延”或“粗延时”对象（Time Delay Fine or Time Delay Course）。

4. 16 初始化数据（功能码 15）

该功能码规定将可组态的数据置入初始的或系统预置的置定值。例如，此功能可用于清除计数器。注意初始置定值不被包含在此请求中。

|    |       |                                              |
|----|-------|----------------------------------------------|
| AC | FC=15 | Object Header For data objects to initialize |
|----|-------|----------------------------------------------|

图 4-28 主站初始化数据控制报文

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4-29 外站对初始化请求的响应

响应仅指明所请求的操作成功还是失败

4. 17 初始化应用程序（功能码 16）

该功能码规定在外站的应用程序进入执行的准备。

1    2    3            4            5            6            7.....            字节

|    |       |                                     |                               |                        |                                   |                                     |
|----|-------|-------------------------------------|-------------------------------|------------------------|-----------------------------------|-------------------------------------|
| AC | FC=16 | Object GV<br>Group=G<br>Variation=V | Qualifier<br>0000 101<br>1 11 | Range<br>Quantity<br>1 | Application<br>Identifier<br>Size | Application<br>Object<br>Identifier |
|----|-------|-------------------------------------|-------------------------------|------------------------|-----------------------------------|-------------------------------------|

图 4—30 主站初始化应用程序的控制报文

- 对象的组与变体必须指定一个应用的控制报文。
- **Q**—码指明 **Range** 段是一个 **8** 位的量，这个量指定了后随的对象识别码对象。
- 应用程序识别码的规模段指明后随的“应用对象识别码”之规模。

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—31 在初始化应用程序之后，外站的响应

外站收到请求后应先初始化所指定的应用，然后以处理成功或失败作为响应。

#### 4. 18 起动应用程序（功能码 17）

该功能码被应用以起动所指定的应用程序于外站。

|    |       |                                     |                                 |                        |                                   |                                     |    |
|----|-------|-------------------------------------|---------------------------------|------------------------|-----------------------------------|-------------------------------------|----|
| 1  | 2     | 3                                   | 4                               | 5                      | 6                                 | 7.....                              | 字节 |
| AC | FC=17 | Object GV<br>Group=G<br>Variation=V | Qualifier<br>0001 10 11<br>1 11 | Range<br>Quantity<br>1 | Application<br>Identifier<br>Size | Application<br>Object<br>Identifier |    |

图 4—32 主站起动应用程序的控制报文

- 对象组与应用程序必须指定一个应用程序的识别码对象。
- **Q**—码指明 **Range** 段是一个 **8** 位的数量，用以指定了后随的对象识别码对象之数量。
- 应用程序识别码的规模段指明后随的“应用对象识别码”之规模

|    |    |     |                              |                      |
|----|----|-----|------------------------------|----------------------|
| AC | FC | IIN | Object<br>Header<br>for time | Time Delay<br>Object |
|----|----|-----|------------------------------|----------------------|

图 4—33 外站在起动了应用程序后的响应

收到请求的外站应起动所指定的应用程序，然后将处理的成功或失败作为响应。

#### 4. 19 停止应用程序（功能码 18）

该功能码通知外站停止执行所指定的应用程序。

|   |   |   |   |   |   |        |    |
|---|---|---|---|---|---|--------|----|
| 1 | 2 | 3 | 4 | 5 | 6 | 7..... | 字节 |
|---|---|---|---|---|---|--------|----|

|    |       |                                     |                                |                        |                                    |                                     |
|----|-------|-------------------------------------|--------------------------------|------------------------|------------------------------------|-------------------------------------|
| AC | FC=18 | Object GV<br>Group=G<br>Variation=V | Qualifier<br>0001 1011<br>1 11 | Range<br>Quantity<br>1 | Appplication<br>Identifier<br>Size | Application<br>Object<br>Identifier |
|----|-------|-------------------------------------|--------------------------------|------------------------|------------------------------------|-------------------------------------|

图 4—34 主站停止应用程序的控制报文

- 对象的 **G5V** 必须指定一个应用程序的识别码对象。
- **Q**—码指明 **Range** 段为一个 **8** 位的数量用以指定后随的对象识别码对象之数量。
- 应用程序识别码的规模段指明后随的“应用对象识别码”之规模。

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—35 外站在停止了应用程序之后的响应。

外站收到请求后应停止所指定的应用程序，然后以处理成功或失败作为响应。

#### 4. 20 保存组态（功能码 19）

该功能码通知外站停止执行所指定的应用程序。

|    |       |                                     |                                |                        |                                   |                          |    |
|----|-------|-------------------------------------|--------------------------------|------------------------|-----------------------------------|--------------------------|----|
| 1  | 2     | 3                                   | 4                              | 5                      | 6                                 | 7.....                   | 字节 |
| AC | FC=19 | Object GV<br>Group=G<br>Variation=V | Qualifier<br>0001 1011<br>1 11 | Range<br>Quantity<br>1 | File<br>Identifier<br>Object Size | File<br>Object<br>Object |    |

图 4—36 主站保存组态文件的控制报文

- 对象的 **G** 与 **V** 必须指定一份组态或文件的识别码对象。
- **Q**—码指明段 **Range** 是一个 **8** 位的数量，用以规定后随的对象识别码的数量。
- 应用程序识别码的规模段指明后随的“应用对象识别码”之规模。

|    |    |     |                  |                |
|----|----|-----|------------------|----------------|
| AC | FC | IIN | Object<br>Header | Time<br>Object |
|----|----|-----|------------------|----------------|

图 4—37 外站在保存了组态文件后的响应

收到请求的外站应该保存所指定的组态文件，然后将处理的成功或失败和一个时间作为响应。时间响应是（Time Delay Fine 或 Time Delay Coarse）表明外站可以再通信的时间，在这段时间内主站不应对此外站通信。

#### 4. 21 允许自发的报文（功能码 20）

该功能码通知外站可以允许对象标题中所规定的对象作自发的（非请的）报告。

|    |        |               |
|----|--------|---------------|
| AC | FC =20 | Object Header |
|----|--------|---------------|

图 4—38 主站请求开放自发的报文

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—39 外站的响应。

外站将允许所有被指定于对象标题内的对象（类型或点）之自发报文，主站也可以发一个对象标题规定数据的类（class data）。这意谓着任何组态于所指定类别的对象将被允许发自发的报文。

#### 4. 22 停用自发的报文

该功能码通知外站对对象标题中所规定的对象停止其自发的报告。

|    |        |               |
|----|--------|---------------|
| AC | FC =21 | Object Header |
|----|--------|---------------|

图 4—40 主站请求停止自发的报文

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—41 外站对停止自发报文的响应。

外站将对象标题中所规定的所有对象（类型或点）停止其自发报文，主站也可以在对象标题中规定 class data。这意谓着任何对象凡被组态于所指定之 class 将被停止发其自发报告。

#### 4. 23 级别分配（功能码 21）

|    |       |                     |                    |                      |
|----|-------|---------------------|--------------------|----------------------|
| AC | FC=22 | Class Object Header | Data Object Header | Data Object Header n |
|----|-------|---------------------|--------------------|----------------------|

该功能被用以对数据分配等级（class）。

图 4—42 主站请求对数据分配等级的报文

- 等级对象的标题（**class object header**）必需指定等级对象组和一个在 **1** 与 **3** 之间的变体，表明对后随的所有数据对象（由标题所指定的）之级别分配。

- 数据对象标题标识了要按等级标题（前置于数据对象标题者）分配等级的对象之 **G**, **V** 和 **Range**。

- 在一份报文内可以有多组等级对象（**Class Object**）的标题随之以一个或多个数据

对象标题 (**Data Object header**)。然而每一个集必需不跨及多个分段 (**tragment**)。

- 静态数据对象被分配于 **class0**，不可分配到其它的等级，事故数据对象被分

|    |    |     |
|----|----|-----|
| AC | FC | IIN |
|----|----|-----|

图 4—43 外站对等级分配的响应

配至 **classes1, 2** 与 / 或 **3**，并且不得分配于 **Class0**。

外站的响应表明操作的成功或失败（成败决定于 **IINn** 个位的状态）

4. 24 延时测量（功能码 23）

本功能被用以计算某一指定站的通信延时，通常它被用来为外站作时间整步（这个过程  
的细节详见第 6 章：时间同步）。

|    |       |
|----|-------|
| AC | FC=23 |
|----|-------|

图 4—44 主站请求起动时间测量

- 在任何一次独立的请求中只有一个时间对象。

|    |    |     |                           |                           |
|----|----|-----|---------------------------|---------------------------|
| AC | FC | IIN | Time Delay<br>Fine Header | Time Delay<br>Fine Object |
|----|----|-----|---------------------------|---------------------------|

图 4—45 外站对延迟测量请求的响应

外站以“精延时”对象去响应，该请求陈述在收到请求的第一字节之第一毕特至发送响  
应之第一字节的第一毕特之间相距的毫秒时间。

## 第五章：等 级

---

对于对象可以安排一个等级，数据可分四个等级，0 级留给（静态数据反映外站数据之当前值），1、2 和 3 留给事件数据对象（由于外站内的数据变化或某些其它的刺激所生成的对象）。每个事件对象可被分配至 1，2 或 3 级。对象可以按优先级（这种优先级由用户判定的）分群纳入各级，并且数据的各级是以不同的周期召询（polled）的。

将数据分配至各级的能力和组态能力的程度说明于外站设备的简介（），并不要求外站一定有数据分配于 1，2 或 3 级。

等级数据被主站用以在一种需求或可用性的基础上从一个外站请求事先安排好的数据对象。所以，一个等级数据对象的标题只能用于请求（没有伴随数据对象）去指明外站的哪些对象是要回送的。外站在响应中将回送“实际的”（ACTUAL）数据对象之对象标题而“不是”（NOT）等级对象的标题。

## 第六章：时间同步

时间同步是由应用层管理的，但是必需利用数据链路层的专门服务，应用程序必需发送适当的请求或响应以起动时间整步的顺序。

使主站与外站时间整步，必需遵循以下步骤：

1. 主站发送一个“延时测量”的请求给外站，主站记录下发送请求的第一字节第一毕特的时间（主站发送时间）。

2. 外站收到“延时测量”请求第一字节第一毕特的时间为 RTU 接收时间（这时外站的本地时间）。

3. 外站发送响应的第一字节第一毕特的时间为 RTU 发送时间，响应中包含了“时间延迟”对象（精延时或粗延时）。该对象内的时间为 RTU 的换向时间，即

$$\text{RTU Turn Around} = \text{RTU Send Time} - \text{RTU Receive Time}$$

4. 主站收外站响应之第一字节第一毕特的时间为主站接收时间。

5. 现在主站可以算出单次往返的传播延迟为：

$$D = \text{主站接收时间} - \text{主站发送时间} - \text{RTU 换向时间}$$

6. 主站现在发送一条“写”请求之第一字节第一毕特于时间（Master Send）。“写”请求包含“时间与日期”对象，其中的时间等于（Master Send+Delay）。这就是主站要求在外站设定的时间。

7. 外站收外站响应之第一字节第一毕特的时间为主站接收时间(RTU Receive)。

8. 外站将处理“写”请求，将处站的时钟设置于新 RTU 时间（New RTU Time）。采用以下算法：

$$\text{调整量 (Adjustment)} = \text{当前的 RTU 时间} - \text{新 RTU 时间}$$

$$\text{New RTU Time} = \text{在“Time and Date”对象内的时间} + \text{调整量}$$

9. 现在主站与外站的时间同步了。

注：

这个时间整步方案假定外站向主站的传播延时与主站向外站的传播延时是相等的。如属需要，主站可发一个全局性的请求（采用目的地 FFFFH），同时向全部外站整步，仅当全部传播途径的延时相等才行。

## 第七章：带有时间输入的二进制输入

---

当数字输入点变位时，外站将发送“带时间的二进制输入”或“带相对时间的二进制输入”。二进制输入的事件对象是以不同的格式发送的，取决于不同的条件。

1. 外站时间已整步，一个事件对象要送。数据以“带时间的二进制输入”的对象格式。
2. 外站时间已整步，几个事件对象要送，先发送公用的发生时间对象之时间与日期，随后发几个带有相同时间的二进制输入对象，在公用的发生时间对象内的时间是最老的一个对象，相对时间始于 0（即最老的对象）并相对于“日期和时间”对象向上扩伸。
3. 外站的时间未整步，一或多个事件要送，“未整步的公用时间和日期对象”先发，跟着发一个或多个带有相对时间的二进制输入对象，在“公用的发生时间之时间与日期对象”内的时间是最老的对象。相对时间始于 0（即最老的对象）并相对于“时间与日期”对象向上扩伸。

## 第八章： 文件传输

“文件识别对象（FIO）可用于外站与主站之间的文件传输，这个功能通常用于主站向 / 自外站写 / 读组态文件。

FIO 的功能性允许自中间的“数据集中器”向外站送组态信息，一个数据集中器处于主站与一个外站之间—它对于主站是个外站，对于外站它又是一个主站，注意“数据集中器”并非仅是一个通信节点—它不在主站与外站之间直接导送报文。

FIO 总是以“写”功能码的请求传递给一个外站，所要作的活动（读，写或其它）被规定于对象内的“文件功能”段，其响应总是用“响应”功能码，然而外站可以发送一份包含一个 FIO 的非请报文。

FIO 在它的“文件名”段内含有路由（routing）信息，该段描述对象以如何的路由来自主站，通过任何数目的中间“数据集中器”而取得路由。

### 8.1 执行写功能的文件识别对象

本节说明一个“文件识别对象”如何从一个主站通过一个“数据集中器”传递到一个外站。该外站也可以是另一个“数据集中器”，注意请求的报文使用着“写”功能码，对象的“文件功能”段将是“写”（WRITE），“挂起”（APPEND），“插入”（INSERT），“删除”（DELETE）或“擦除”（ERASE）。对象可以包含也可不包含数据（关于 DELETE 或 ERASE 是没有数据的）。

要用以下的术语：

外站应用软件—这是“数据集中器”内的应用软件用以对主站通信。相对于主站讲它是一个外站。

主站应用软件—这是“数据集中器”内的应用软件用以对外站通信，相对于外站讲是一个主站。

主站应用软件—这是“数据集中器”内应用软件用以对外站通信，相对于外站讲是一个主站。

主站将带有 FIO 的请求（“写”功能码）送到外站的应用程序”。关于这个请求，必需满足以下的条件：

- 在请求中使用 **DNP WRITE** 功能码。
- 对象的“文件功能”段被置以 **WRITE, APPEND, INSERT, DELETE 或 ERASE**。

• “文件名”段含有一个 **ASCII** 字符串，字符串的长度及内容由“数据集中器”决定之。  
**HARRIS** 公司的办法是用一个字符，它头上 9 个字符为“/DNPDCAXX”，其中的 XX（2 个 **ASCII** 字符）为该 **F10** 所必需路由（）到“主站应用软件”的号码。该集中器将送此信息至外站。

若上述条件行到满足，就将发生以下的顺序：

- 1) “外站应用软件”送一个“确认”响应给主站（若 **CON** 位是置位的）。
- 2) “外站应用软件”自“文件名”段的头上 9 个字符（关于 **HARRIS** 方法）移去。
- 3) “外站应用软件”将 **F10** 送给“主站应用软件”。
- 4) “主站应用软件”送一个含有 **F10** 的请求给适宜的外站。
- 5) 若要求有“确认”响应，“主站应用程序”就等待这个响应。
- 6) “主站应用软件”现在待候含有 **F10** 的响应，在响应内的对象含有规定于“文件功能段内的命令状态，当这个对象被收到时，“主站应用软件”将它送至“外站应用软件”。若“主站应用软件”收不到这个对象，就送一个否定确认给“外站应用软件”。
- 7) 收到 **F10** 响应或否定的确认（自“主站应用软件”），“外站应用软件”就送一个响应给主站，这个通信回合就完成了。

对请求的响应使用“响应”（**RESPONSE**）功能码，它含有“文件识别码”（**F10**）。该 **F10** 含有在“文件名”段内请求的操作之状态，不带数据记录，如果“外站应用软件”。收到来自“主站应用软件”的响应，该响应含有一个 **F10**。“外站应用软件”无需改变这个响应，如果“外站应用软件”收到一个来自“主站应用软件”的否定确认，它将修改并置以下的字段于响应的 **F10**。

- 起始记录=终止记录=0 （**START\_RECORD=END\_RECORD=0**）
- 文件规模=0 （**FILE\_SIZE\_0**）
- 文件功能=**RESPONSE** （**FILE\_FUNCTION=RESPONSE**）
- 允许=0 （**PERMISSION=0**）
- 状态= 2: “主站应用软件”不能将 **F10** 传送给“主站应用软件”。
- 4: 操作失败因为 **FILE\_ID** 所要的地址上设备不在线。
- 4: 操作失败因为从“主站应用软件”收到一个否定的确认。

“文件名”段是这样设计的名 **F10** 含有能自任何数目的中间数据集中器下装到一个外站的组态文件。

图 8—1 是一张极度简化了的例子，显示 **F10** 是如何通过两个数据集中器从一个主站传

递到一个外站的。

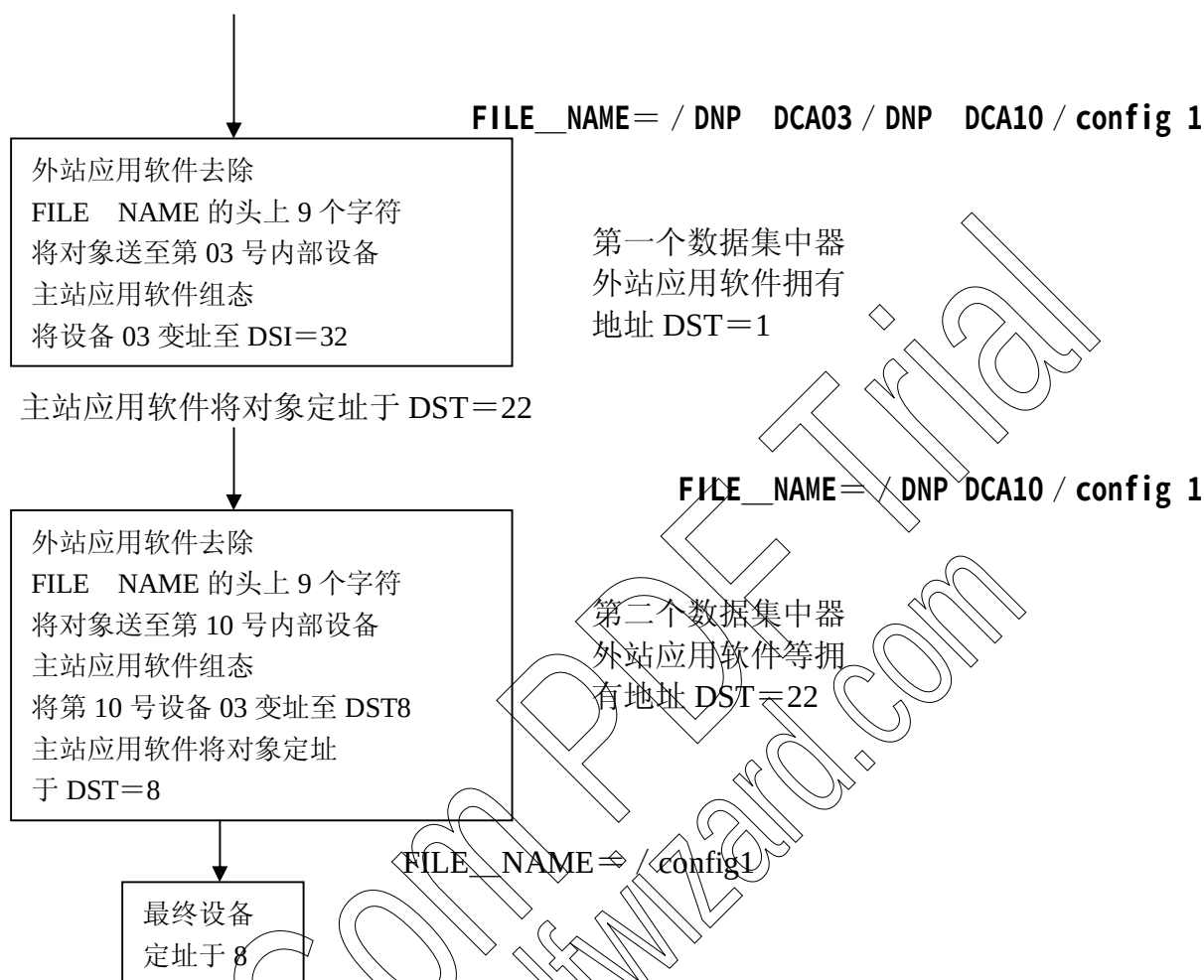


图 8—1 通过数据集中器传送一个文件识别对象

在图 8—1 中：

- 1) “主站将“文件识别对象 (FIO)”“写”入第一个数据集中器 (DC) , “文件名”段指定对象被送至第一个 DC 的第 3 号设备。
- 2) 在第一个 DC 内的“外站应用软件”去除“文件名”的头上的 9 个字符, 然后将对象通到 (routs) 作为 3 号设备的“主站应用软件”。
- 3) “主站应用软件”组态文件规定第 3 号对象的 DNP 地址为 22, 第一个 DC 内的“主站应用软件”将 FIO “写”入第二个 DC。
- 4) 第二个 DC 收到“写”请求。“文件名”段规定对象被送至第二个 DC 之第 10 号对象。
- 5) 在第二 DC 内的“外站应用软件”去除掉“文件名”的头九个字符。然后将对象开通至被规定为第 10 号设备的“主站应用软件”。
- 6) “主站应用软件”组态规定第 10 号对象拥有 DNP 目的址 8, 在第二个 DC 内的“主站

应用软件”将 FIO “写”入外站。

7) 外站接收 “写”请求，它用含有被修改的 FIO 之 “RESPONSE”去响应，该对象包含操作的状态，它被发送至第二个 DC 的 “主站应用软件”。

8) 第二 DC 之 “主站应用”将响应的 FIO 传送到 “外站应用”。

9) “外站应用”将含有 FIO 的 RESPONSE 传送给第一 DC。

10) 第一 DC 之 “主站应用”将 FIO 传给 “外站应用”。

11) 第一 DC 之 “外站应用”将含有 FIO 的 RESPONSE 发给 DNP 主站。

注意：

- 这种功能要求 DNP 主站比第一 DC 同内的 “外站应用”对象响应具有更大的逾时时限。
- DNP 主站必需有详细的组态和路由信息以便构建 “文件名”() 段。
- 一个 “外站应用”当它等待下游设备的响应时，它是不会收报的，它被 “锁定”于主站请求。

## 8. 2 文件识别对象 (FIO) 执行读功能

本节说明一个 FIO 是如何被用以执行读功能的，注意对象是通过使有 “”功能码之请求将对象传送至应用的。“文件名”段被置于 READ。

当下列条件满足时，主站可以读 FIO：

- 采用 DNP WRITE 功能码于请求中。
- 在请求中所收的 FIO，其 “文件名” (File\_Name) 段被置于 READ。
- “文件名”段内含有一个 ASCII 字符串，字符串的长度与内容取决于 DC。HARRIS 公司的办法用的字符串，头上 9 个字符是 “ / DNP DCA××”，其中×× (2 个 ASCII 字符) 含有 “主站应用”为该 FIO 所定的目的址号码。

该信息将对象通过 DC 开通给 (routes) “主站应用”，它又将对象自 DC 送至外站。

如果以上的条件满足了，就发生以下顺序：

- 1) “外站应用”送一个肯定的响应给 DNP 主站 (如果被要求如此)。
- 2) “外站应用”从 “文件名”段 (按 HARRIS 的办法) 上移去头上九个字符，按需要修改 FIO。
- 3) “外站应用”发送带有 FIO 的 READ 命令的给 “主站应用”。
- 4) “主站应用”送带有 FIO 的 READ 命令给适当的外站。
- 5) 若要求有肯定的响应，“主站应用”未收到对象，则发一个否定的确认给“外站应用”。

6) “主站应用”现在等待着含有所要求的 **F10** 之响应，当这些对象被收到时，“主站应用”就发送响应给“外站应用”。如果“主站应用”未收到对象，则发一个否定的确认给“外站应用”。

7) 收到了一个 **F10** 的响应或否定的确认时，“外站应用”送一个确认给主站，通信回合就完成了。

在以上的顺序中可能遇到某些出错的情况，在“外站应用不能将请求传给”主站应用”收到一个否定的确认情况下，“外站应用”就将所收到的 **F10** 作为请求的一部份回送给主站。

“外站应用”将修改或设置在响应的 **F10** 中之以下字段：

- **START\_RECORD=END\_RECORD=0**
  - **FILE\_SIZE=0**
  - **FILE\_FUNCTION=RESPONSE**
  - **PERMISSION=0**
  - **STATUS=2**：“外站应用”未能将 **F10** 传给“主站应用”。
- 4: 操作不成功因为由 **FILE\_ID** 选址的设备离线。
- 4: 操作不成功因为自“主站应用”收到一个否定的确认。

# 第四部分

## *DNP V3.00*

### 数据对象库

文件版本: 0.02

内部文号: P009—OBL

所配属的软件: DNP 版本 3.00

**Harris Controls** 公司的分布式自动化产品

# 目 录

---

|                                  |     |
|----------------------------------|-----|
| 关于此文件.....                       | 127 |
| 本文件的目的.....                      | 127 |
| 谁该用此文件.....                      | 127 |
| 帮助与附加文件.....                     | 127 |
| 本文件是如何组织的.....                   | 127 |
| 本文所使用的惯例.....                    | 128 |
| 概述.....                          | 128 |
| 第一章 对信息单元说明的规则.....              | 129 |
| 1.1 概述.....                      | 129 |
| 1.2 数据类型.....                    | 129 |
| 1.3 数据规模.....                    | 129 |
| 1.4 毕特的位置.....                   | 129 |
| 1.5 单元值.....                     | 130 |
| 1.6 复合单元.....                    | 130 |
| 1.7 序列单元.....                    | 130 |
| 第二章 通则.....                      | 130 |
| 2.1 库结构.....                     | 131 |
| 2.2 点号码.....                     | 132 |
| 第三章： 二进制输入对象的定义.....             | 135 |
| 3.1 单位二进制输入.....                 | 135 |
| 3.2 带 <b>status</b> 的二进制输入 ..... | 135 |
| 3.3 不带时间的二进制变位输入 .....           | 136 |
| 3.4 带变位时间的二进制输入.....             | 138 |
| 3.5 带有相对时间的二进制输入.....            | 139 |
| 第四章： 二进制输出对象的定义.....             | 141 |

|                                |     |
|--------------------------------|-----|
| 4.1 二进制输出.....                 | 141 |
| 4.2 二进制输出的质量状态.....            | 141 |
| 4.3 控制继电器的输出块.....             | 144 |
| 4.4 模式控制块 (PCB) .....          | 146 |
| 第五章：计数对象.....                  | 147 |
| 5.1 32 位二进制计数器 .....           | 147 |
| 5.2 16 位二进制计数器.....            | 148 |
| 5.3 32 位增值计数器.....             | 149 |
| 5.4 16 位增值计数器.....             | 149 |
| 5.5 32 位不带标志的二进制计数器.....       | 150 |
| 5.6 16 位不带标志的二进制计数器.....       | 150 |
| 5.7 32 位不带标志的增值计数器.....        | 151 |
| 5.8 16 位不带标志的增值计数器.....        | 151 |
| 5.9 32 位的冻结计数器.....            | 151 |
| 5.10 16 位的冻结的增值计数器.....        | 152 |
| 5.11 32 位冻结的增值计数器.....         | 153 |
| 5.12 16 位冻结的增值计数器.....         | 153 |
| 5.13 带冻结时间的 32 位冻结计数器.....     | 153 |
| 5.14 带冻结时间的 16 位冻结计数器.....     | 153 |
| 5.15 带冻结时间的 32 位冻结增值计数器.....   | 154 |
| 5.16 带冻结时间的 16 位冻结的增值计数器.....  | 154 |
| 5.17 无标志的 32 位冻结计数器.....       | 154 |
| 5.18 无标志的 16 位冻结计数器.....       | 154 |
| 5.19 不带标志的 32 位冻结的增值计数器.....   | 155 |
| 5.20 不带标志的 16 位冻结的增值计数器.....   | 155 |
| 5.21 不带标志的 32 位事件变化计数器.....    | 155 |
| 5.22 不带时间的 16 位事件变化计数器.....    | 156 |
| 5.23 不带时间的 32 位事件变化的增值计数器..... | 156 |

|                                |     |
|--------------------------------|-----|
| 5.24 不带时间的 16 位事件变化的增          | 156 |
| 5.25 带时间的 32 位事件变化计数器          | 156 |
| 5.26 带时间的 16 位事件变化计数器          | 157 |
| 5.27 带时间的 32 位事件变化增值计数器        | 157 |
| 5.28 带时间的 16 位事件变化增值计数器        | 157 |
| 5.29 不带时间的 32 位冻结计数器事件         | 159 |
| 5.30 不带时间的 16 位冻结计数器事件         | 159 |
| 5.31 不带时间的 32 位冻结的 Delta 计数器事件 | 159 |
| 5.32 不带时间的 16 位冻结的 Delta 计数器事件 | 160 |
| 5.33 带时间的 32 位冻结计数器事件          | 160 |
| 5.34 带时间的 16 位冻结计数器事件          | 161 |
| 5.35 带时间的 32 位冻结的 Delta 计数器事件  | 161 |
| 5.36 带时间的 16 位冻结的 Delta 计数器事件  | 161 |
| 第六章：模拟量输入对象的定义                 | 164 |
| 6.1 32 位模拟量输入                  | 164 |
| 6.2 16 位模拟量输入                  | 165 |
| 6.3 不带标志的 32 位模拟输入             | 165 |
| 6.4 不带标志的 16 位模拟输入             | 165 |
| 6.5 冻结的 32 位模拟输入               | 166 |
| 6.6 冻结的 16 位模拟输入               | 166 |
| 6.7 带冻结时间的 32 位冻结的模拟输入         | 167 |
| 6.8 带冻结时间的 16 位冻结的模拟输入         | 167 |
| 6.9 无标志 32 位冻结了的模拟输入           | 168 |
| 6.10 无标志 16 位冻结了的模拟输入          | 168 |
| 6.11 不带时间的 32 位模拟量变化事件         | 168 |
| 6.12 不带时间的 16 位模拟量变化事件         | 169 |
| 6.13 带时间的 32 位模拟量变化事件          | 169 |
| 6.14 带时间的 16 位模拟量变化事件          | 170 |

|                               |      |
|-------------------------------|------|
| 6.15 不带时间的 32 位冻结了的模拟量事件..... | 170  |
| 6.16 不带时间的 16 位冻结了的模拟量事件..... | 170  |
| 6.17 带时间的 32 位冻结了的模拟量事件.....  | 171  |
| 6.18 带时间的 16 位冻结了的模拟量事件.....  | 171  |
| 第七章 模拟输出对象定义.....             | 174  |
| 7.1 32 位模拟输出状态.....           | 174  |
| 7.2 16 位模拟输出状态.....           | 174  |
| 7.3 32 位模拟输出块.....            | 175  |
| 7.4 16 位模拟输出块.....            | 175  |
| 第八章 时间对象定义.....               | 177  |
| 8.1 时间与日期.....                | 177  |
| 8.2 带间隔的时间与日期.....            | 177  |
| 8.3 CT0 的时间与日期.....           | 178  |
| 8.4 非同步的 CT0 时间与日期.....       | 179  |
| 8.5 粗延时.....                  | 179  |
| 8.6 精延时.....                  | 180  |
| 第九章 级类对象的定义.....              | 181  |
| 9.1 0 类数据.....                | 181  |
| 9.2 1 类数据.....                | 181  |
| 9.3 2 类数据.....                | 182  |
| 9.4 3 类数据.....                | 182  |
| 第十章 文件对象定义.....               | 183  |
| 10.1 文件识别码.....               | 183  |
| 第十一章 设备对象定义.....              | 190  |
| 11.1 内部信号.....                | 190. |
| 11.2 存储对象.....                | 190  |
| 11.3 设备简表.....                | 191  |
| 11.4 保密登记对象.....              | 193  |

|                        |     |
|------------------------|-----|
| 11.5 保密登记的说明项.....     | 194 |
| 第十二章： 应用程序对象的定义.....   | 197 |
| 12. 1 应用程序之识别符.....    | 197 |
| 第十三章： 另一种数码对象的定义.....  | 198 |
| 13.1 短浮点.....          | 198 |
| 13.2 长浮点.....          | 200 |
| 13.3 扩充的浮点.....        | 201 |
| 13.4 小包装的二进制编码十进制..... | 202 |
| 13.5 中包装的二进制编码十进制..... | 202 |
| 13.6 大包装的二进制编码十进制..... | 203 |
| 术语词汇.....              | 205 |
| 缩写的清单.....             | 206 |

# 关于此文件

---

## 本文件的目的

本文件规定了 **DNP** 信息单元或用于 **DNP** 应用层的数据对象之编码规范。对象的语法被规定得和每个对象的语义一样。在复合对象中，每个组成部分的语义均被说明。

## 谁该用此文件

任何程序员，操作员或工程师凡有志于为 **DNP** 应用层构筑应用信息对象者。

## 帮助与附加文件

**DNP V3.00 Data Link (P009-09D.DL)**

**DNP V3.00 Application Layer (P009-09D.APP)**

**DNP V3.00 Transport Layer (P009-09D.TF)**

## 本文件是如何组织的

本文件分为十三章，简述如下：

第一章：对信息单元说明的规则—包括构造与解释数据对象的规则。

第二章：基本规则—对管理每个当前定义的对象作出说明。

其余各章为每种类型的对象提供详细的定义

第三章：二进制输入对象的定义

第四章：二进制输出对象的定义

第五章：计数对象的定义

第六章：模拟输入对象的定义

第七章：模拟输出对象的定义

第八章：时间对象的定义

第九章：级类对象的定义

第十章：文件对象的定义

第十一章：设备对象的定义

第十二章：应用对象的定义

第十三章：另一种代用数码对象的定义

还有术语名词表和缩写名词表

## 本文件所使用的惯例

在毕特位置号码方面，本文件偏离了 IEC 的惯例。毕特的位号自 0 编至 n，以 0 为右顶端而以 n 为左底端。

## 概述

采用 DNP 应用层规约的智能化设备可以监视、控制与/或既在软件方面又在硬件方面产生大量的数据。这些数据被称之为“信息单元”(information elements) (IEC870—5—3: 应用数据的基本结构)，被作为信息对象处理和存储，并且为了传输可作为“应用数据单元”而分组打包。所有的设备提供所存的信息单元作为同样的信息对象，本文件中描述这种格式。

本文件可以被修订，根据必要可增加新的信息单元或对象，但应得到“DNP 用户集团”的授权。

# 第一章：对信息单元说明的规则

## 1. 1 概述

本章为信息单元的说明介绍其基本规则，这些规则来自 IEC TC5.7 870 系列的标准和草案。这种规则为描述与表现数据提供毫不含糊的方法，不论它的数据源是什么，设备的简要文件（**profile documents**）则用以为每个远动设备的数据对象指明其确切的数据源及意义。

## 1. 2 数据类型(Data Types)

所有的数据可以按它最基本的形式作为一种“数据类型”来描述。数据的类型被认为是在大多数语言编译中的标准结构。

DNP 信息单元使用的结构如同它的说明基础一样也被 IEC870—5—4 所支持。表 1—1 列出可用的数据类型及其意义。

| 数据类型       | 符号 | 意义     |
|------------|----|--------|
| 1. 无符号整数   | UI | 正整数    |
| 2. 整数      | I  | 正或负整数  |
| 3. 无符号的定点数 | UF | 正的定点数  |
| 4. 定点      | F  | 正或负定点数 |
| 5. 实数      | R  | 正或负浮点数 |
| 6. 位串      | BS | 独立位之组合 |
| 7. 字节串     | OS | 字节之组合  |

表 1—1 数据类型

## 1. 3 数据规模 (Data Size)

每个数据单元是由一个数据类型和一个数据规模所组成。数据规模 **L** 是设在数据类型符号之后的，它是规定数据段长度的基数，其单位为毕特或字节，一个例子如下：

**BS12** 是指有一个 12 毕特的位串。

## 1. 4 毕特的位置 (Bit Position)

信息对象是若干信息元之组合，在定义信息对象时，各个毕特的位置是可以注明的。一个数据规模为 **i** 的特定数据字段，其毕特位置系以方括号 [**P1---Pn**] 标明，其中 **P1** 和 **Pn**

表示字段的第一个和最后一个毕特。“毕特的次序”示于表 1—2，如下：

| <b>BITS 毕特</b> | <b>7</b>      | <b>6</b>    | <b>5</b>    | <b>4</b>    | <b>3</b>    | <b>2</b>    | <b>1</b>    | <b>0</b>    |
|----------------|---------------|-------------|-------------|-------------|-------------|-------------|-------------|-------------|
| 字节             | 数据规模 <b>j</b> |             |             |             |             |             |             |             |
| <b>1</b>       | <b>7</b>      | <b>6</b>    | <b>5</b>    | <b>4</b>    | <b>3</b>    | <b>2</b>    | <b>1</b>    | <b>0</b>    |
| <b>2</b>       | <b>15</b>     | <b>14</b>   | <b>13</b>   | <b>12</b>   | <b>11</b>   | <b>10</b>   | <b>9</b>    | <b>8</b>    |
| .              |               |             |             |             |             |             |             |             |
| .              |               |             |             |             |             |             |             |             |
| <b>j</b>       | <b>8j-1</b>   | <b>8j-2</b> | <b>8j-3</b> | <b>8j-4</b> | <b>8j-5</b> | <b>8j-6</b> | <b>8j-7</b> | <b>8j-8</b> |

表 1—2 毕特的位置

### 1. 5 单元值 (Element Value)

如若可行，则所说明的数据段之值，其选定的变程及选定的码型被标注于尖角括号之内：**<V1—Vn Code>**。通常这是用允许值之变程和所用的码型 (**code**) 来说明的。码型这一项取：**(BIN)** 为二进制码，**(BCD)** 为二十进制码，**(ASCII)** 为 **ASCII** 码，等等。如果没有用码型这一项，则系统预定为二进制码。

### 1. 6 复合单元 (Compound Elements)

复合的数据段由位置连续的不同数据段所组成。复合数据段由大括号内列出的各数据段所说明。各数据段或用逗号分开或被于一列中。一份后随的清单说明各数据段的数据类型、规模、位分配和功能性的目的。第一个被说明的数据段自 **0** 位开始，其它的段采用连续下去的位分配。：

信息元: **(information element) = Cpi {data field1,data field2,...}**

或 **{data field1 = data type1 size i1[0,...i1-1] = function1**

**data field2 = data type2 size i2[i1...i1+i2-1] = function2**

**etc.}**

### 1. 7 序列单元 (Sequence Elements)

序列数据段是由不同的数据所组成的信息单元。序列数据段被作为复合数据段来说明的，然而每个字段的码位均以 **0** 作开始。

**Information Element =SQi{data field 1,data field2 . . . }**

或**{data field size j1}[1---j1]=function 1**

**data field size j1 [1---j1]=function 2**

**etc.**

## 第二章：通 则

本章将说明应用于 **DNP** 数据对象的通则，这些规则应用于所有现行的对象（有注释处除外）和今后的对象。

### 2. 1 库结构(Library Structure)

**DNP** 应用层拥有 **8** 位的变体段(**variation**)用以注明数据对象，**8** 位的对象段注明数据的基本类型，例如静态的二进制数。该对象的变体段说明同一数据的表现方式，例如对象的规模或该对象有无标志(**flay**)信息。

在每一个数据类型中通常四个不同的类，现概述于下：

- 静态对象：这种对象反映现场点或软件点上的的当前值。
- 事件对象：这种对象的产生被作为数据变化的结果或某种激励的结果。这些是历史对象，反映着过去某一时间的数据值。
- 冻结的静态对象：反映冻结的现场点或软件点的当前值，数据冻结是数据冻结请求的结果。
- 冻结了的事件对象：这种对象的发生是冻结数据变化或冻结某些其它的激励之结果，这些是反映在过去某个时间发生变化的数据之值。

每一类应用一个不同的对象来表现。个别数据类型 (**type**) 的所有级类 (**classes**) 也应被组织进同一范围的对象编号。迄今，关于传统的 **SCADA / DA** 之数据类型已产生以下的组别 (**groupings**)。

#### 二进制输入

二进制输入组包含表现二进制（状态或布尔数）输入信息的所有对象。**1** 至 **19** 号对象为这组对象而保留的。

#### 二进制输出

二进制输出组包含表现二进制输出信息的所有信息，**10** 至 **19** 号对象为这组对象保留的。

#### 计数值 (Counters)

计数值组包含表现计数值的所有对象，**20** 至 **29** 号对象为这些对象而保留的。

#### 模拟输入

模拟输入组包含表现模拟信息的全部对象。**30 至 39** 号对象为这些对象保留的。

## 模拟输出

模拟输出组合含表现模拟输出信息的所有对象。**40 至 49** 号对象为这种对象的。

## 时间 (time)

时间组包含用任何分辨率，以绝对方式或相对方式表现时间的所有对象。**50 至 49** 号对象为这种对象的。

## 级类 (class)

级类包含表现数据级别或数据优先级的全部对象，**60 至 69** 号对象为这种对象保留的。

## 文件(Files)

文件组包含表现文件或一个文件系统的全部对象，**70—79** 号对象是为这种对象保留的。

## 设备 (Devices)

设备组包含表现设备（而不是点）信息的所有对象。**90 至 99** 号对象是为这种对象保留的。

## 应用 (Applications)

应用程序包含表现软件应用或操作系统过程的所有对象。**90 至 99** 号对象是为这种对象保留的。

## 代用的数码 (Alternate Numeric)

代用的数码群包含表现代用的或定制的数码表示法之所有对象，**100 至 109** 号对象为这种对象保留的。

## 日后扩充 (Future Expansion)

日后扩充组是为 **DNP** 规约母后的或定制的扩充而保留的。**110 至 254** 号是为这种对象保留的。

## 保留 (Reserved)

**0 至 255** 号对象被永久性保留起来，不可用以指示任何 **DNP** 对象。使用这两个对象号的应用与日后 **DNP** 的版本可能不兼容。

## 2. 2 点号码 (Point Numbering)

以下的规则用在对于有关对象 (**Obj**) 与变体 (**Var**) 的对象号码 (**DNP** 应用层字段) 之解释。

### 守则 1:

**Obj X,Var Y**的 **Point i** 与 **ObjX, Var Z**之 **Point i** 表示同一个物理点, 其中 **Y** 与 **Z** 都是 **Obj X** 的变体。

**例:** 一个设备拥有 **16** 个在运行中的计数器 (对象 **20** 号), 计数器之编号为 **0** 至 **15**, 点 **5** 可以用 **4** 种不同的方式去询问。

- **Obj20 ,Var1,range5** 以 **32** 位的格式回送 **5** 号计数器的运行值。
- **Obj20 ,Var2,range5** 报告的信息同前, 只是用 **16** 位格式。
- **Obj20 ,Var3,range5** 回答是 **5** 号计数器自上一次报告之后所计算的值, 仍用 **32** 位格式。
- **Obj20 ,Var4,range5** 所报告的信息同上, 只是用 **16** 位格式。

#### 守则 2:

**ObjX** 之 **Point i** 可以用多种变体之一 (即, 它可以是一个 **16** 位的或 **32** 位的计数器) 报数。当作为一个事件报告时, **Point i** 可用该对象的为变体之一回送。究竟用哪个变体回送是一个应用的具体决策, 然而一个应用应该为每个事件仅用任一变体报告 “一” 事件对象。当响应一个对级类数据 (**Class dat**)

或对 **Obj X** 之 **Var0** 之请求时, 应该仅用所回送对象的一个变体。

#### 守则 3:

在同一组 (**grouping**) 内不同对象内的 **Point i** 不必一定是唯一的, 然而在二进制输入, 二进制输出, 模拟输出和计数器组内, 就要实施以下的规则。

- (a) 在静态对象内的 **Point i** 和事件对象内的 **Point i** 是同一个物理点。
- (b) 被冻结对象内的 **Point i** 与被冻结的事件对象内的 **Point i** 是同一个物理点。

**例:** 关于二进制输入, 在 **Obj1 var1** 与 **2** 的 **point i** 和 **Obj2 var1, 2** 与 **3** (静态的与事件的相关) 之 **Point i** 是同一个点。关于计数器, 在 **Obj20 var1, 2, 3** 与 **4** 的 **point i** 和 **Obj22 var1, 2, 3, 4, 5, 6, 7** 与 **8** (静态的与事件的相关) 之 **Point i** 是同一个点。

此外, 在 **Obj21 var1, 2, 3, 4, 5, 6, 7** 与 **8** 的 **point i** 和 **Obj23 var1, 2, 3, 4, 5, 6, 7** 与 **8** (冻结与被冻结的静态相关) 之 **Point i** 是同一个点。

#### 注:

**Obj 20** 内的 **point i** 与 **Obj 21** 的 **point i** “不”必要是同一个点。在被冻结的与非冻结的对象之间没有直接的关联。

#### 守则 4:

当使用一个要用，一个点号的报文格式时，若按定义或由于设备的限制每个对象只有一个点或是不需要点号的地方则对象组 (**groupings**) 应该用范围号 **0** 或数量等于 **1** (**range number 0 or quantity equal to 1**) .

DocuCom PDF Trial  
www.pdfwizard.com

### 第三章：二进制输入对象定义

本章规定二进制输入数据对象使用第一章所建立的规则。

#### 3. 1 单位二进制输入(Single bit binary input)

**Data Object 01—Variation:01 Type:Static**

说明:

“单位二进制输入”对象系用以表示一个数字输入点（硬件或软件）的状态（**state**）。

对象编码:

位串 **BS1** [0..0]

状态 **State** = **BS1[0]** <0,1 BIN>

(位串 **1** 之码位 (**bit position**) 在 **0**, 所选定的变程为二进制的, **1**。)

叙记:

单位二进制输入的表示法系用以在一个压缩格式中传送二进制输入状态。数据对象的传输总是以整字节去实现的。字节中未占用的码位均置成零。以下是 **n** 个这种数据对象压缩格式之例。(BS[1, 2.....])

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 7  | 6  | 5  | 4  | 3   | 2   | 1   | 0   |
| 15 | 14 | 13 | 12 | 11  | 10  | 9   | 8   |
|    |    |    |    |     |     |     |     |
| 0  | 0  | 0  | n  | n-1 | n-2 | n-3 | n-4 |

注:

本章所述的变体 (**variation**) 并不包含点的质量状态 (**status**) 的信息。例如在线, 再起动等等都是 **status** 的二进制输入不是本章所述的变量。采用单位二进制输入的变量就隐含上此点是在线的且其它的 **status** 位均已复位(即, 再起动, 通信断等等各位均复位)。

按 **1**. 在本文件中, 将 **state** 定义为运行的状态, 而将 **status** 定义为通常在工控中称为数据质量的状态, 具体内容见 **3.2** 节。

**2**. 在本文件中 **variation** 常有变量的涵义。

#### 3. 2 带有 Status 的二进制输入

**Data Object 01 – Variation: 02 Type: Static**

说明:

“带有 **status** 的二进制输入”对象系用以表现一个数字输入点（硬件或软件）的状态（**state**），并且也指出该点的质量状态（**status**），如下：

“**on-line**”(在线)位指出二进制点已被成功地读出。若该段被置于 **off-line**（离线），则该-数字点之状态可能是不正确的。

“**restart**”(再起动力)位指出发生此数据对象的现场设备当前正在再起动力之中，该设备可以是报告此数据对象的设备。

“**communication lost**”（通信失落）位指出报告此数据对象的设备失去了对此数据发生源的通信。

“**remote forced data**”（远方强置数据）位指出二进制输入的状态在一个非末端设备（**end device**）上被强置于它的当前状态。

“**local forced data**”（地强置数据）位指出二进制输入的状态在（**end device**）上被强置于当前的状态。

“**chatter filter**”(自动滤波)位指出二进制输入点已被滤波，以便去除点状态之非必要的变位信息。

“**state**”(状态)位指出二进制输入点之当前状态。

对象的编码：

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

BS8 [0...7]

|                    |        |                        |
|--------------------|--------|------------------------|
| On-line            | BS1[0] | <0,off-line;1,on-line> |
| Restart            | BS1[1] | <0,normal;1,restart..> |
| Communication lost | BS1[2] | <0,正常;1,失去>            |
| Remote forced data | BS1[3] | <0,正常;1 强置,>           |
| Local forced data  | BS1[4] | <0,正常;1,强置>            |
| Chatter filter     | BS1[5] | <0,正常;1, 滤波>           |
| Reserved           | BS1[6] | <0 >                   |
| State              | BS1[7] | <0,1 BIN>              |

### 3. 3 不带时间的二进制变位输入

Data Object 02-Variation: 01 Type: Event

说明：

“不带时间的二进制变位输入”对象表现一个数字输入点（硬件或软件）变化了的状

态并且也指明该点的质量状态（**status**）如下：

“**on-line**”(在线)位表明二进制点已被成功地读出。若这个字段被置于 **off-line**（离线），则该数字点之状态可能是不正确的。

“**restart**”(再起动)位表明发生此数据对象的现场设备已被再起动，该设备可以是报告此数据对象的设备。

“**communication lost**”（通信失落）位指出报告此数据对象的设备对此数据的发生源失去了通信。

“**remote forced data**”（远方强置数据）位表明二进制输入的状态在源设备已被强置于其当前的状态。

“**local forced data**”（本地强置的数据）位表明二进制输入的状态在报告此数据对象的设备上被强置于其当前的状态。

“**chatter filter**”(颤动滤波)位指出二进制输入点已被滤波，以便去除点状态之非必要的变位信息。

“**state**”(状态)位指出二进制输入点变位所变到的状态。

对象的编码：

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

位串 BS8 [0....7]

|                    |   |        |                         |
|--------------------|---|--------|-------------------------|
| On-line            | = | BS1[0] | <0,off-line;1,on-line>  |
| Restart            | = | BS1[1] | <0,normal;1,restart..>  |
| Communication lost | = | BS1[2] | <0,normal;1,lost>       |
| Remote forced data | = | BS1[3] | <0,normal;1forced,>     |
| Local forced data  | = | BS1[4] | <0,normal;1,forced>     |
| Chatter filter     | = | BS1[5] | <0,normal;1, filter on> |
| Reserved           | = | BS1[6] | <0>                     |
| State              | = | BS1[7] | <0,1 BIN>               |

叙记：

这种对象仅在当前值不同于上次所记录或上次所测得的值是时方才上报，如果用上了滤波器，它仅将在某一段时间周期内已保持恒值的新状态上报。

3. 4 带变位时间的二进制变位输入

Data Object 02—Variation:02 Type:Event

说明：

“带变位时间的二进制变位输入”系用以表示一个数字输入点（硬件或软件）的变位以及发生变位的时间，它也表现点的质量状态（**status**）如下：

|                       |         |
|-----------------------|---------|
| “on-line”位            | 同 3.3 节 |
| “restart”位            | 同 3.3 节 |
| “communication lost”位 | 同 3.3 节 |
| “remote forced data”位 | 同 3.3 节 |
| “local forced data”位  | 同 3.3 节 |
| “chatter filter”位     | 同 3.3 节 |
| “state”位              | 同 3.3 节 |

“**Time of Occurrence**”（发生时间）表示运动设备检出变位的绝对时间。这种时间的确度取决于各设备的的确度。

对象编码：

| 标志（Flag）                  |    |    |    |    |    |    |    |
|---------------------------|----|----|----|----|----|----|----|
| 7                         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 发生的时间（Time of Occurrence） |    |    |    |    |    |    |    |
| 7                         | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 15                        | 14 | 13 | 12 | 11 | 10 | 9  | 8  |
| 23                        | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| 31                        | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
| 39                        | 38 | 37 | 36 | 35 | 34 | 33 | 32 |
| 47                        |    | 45 | 44 | 43 | 42 | 41 | 40 |

两个数据段的顺序数据段 S02{Flag =BS8 [0...7]

Time of occurrence = UI48[0...47]<2<sup>48</sup>-1ms>}

其中 Flag ={BS8[0...7]

|                    |   |        |                         |
|--------------------|---|--------|-------------------------|
| On-line            | = | BS1[0] | <0,off-line;1,on-line>  |
| Restart            | = | BS1[1] | <0,normal;1,restaxt...> |
| Communication lost | = | BS1[2] | <0,normal;1,lost>       |
| Remote forced data | = | BS1[3] | <0,normal;1forced,>     |
| Local forced data  | = | BS1[4] | <0,normal;1,forced>     |
| Chatter filter     | = | BS1[5] | <0,normal;1, filter on> |

Reserved = BS1[6] <0>  
 State = BS1[7] <0,1 BIN>}

叙记:

“发生时间”系自 1970 年 1 月 1 日午夜，零点零分零秒零毫秒起始毫秒计算的。

### 3. 5 带有相对时间的二进制变位输入

#### Data Object 02- Variation:03 Type: Event

说明:

“带相对时间的二进制变位输入”系用以表示一个数字输入点（硬件或软件）的变位状态，以及状态变位的相对时间。它也指明该点的质量状态（**status**）如下：自 “On-line” 至 “**state**”同 3.3 节。

毫秒段（**MSEC**）指明远动设备（按：应是主站端）检出此变位的时间。这个时间的确度将取决于个别设备。

对象编码:

| Flag |    |    |    |    |    |   |   |
|------|----|----|----|----|----|---|---|
| 7    | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| MSEC |    |    |    |    |    |   |   |
| 7    | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| 15   | 14 | 13 | 12 | 11 | 10 | 9 | 8 |

SQ2{FLAG = BS8[0.....7]  
 MSEC = UI16[0.....15] <0.....2<sup>16</sup>-1>  
 FLAG = [BS8[0.....7]  
 On-line = BS1[0] <0,off-line;1,on-line>  
 Restart = BS1[1] <0,normal;1,restaxt...>  
 Communication lost = BS1[2] <0,normal;1,lost>  
 Remote forced data = BS1[3] <0,normal;1forced,>  
 Local forced data = BS1[4] <0,normal;1,forced>  
 Chatter filter = BS1[5] <0,normal;1, filter on>  
 Reserved = BS1[6] <0>  
 State = BS1[7] <0,1 BIN>}

叙记:

这个对象“必需”在 DNP 应用的信息中，有一个绝对的时间对象（公用时间对象，CTO）或一个失步的时间差 CTO。CTO 被用作以后所有“带相对时间的二进制变位输入”的一个绝对时基。（按：对一个 RTU 的变位输入讲只有一个 CTO），每个二进制输入对象的相对时间加上 CTO 绝对时间才能得出设备（按：应是 RTU 或其它现场设备）检出变位之绝对时间。

DocuCom PDF Trial  
www.pdfwizard.com

## 第四章 二进制输出对象

本章用第一章所建立的规则来定义二进制输出定义。

### 4. 1 二进制输出

#### Data Object 10—Variation :01 Type:Static

说明:

“二进制输出”对象系用以控制一个数字输出点（硬件或软件）。其状态（**state**）位用以指明对数字输出点所要求的逻辑状态。

对象编码:

BS1 [0..0]

State = BS1 [0] <0.1 BIN>

叙记:

数据对象的传输总是预先形成完整的字节（八位字），其中不占用的比特均被置零。以下就是将几个这种数据对象打包之例。

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 7  | 6  | 5  | 4  | 3   | 2   | 1   | 0   |
| 15 | 14 | 13 | 12 | 11  | 10  | 9   | 8   |
|    |    |    |    |     |     |     |     |
| 0  | 0  | 0  | n  | n-1 | n-2 | n-3 | n-4 |

### 4. 2 二进制输出的质量状态（status）

#### Data Object 10 - Variation: 02 Type: Static

说明:

“二进制输出的 **status**”对象系用以指明一个受控的数字点之当前状态以及那个点的 **status** 如下:

“on-line”（在线）位指明设备对二进制输出点的控制的控制正在正确地运行，对该点的二进制输出令可正确地工作。若该字段被置于“离线”，对该点的二进制输出令将不会成功。

“restart”（再起动）位表明产生此数据对象的现场设备已被再起动，该设备可以是上报此数据对象之设备。

“communication lost”（通信失落）位表明由于对受控设备的通信已经失落，该数字输

出点已经失控。

“**remote forced data**”(远方强置数)位表明数字输出点已受控于源方设备且其当前状态已示于 “**state**”.位。

“**local forced data**”(本地强置数)位表明数字输出点已受控于这个设备，且其当前状态（**state**）已示于 “**state**”.位。：

“**state**”(状态)位表明二进制输出点的当前状态。

对象的编码：

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|

BS8 [0...7]

|                    |          |                         |
|--------------------|----------|-------------------------|
| On-line            | = BS1[0] | <0,off-line;1,on-line>  |
| Restart            | = BS1[1] | <0,normal;1,restart...> |
| Communication lost | = BS1[2] | <0,normal;1,lost>       |
| Remote forced data | = BS1[3] | <0,normal;1forced,>     |
| Local forced data  | = BS1[4] | <0,normal;1,forced>     |
| Chatter filter     | = BS1[5] | <0,normal;1, filter on> |
| Reserved           | = BS1[6] | <0>                     |
| State              | = BS1[7] | <0,1 BIN>               |

#### 4. 3 控制继电器的输出块

Data Object 12 --Variation :01 Type:Static

说明：

“控制继电器的输出块信息”对象包括着数字输出的控制参数。这些参数规定了数字输出的型式与持续时间。

“控制码”段表明所要实现的控制功能。这种代码的可实用性取决于末端所用的设备之型式。

“计数”表明控制操作应该连续执行的次数。

“合时间”（**on-time**）段规定数字输出合闸的时间长（不一定用于所有的输出型式。

“分时间”（**off-time**）段规定数字输出分闸的时间长（不一定用于所有的输出型式。

对象编码：

|                    |   |   |   |   |   |   |   |   |             |
|--------------------|---|---|---|---|---|---|---|---|-------------|
| 控制码 (Control Code) | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | Contrd Code |
| 计数码 (Count)        | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |             |
| 合时间 (ON Time)      |   |   |   |   |   |   |   |   | Count       |
| 31                 |   |   |   |   |   |   |   | 0 |             |
| 分时间 (OFF Time)     |   |   |   |   |   |   |   |   | Status      |
| 31                 |   |   |   |   |   |   |   | 0 |             |
| 性质状态 (status)      | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |             |

数据序列 (Sequence of data)

SQ4 {Control code = BS8 [0...7]  
Count = UI8[0...7] <0...255> 无符号整数  
On-time = UI32[0...31] <0...2<sup>32</sup>-1,ms>  
Off-time = UI32[0...31] <0...2<sup>32</sup>-1,ms>  
Status = UI7 [0...6] <0...127>  
Reserved = UI1[7] <0...1>  
}

其中 Control code = {  
code = BS4[0...3] <0...15>  
Queue = BS1[4] <0,normal;1,requeue>  
Clear = BS1[5] <0,normal;1,clear>  
Trip /Close = BS2[6...7] <00,NUL;01,Closed;10,Trip>  
} 无 合闸 脱扣

叙记:

“Trip /Close”:在用一对脱扣与合闸继电器使现场的点上电（赋能）和断电（失能）之处，这个字段决定激活该系统中的哪个控制继电器。本字段的 **NUL** 值仅可用以激活现场的选点继电器而不能激活其脱扣或合闸继电器。在现场没有选\点继电器的系统中，**NUL** 值不能实现任何控制操作。在一个没有脱扣 / 合闸继电器的系统中，这个字段应该总是 **NUL**，用以表明一个正常的数控操作，其确切的控制点是原来知道的。本字段不支持同时既有脱扣又有合闸的属性，因为这是系统的非法操作。

“Count”（计数）：“Count”段决定控制执行多少次。如果 “Count”是 0 就不执行控制。当计数达到了 0，控制就完成了。

“**Code**”(代码): “控制块”能用于支持逐点排队控制的设备或具有其它控制机制的设备。在前者中, 任何控制命令应为所关心的特定点而排队。在后者中, 每个命令在那个点收到下一条控制命令之前就被执行直到完毕。

“**Queue**”(队列): 若“控制码”是 **NUL**, 则无控制操作被列入队列, 并且若。“**clear**”(清除)属性被置位则队列中所有的控制令, 包括正在运行中的控制, 均被清除。

当“控制功能”被执行完毕, 它就被从队列中去除掉。如果那个操作的功能块具有已置位的“**queue**”属性, 则此控制操作又被重排入队列(排在队列的后面)。

“**Clear**”(清除): 若控制操作的“**clear**”属性置位, 则全部控制操作被从队列中去除掉, 包括正在运行的控制, 但这个控制操作被完成了。

“**Code**”字段的意义和所要执行的操作由以下各项决定的:

0: **NUL** 操作。无规定的操作。仅 **R** 的属性被处理。

1: 脉冲 **ON**—一个(或多个)点在指定的 **on**—时间转变为 **on**, 然后在指定的 **off**—时间转变为 **off**, 并保持在 **off** 态。

2: 脉冲 **off**—一个(或多个)点在指定的 **off**—时间转变为 **off**, 然后在指定的 **on**—时间转变为 **on**, 并保持在 **on** 状态。

3: 栓锁于 **on**—使一个(或多个)点栓锁在 **on**。

4: 栓锁于 **off**—使一个(或多个)点栓锁在 **off**。

5—15: 未定义

“**Queue**”(队列): 当完成后, 将操作放在队列的背后。

“**Clear**”(清除): 撤消当前正在运行的操作并将紧挨着激活这次新操(若不是 **NUL**)之前, 对所影响之点, 所排入队列的操作去除掉。

在操作之后控制点的保留位可以判别控制输出是否为 **on**。

所请求的控制操作之成功或失败在“状态”(status)字段内回送, 该字段的意义决定如下:

0: 请求已收到, 已初始化或已排入队列。

1: 请求未收到, 因为“操作”(operate)是在“arm”定时器逾时后收到的。“arm”定时器是在一个点收到“选择”(select)令时起动的。

2: 先前没能适配的“选择”报文(即, 对于事先未用 **select** 报文使之“待发”(armed)的控制点发送了一条“操作”(operate)报文。)

3: 请求令未被收到, 因为在 **control** 请求 (或是 **select ,operate** 或 **direct operate**) 中存在格式上的错误。

4: 控制 (**control**) 操作不为此点所支持。

5: 请求令未被收到, 因为控制队列已满或该点已被激活,

6: 请求令未被收到, 因为控制的硬件有问题。

7—127: 未定义。

#### 4. 4 模式控制块 (Pattern Control Block ---PCB)

**Data Object 12—Variation :02 Type :Static**

说明:

**PCB** 对象包含的是模式型控制的数字输出之控制参数, 这些参数对每个所影响的点规定其数据输出之类型与持续时间。**PCB** 对象为报文中该对象的后续模式屏蔽 (**pattern mask**) 对象规定了所有的参数。包含在 **PCB** 内的这些参数。包含在 **PCB** 内的这些参数影响及紧跟在 **PCB** 对象之后的 **PCB** 对象, 直到报文中的下一个 **PCB** 为止。

这个对象和后随的多个模式屏蔽 (**pattern mask**) 对象之组合可以指定一些 (一个或多个) 控制操作之实现, 即在不同的点上实现相同的操作。所有这些控制必需并行地实现。这意谓着对 “控制继电器的输出块” (**control relay output block**) 讲, 所有的字段, 属性与参数都是相同的, 但排队属性除外。这就是说, 模式控制是打算再排队的。

对象码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |
| 31 |   |   |   |   |   |   | 0 |
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |

控制代码 (**Control cote**)

计数 (**count**)

**On** 时间

**Off** 时间

状态 (**status**)

## 第五章：计数器对象的定义

本章对于在第一章：〈为信息单元说明规则〉所建立的规则定义其所使用的计数器数据对象。

### 5.1 32 位二进制计数器

**Data Object20 ---Variation :01    Type:Static**

**说明：**

**32 位二进制计数器**表现一个累加值。该值可以是一个硬件或软件的点，累计其脉冲数或变位次数。

这个“数值”的字段表现计数器在报数时间的当前值或是来源设备的最近一次的所报值。这种数值无限制地递增，直到清零操作为止，清零后其值为零。

“标志”字段与先前的对象相同，带有涵义如下：

• “溢出”(roll-over)位置位时表示累计值已超过上次报出的可记录值( $2^{32}-1$ )。在溢出时计数值被置零，并且恢复常态计数。当计数值(与“溢出”态)都被报出后，“roll-over”位又被复位。

对象编码：

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |

标志 (Flag)  
数值

**SQ2{Flag =BS8 [0...7]**

**Value =U132[0...31]<0... $2^{32}-1$ >}**

**FLAG ={**

**在线 On-line    = BS1[0]**

**<0,off-line;1,on-line>**

**再起动 Restart    = BS1[1]**

**<0,normal;1,restart>**

**通信失败 Communication lost = BS1[2]**

**<0, normal;1,lost>**

**远方强置数 Remote forced data = BS1[3]**

**<0,normal;1,forced>**

**本地强置数 Local forced data    = BS1[4]**

**<0, normal;1,forced>**

**溢出 (翻滚) Roll-over    = BS1[4]**

**<0,normal;1,roll-over>**

**溢出 Roll-over    = BS1[5]**

**<0,normal;1,roll-over>**

```

保留 Reserved = BS1[6]          <0>
保留 Reserved = BS1[7]          <0>
}

```

## 5. 2 16 位二进制计数器

**Data Object20 ---Variation :02    Type:Static**

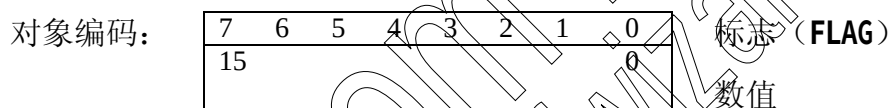
说明:

**16 位二进制计数器**表现一个累计值，该计数器可以自一个硬件或软件的点累计其脉冲数或变位的次数。

该“数值”段表现计数器在报数时当前值或是来自源发设备的最近一次的所报数。这种数值无限制地递增直到清零为止，清零后其值被复位于零。

“标志”字段与以前的对象相同，带有涵义如下:

•当“溢出”(roll-over)位置时表示累计值已超过上次报出的可记录的最大值( $2^{16}-1$ )。在溢出时，该计数器值已被复位于 **0**，并恢复常态的计数。当计数值(加上“溢出”状态)报出时这一位就被复位了。



**SQ2{FLAG = BS8 [0...7]**

**Value = U132[0...31]    <0...2<sup>32</sup>-1>**

**FLAG = {**

**On-line = BS1[0]    <0,off-line;1,on-line>**

**Restart = BS1[1]    <0,normal;1,uestart>**

**Communication lost = BS1[2]    <normal;1,lost>**

**Remote forced data = BS1[3]    <0,normal;1,forced>**

**Local forced data = BS1[4]    <0, normal;1,forced>**

**Roll-over = BS1[4]    <0,normal;1,roll-over>**

**Roll-over = BS1[5]    <0,normal;1,roll-over>**

**Reserved = BS1[6]    <0>**

**Reserved = BS1[7]    <0> }**

## 5. 3 32 位增值计数器 (32-bit Delta Counter)

**Data Object 20 —Variation :03 Type:Static**

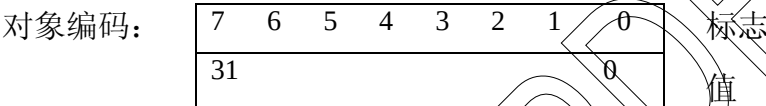
说明:

32 位的增值计数器表示自报出上一次的值以后所累计的值，这能够累计来自一个硬件或软件点的脉冲数或变位次数。

“数值”段显示计数器在报数的当前值或是最近一次来自源设备的所报值。这个数值递增到上报此点时为止，并被复位于Φ。

“标志”字段与先前的对象相同，带有涵义如下：

当 **roll-over** 位置位时，表示所累计的值已超过最大可能的记录值 ( $2^{32}-1$ )。该计数值在溢出时被复位于Φ，并使计数恢复正常。当计数器的值（加上 **roll-over** 状态）被上报后就使该 **roll-over** 位复位。



**SQ2{FLAG = BS8 [0...7]**  
**Value = UI32[0...31]**  $\langle 0 \dots 2^{32}-1 \rangle$   
**FLAG = {**  
**On-line = BS1[0]**  $\langle 0, \text{off-line}; 1, \text{on-line} \rangle$   
**Restart = BS1[1]**  $\langle 0, \text{normal}; 1, \text{restart} \rangle$   
**Communication lost = BS1[2]**  $\langle \text{normal}; 1, \text{lost} \rangle$   
**Remote forced data = BS1[3]**  $\langle 0, \text{normal}; 1, \text{forced} \rangle$   
**Local forced data = BS1[4]**  $\langle 0, \text{normal}; 1, \text{forced} \rangle$   
**Roll-over = BS1[4]**  $\langle 0, \text{normal}; 1, \text{roll-over} \rangle$   
**Roll-over = BS1[5]**  $\langle 0, \text{normal}; 1, \text{roll-over} \rangle$   
**Reserved = BS1[6]**  $\langle 0 \rangle$   
**Reserved = BS1[7]**  $\langle 0 \rangle$   
**}**

**5. 4 16 位增值计数器**

**Data Object 20 —Variation :04 Type:Static**

说明:

16 位的增值计数器表示自报出上一次的值以后所累计的值，它能够累计来自一个硬件或软件点的脉冲数或变位次数。

“数值”段显示计数器在报数的当前值或是最近一次来自源设备的所报的数值。这个数值递增到上报此点时为止，并被复位于Φ。

“标志”字段与先前的对象相同，带有涵义如下：

- 当 **roll-over** 位置位时，表示所累计的值已超过了最大可能的可记录值 ( $2^{16}-1$ )。该计数值在溢出时被复位于Φ，并使计数恢复正常。当计数器的值（加上 **roll-over** 状态）被上报后就使该 **roll-over** 位复位

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码： | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 15 |   |   |   |   |   |   |   | 数值 |

**SQ2{FLAG =BS8 [0...7]**

**Value =UI16[0...15]}**

**FLAG** (Same with that for 32-bit 增量计数器)

## 5. 5 32 位不带标志的二进制计数器

**Data Object 20 —Variation :05 Type:Static**

说明：

32 位的二进制计数器表现一个累加值，它可以累计来自一个硬件或软件点的脉冲数或变位次数。

数值“数值”段显示计数器在报数时的当前值或是来自源设备的最近一次的所报值。此数值无限制地递增直到计数器清零清零操作为止，清零后其值为Φ。

|       |    |   |
|-------|----|---|
| 对象编码： | 31 | 0 |
|-------|----|---|

**SQ2 { Value =UI32[0...31] <0...2<sup>32</sup>-1>}**

**注意：**这种变体用于输入点全是在线的且其它各标志位为“正常”(normal) 的场合（即，此变体暗含 **flag=1**）。

## 5. 6 16 位不带标志的二进制计数器

**Data Object 20 —Variation :06 Type:Static**

说明：全 32 位不带标志的二进制计数器。

|       |    |   |    |
|-------|----|---|----|
| 对象编码： | 15 | 0 | 数值 |
|-------|----|---|----|

**SQ2 { Value =UI16[0...15] <0...2<sup>16</sup>-1>}**

注意：全 32 位不带标志的二进制计数器。

## 5. 7 32 位不带标志的增值计数器

**Data Object 20 —Variation :07 Type:Static**

说明：

32 位的增值计数器表现自上次报出数值以后所新累计的数，它能够累计来自一个硬件或软件点的脉冲数或变位次数。

“数值”段显示计数器在报数时的当前值或是最近一次来自源设备的的所报值。这个数值递增到上报此点为止，并被复位为Φ。

**SQ2 { Value =UI32 [0...31]} <0...2<sup>32</sup>-1>**

注意：该变体用于输入点都是在线的，且其余所有标志位均为正常的场合（即该变体隐含 flag=1）

## 5. 8 16 位不带标志的增量计数器

**Data Object 20 —Variation :08 Type:Static**

说明：

（全 32 位不带标志的增量计数器）

对象编码： 

|    |  |   |
|----|--|---|
| 31 |  | 0 |
|----|--|---|

 数值

**SQ1 { Value =UI16 [0...15] <0...2<sup>16</sup>-1> }**

注意：（全 32 位不带标志的增量计数器）

## 5. 9 32 位冻结计数器（32-bit Frozen Counter）

**Data Object 21 —Variation :01 Type: Frozen static**

说明：

32 位的冻结计数器是一个复合的信息对象，它包含先前已被冻结的计数器之信息。

“冻结值”显示了源发设备执行了上一次的计数器冻结令时，计数器的内含数值。

“标志”段的意义与先前的对象相同。

对象编码： 

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |

 标志  
冻结值（Frozen Value）

**SQ2 {FLAG = BS8 [0...7]**

```

Frozen Value = UI32[0...31]          <0...232-1>
FLAG      = {
On-line   = BS1[0]                   <0,off-line;1,on-line>
Restart   = BS1[1]                   <0,normal;1,restart>
Communication lost = BS1[2]          <0,normal;1,lost>
Remote forced data = BS1[3]          <0,normal;1,forced>
Local forced data  = BS1[4]          <0, normal;1,forced>
Roll-over  = BS1[4]                   <0,normal;1,roll-over>
Roll-over  = BS1[5]                   <0,normal;1,roll-over>
Reserved   = BS1[6]                   <0>
Reserved   = BS1[7]                   <0>
}

```

## 5. 10 16 位的冻结计数器

**Data Object 21 —Variation :02 Type: Frozen Static**

说明:

**16 位**冻结的计数器是一个复合的信息对象，它包含先前已被冻结的计数器之信息。该计数器累计了来自一个硬件或软件点的脉冲数或变位次数。

“冻结值”显示了源发设备执行了上一次的计数器冻结令时的计数器的内含数值。

“标志”段的意义与先前的对象相同。

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 15 |   |   |   |   |   |   | 0 | 冻结值 |

**SQ2 {FLAG = BS8 [0...7]**

**Frozen Value = UI16[0...31] <0...2<sup>16</sup>-1>**

```

FLAG      = {
On-line   = BS1[0]                   <0,off-line;1,on-line>
Restart   = BS1[1]                   <0,normal;1,restart>
Communication lost = BS1[2]          <0,normal;1,lost>
Remote forced data = BS1[3]          <0,normal;1,forced>
Local forced data  = BS1[4]          <0, normal;1,forced>

```

Roll-over = BS1[4] <0,normal;1,roll-over>  
 Roll-over = BS1[5] <0,normal;1,roll-over>  
 Reserved = BS1[6] <0>  
 Reserved = BS1[7] <0>  
 }

## 5. 11 32 位的冻结的增值计数器 (32-bit Frozen Delta Counter)

**Data Object 21 —Variation :03 Type: Frozen Static**

说明:

32 位冻结的增值计数器表现自上次数据上报后所累计的冻结值，它可以累计一个硬件或软件点的脉冲数或变位次数。

“冻结值”显示在源发设备上执行上一次的计数器冻结令时的计数器的计数值。

“标志”段的意义与先前的对象相同。

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 31 |   |   |   |   |   |   | 0 | 冻结值 |

**SQ2 {FLAG = BS8 [0...7]**

**Frozen Value = UI32 [0...31] <0...2<sup>32</sup>-1>**

**FLAG** (同前一对象)

## 5. 12 16 位的冻结的增值计数器

**Data Object 21 —Variation :04 Type: Frozen Static**

说明: (同 32 位冻结的增值计数器)

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 15 |   |   |   |   |   |   | 0 | 冻结值 |

**SQ2 {FLAG = BS8 [0...7]**

**Frozen Value = UI16 [0...31] <0...2<sup>16</sup>-1>**

**FLAG** (同前)

## 5. 13 32 位带冻结的增值计数器

**Data Object 21 —Variation :05 Type: Frozen Static**

说明:

32 位带冻结时间的计数器是一个复合的信息对象，它包含了先前已被冻结的计数器之

信息。该计数器累计来自一个硬件或软件点的脉冲数或变位次数。

“冻结值”显示 “冻结时刻”的计数器之值。

“冻结时刻”段包含该对象被冻结的时刻，这个时间相应于 “冻结值”，所以该对象在 “冻结时刻”的当前值就是 “冻结值”。

“标志”段的意义与先前的对象相同。

|       |    |   |   |   |   |   |   |   |                       |
|-------|----|---|---|---|---|---|---|---|-----------------------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志                    |
|       | 31 |   |   |   |   |   |   | 0 | 标冻结值 (Frozen Value)   |
|       | 47 |   |   |   |   |   |   | 0 | 冻结时刻 (Time of Freeze) |

```
SQ3 {FLAG          = BS8 [0...7]
    Frozen Value    = UI32[0...31] <0...232-1>
    Time of Freeze  = UI 32[0...47] <0...248-1ms>
    FLAG           = {
    On-line        = BS1[0]          <0,off-line;1,on-line>
    Restart        = BS1[1]          <0,normal;1,restart>
    Communication lost = BS1[2]      <0,normal;1,lost>
    Remote forced data = BS1[3]      <0,normal;1,forced>
    Local forced data  = BS1[4]      <0, normal;1,forced>
    Roll-over        = BS1[4]        <0,normal;1,roll-over>
    Roll-over        = BS1[5]        <0,normal;1,roll-over>
    Reserved         = BS1[6]        <0>
    Reserved         = BS1[7]        <0>
    }
```

叙述:

“冻结时刻”系自 1970 年 1 月 1 日午夜 0 点，0 分，0 秒，0 毫秒起算，并以毫秒为计时单位。

5. 14 16 位带冻结时间的冻结计数器

Data Object 21 —Variation :06 Type: Frozen Static

说明:

(同 32 位带有冻结时间的冻结计数器)

|       |    |   |   |   |   |   |   |   |                       |
|-------|----|---|---|---|---|---|---|---|-----------------------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志                    |
|       | 31 |   |   |   |   |   |   | 0 | 标冻结值 (Frozen Value)   |
|       | 47 |   |   |   |   |   |   | 0 | 冻结时刻 (Time of Freeze) |

SQ3 {FLAG = BS8 [0...7]

Frozen Value = UI16[0...31] <0...2<sup>16</sup>-1>

Time of Freeze =UI 48 [0...4] <0...2<sup>48</sup>-1ms>

FLAG =}

叙述: (同带冻结时间的 32 位)

## 5. 15 带冻结时间的 32 位冻结的增值计数器 (32=bit Frof Freeze)

Data Object 21 —Variation :07 Type: Frozen Static

说明:

“带冻结时间的 32 位冻结的增值计数器”表现自上次报数后的一个被冻结了的累计值。该计数器能够累计来自是一个硬件或软件点的脉冲数或变位的次数。

“冻结值”显示在被冻结的时刻之计数器值。

“冻结时间”段包含该对象被冻结的时刻，该时间对应于“冻结值，所以“冻结值”就是该对象在“冻结时刻”的当前值。

“标志”段具有与先前的对象相同的意义。

|       |    |   |   |   |   |   |   |   |                       |
|-------|----|---|---|---|---|---|---|---|-----------------------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志                    |
|       | 31 |   |   |   |   |   |   | 0 | 标冻结值 (Frozen Value)   |
|       | 47 |   |   |   |   |   |   | 0 | 冻结时刻 (Time of Freeze) |

SQ4 {FLAG = BS8 [0...7]

Frozen Value = UI16 <0...2<sup>32</sup>-1>

Time of Freeze =UI 48 <0...2<sup>48</sup>-1>

}

FLAG } 同前

及叙述:

## 5. 16 带冻结时间的 16 位冻结了的增值计数器

Data Object 21 —Variation :08 Type: Frozen Static

说明: (同前)

|       |    |   |   |   |   |   |   |   |      |
|-------|----|---|---|---|---|---|---|---|------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志   |
|       | 15 |   |   |   |   |   |   | 0 | 标冻结值 |
|       | 47 |   |   |   |   |   |   | 0 | 冻结时刻 |

**SQ4 {FLAG = BS8 [0...7]**

**Frozen Value = UI16[0...15]**

**Time of Freeze = UI 48 [0...47]**

**}**

**FLAG** }  
及叙述: (同前)

## 5. 17 无标志的 32 位冻结计数器

**Data Object 21 —Variation :09 Type: Frozen Static**

说明:

“32 位带冻结计数器”是一个复合的信息对象，它包含有一个先前被冻结的计数器之信息。

“冻结值”显示在源发设备上执行上次计数器冻结令时的计数器之值。

|       |    |   |     |
|-------|----|---|-----|
| 对象编码: | 31 | 0 | 冻结值 |
|-------|----|---|-----|

**SQ2 { Frozen Value =UI32 [0...31] <0...2<sup>32</sup>-1>}**

注意:

该变体的用途所遵循的原则是输入点是在线的，其它所有的标志位处于常态（即此变体用于标志=1）

## 5. 18 16 位不带的标志的冻结计数器

**Data Object 21 —Variation :10 Type: Frozen Static**

说明:

“16 位冻结计数器”是一个复合的信息对象，它包含先前已冻结了的计数器之信息。计数器累计来自一个硬件或软件点的脉冲数和变位次数。

“冻结值”显示在源发设备上实现上一次计数器冻结令时的计数器之值。

|       |    |   |     |
|-------|----|---|-----|
| 对象编码: | 15 | 0 | 冻结值 |
|-------|----|---|-----|

**SQ2 { Frozen Value =UI16 [0...15] <0...2<sup>16</sup>-1>}**

叙记：（同前，即适用于标志=1）

### 5. 19 32 位不带标志的冻结了的增值计数器

**Data Object 21 —Variation :11      Type: Frozen   Static**

说明:

“32 位不带标志的的增值计数器”表现一个冻结值，它是自上次报数后所累的值。它能累计来自是一个硬件或软件点的脉冲数或变位的次数。

“冻结值”显示在源发设备上实现上一次的计数器冻结时的计数值。

|       |    |   |                |
|-------|----|---|----------------|
| 对象编码: | 31 | 0 | <del>冻结值</del> |
|-------|----|---|----------------|

**SQ2 { Frozen Value =UI32 [0...31] <0...2<sup>32</sup>-1>}**

**注意:**该变体的用法所遵循的准则是输入点为在线的且其它所有的标志位均为常态(即该变体用于标志=1)。

## 5. 20 不带标志的 16 位冻结了的增值计数器

**Data Object 21 —Variation :12      Type: Frozen Static**

说明：（同前）

对象编码: 15 冻结值

**SQ2 { Frozen Value = UI16[0...15] <0...2<sup>16</sup>-1>}**

注意：（同前）

### 5. 21 不带时间的 32 位事件变化计数器

**Data Object 22 — Variation 01 Type: Event**

说明:

“**32**位事件变化计数器”表现一个计数值,它是自上次报数后,已超过了所组态计数值。它能累计来自是一个硬件或软件点的脉冲数或变位的次数。

“当前值”字段显示产生事件的计数器值。

“标志”段与以前的对象相同

|       |    |   |   |   |   |   |   |   |                     |
|-------|----|---|---|---|---|---|---|---|---------------------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志                  |
|       | 31 |   |   |   |   |   |   | 0 | 当前值 (Current Value) |

**SQ4** {FLAG = BS8 [0...7]

Current Value = UI32 [0...31]  $\langle 0 \dots 2^{32}-1 \rangle$

}

FLAG (同 32 位二进制计数器)

## 5. 22 不带时间的 16 位事件变化计数器

Data Object 22 —Variation 02 Type: Event

说明: (同前者)

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |

SQ 4 {FLAG = BS8 [0...7]}

Value = UI16 [0...15]  $\langle 0 \dots 2^{16}-1 \rangle$

## 5. 23 不带时间的 32 位事件变化的增值计数器

Data Object 22 —Variation 03 Type: Event

说明:

“不带时间的 32 位事件变化的增值计数器”表示一个计数值，自上次报数后，它已超过了所组态计数值。它能累计来自是一个硬件或软件点的脉冲数或变位的次数。

“增加值”显示发生事件的计数器之值。

“标志”段与以前的对象相同

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |

标志

增加值 (Delta Value)

SQ 4{FLAG = BS8 [0...7]}

Current Value = UI32[0...31]  $\langle 0 \dots 2^{32}-1 \rangle$

FLAG (同 32 位二进制计数器)

## 5. 24 不带时间的 16 位事件变化的增值计数器

Data Object 22 —Variation 04 Type: Event

说明: (同前)

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |

标志

增加值 (Delta Value)

SQ 2 {FLAG = BS8 [0...7]}

Current Value = UI16[0...15] <0...2<sup>16</sup>-1>

FLAG (同 32 位二进制计数器)

## 5. 25 带时间的 32 位事件变化计数器

Data Object 22 —Variation 05 Type: Event

说明:

“带时间的 32 位事件变化计数器”表现一个计数值，它是自上次报数后，它已超过了所组态计数值。它能累计来自是一个硬件或软件点的脉冲数或变位的次数。

“数值”显示产生事件计数器之值。

“时间”段含有处理所发生事件的时间。

“标志”段与以前的对象相同

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 31 |   |   |   |   |   |   | 0 | 数值 |
|       | 47 |   |   |   |   |   |   | 0 | 时间 |

SQ4 {FLAG = BS8 [0...7]}

Value = UI32[0...31] <0...2<sup>32</sup>-1>

Time =UI48 [0...47] <0...2<sup>48</sup>-1ms>

FLAG = } (同带冻结时间的 32 位冻结计数器)  
及叙述:

## 5. 26 带时间的事件变化计数器 (16 位)

Data Object 22 —Variation 06 Type: Event

说明: (同前)

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 31 |   |   |   |   |   |   | 0 | 数值 |
|       | 47 |   |   |   |   |   |   | 0 | 时间 |

SQ4 {FLAG = BS8 [0...7]}

Value = UI32[0...15] <0...2<sup>16</sup>-1>

Time =UI 48 [0...47] <0...2<sup>48</sup>-1>

FLAG } (同带冻结时间的 32 位冻结计数器)

叙记:

## 5. 27 带时间的事件变化计数器 (32 位)

**Data Object 22 —Variation 07 Type: Event**

说明:

“带时间的事件变化增值计数器 (32 位)”表示一个计数器的增加值 (**delta**), 它已超过了所组态的死区 (**dead baud**)。它能累计一个硬件或软件点的脉冲数或变位的次数。

“数值”段显示产生事件变化的数值。

“时间”段含有处理所发生事件的时间。

“标志”段与以前的对象意义相同

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 31 |   |   |   |   |   |   | 0 | 值  |
|       | 47 |   |   |   |   |   |   | 0 | 时间 |

**SQ4 {FLAG = BS8 [0...7]**

**Value = UI32[0...31] <0...2<sup>32</sup>-1>**

**Time =UI 48 [0...47] <0...2<sup>48</sup>-1ms>**

**FLAG**

叙记: } (同带冻结时间的 32 位冻结计数器)

## 5. 28 带时间的事件变化增值计数器 (16 位)

**Data Object 22 —Variation 08 Type: Event**

说明: (同前)

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 15 |   |   |   |   |   |   | 0 | 数值 |
|       | 47 |   |   |   |   |   |   | 0 | 时间 |

**SQ4 {FLAG = BS8 [0...7]**

**Value = UI16[0...15] <0...2<sup>16</sup>-1>**

**Time =UI 48 [0...47] <0...2<sup>48</sup>-1>**

**FLAG**

叙记: } (同前)

## 5. 29 不带时间的 32 位事件变化计数器

## Data Object 23—Variation 01      Type: Frozen Event

说明:

“不带时间的 32 位冻结计数器事件”表现一个冻结了的计数值，该值被作为事件上报的，它能够累计来自一个硬件或软件点的脉冲数或变位的次数。

“冻结值”显示在冻结时的值。

“标志”段的意义与以往的对象相同

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 31 |   |   |   |   |   |   | 0 | 冻结值 |

SQ4 {FLAG = BS8 [0...7]}

Frozen Value = UI32[0...31] <0...2<sup>32</sup>-1>}

FLAG (同以往的 Object)

### 5. 30 不带时间的 16 位冻结 Delta 计数器事件

## Data Object 23—Variation 02      Type: Frozen Event

说明: 同前。

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 15 |   |   |   |   |   |   | 0 | 冻结值 |

SQ4 {FLAG = BS8 [0...7]}

Frozen Value = UI16[0...15] <0...2<sup>16</sup>-1>}

FLAG (同前)

### 5. 31 不带时间的 32 位冻结 Delta 计数器事件

## Data Object 23—Variation 03      Type: Frozen Event

说明:

“不带时间的 32 位冻结计数器事件”表现一个作为事件上报的冻结了的 Delta 计数器值，它能够累计来自一个硬件或软件点的脉冲数或变位的次数。

“冻结了的 delta 计数值”显示在冻结时的计数值。

“标志”段的意义与以往的对象相同。

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 31 |   |   |   |   |   |   | 0 |    |

冻结了的 **delta** 值

**SQ 4 {FLAG = BS8 [0...7]**

**Frozen Value = UI32[0...31] <0...2<sup>32</sup>-1>}**

**FLAG** (同前)

### 5. 32 不带时间的 16 位冻结 **Delta** 计数器事件

**Data Object 23—Variation 04 Type: Frozen Event**

说明：同前。

对象编码：

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |

标志

冻结了的 **delta** 值

**SQ4 {FLAG = BS8 [0...7]**

**Frozen Value = UI16[0...15] <0...2<sup>16</sup>-1>}**

**FLAG** (同前)

### 5. 33 带时间的 32 位冻结计数器事件

**Data Object 23—Variation 05 Type: Frozen Event**

说明：

该对象作为事件上报的冻结了的计数器值，它能够累计来自一个硬件或软件点的脉冲数或变位的次数。

“冻结值段”显示在冻结时的计数器值。

“冻结时间”段含有该对象被冻结的时间。

“标志”段的意义与以往的对象相同

对象编码：

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |
| 47 |   |   |   |   |   |   | 0 |

标志

冻结值

冻结的时间

**SQ4 {FLAG = BS8 [0...7]**

**Frozen Value = UI32[0...31] <0...2<sup>32</sup>-1>**

**Time of Freeze = UI 48 [0...47] <0...2<sup>48</sup>-1>**

**}**

**FLAG**

}

(同 32 位二进制带冻结时间的冻结计数器)

叙记:

### 5. 34 带时间的 16 位冻结计数器事件

**Data Object 23—Variation 06      Type: Frozen Event**

说明: (同前)

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |
| 47 |   |   |   |   |   |   | 0 |

标志

冻结值

时间

**SQ4      {FLAG = BS8 [0...7]**

**Frozen Value = UI16[0...15]   <0...2<sup>16</sup>-1>**

**Time of Freeze   =UI 48 [0...47] <0...2<sup>48</sup>-1>**

**}**

**FLLAG    }**      (同前)

叙记:

### 5. 35 带时间的 32 位冻结计数器事件

**Data Object 23—Variation 07      Type: Frozen Event**

说明:

“带时间的 32 位冻结了的 Delta 计数器事件”表示一个作为事件报告的冻结了的 Delta 计数值,它能够累计来自一个硬件或软件点的脉冲数或变位的次数。

“冻结值”显示在冻结时刻的计数器中的变化。

“冻结时间”段含有对象被冻结的时间。

“标志”段的意义与以往的对象相同

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |
| 47 |   |   |   |   |   |   | 0 |

标志

冻结值

时间

**SQ4      {FLAG = BS8 [0...7]**

**Frozen Value = UI32[0...31]   <0...2<sup>32</sup>-1>**

**Time of Freeze   =UI 48 [0...47] <0...2<sup>48</sup>-1>**

**}**

**FLAG    }**      (同 32 位二进制带时间的冻结计数器)。

叙记:

### 5. 36 带时间的 16 位冻结了的 Delta 计数器事件

**Data Object 23—Variation 08 Type: Frozen Event**

说明: (同上)

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |
| 47 |   |   |   |   |   |   | 0 |

标志

冻结值

冻结的时间

**SQ4 {FLAG = BS8 [0...7]}**

**Frozen Value = UI16[0...15] <0...2<sup>16</sup>-1>**

**Time of Freeze = UI 48 [0...47] <0...2<sup>48</sup>-1>**

}

**FLAG**

叙记: } (同上)

译者小结:

| Data Object | 类 别                          |
|-------------|------------------------------|
| 00 系列       | Binary Input Object 二进制输入对象  |
| 10 系列       | Binary Output Object 二进制输出对象 |
| 20 系列       | Counter Object 计数器对象         |

| Type Data Object | Static<br>20 |      |      |      | Frozen Static<br>21     |      |      |      |       |       |    |    |
|------------------|--------------|------|------|------|-------------------------|------|------|------|-------|-------|----|----|
| Flag             | 1            |      | 0    |      | 1                       |      |      |      | 0     |       |    |    |
| Time             | 0            |      | 0    |      | 0                       |      | 1    |      |       |       |    |    |
| B/D              | B            | D    | B    | D    | B                       | D    | B    | D    | B     | D     | B  | D  |
| 32-b             | 20-1         | 20-3 | 20-5 | 20-7 | 21-1                    | 21-3 | 21-5 | 21-7 | 21-09 | 21-11 | -  | -- |
| 16-b             | 20-2         | 20-4 | 20-6 | 20-8 | 21-2                    | 21-4 | 21-6 | 21-8 | 21-10 | 21-12 | -- | -- |
|                  |              |      |      |      | 当前值与冻结值相同时冻结,<br>并显示冻结值 |      |      |      |       |       |    |    |

B—二进制计数器

D—Delta 计数器

| Type Data Object | Event<br>22 |  | Frozen Static<br>23 |  |  |
|------------------|-------------|--|---------------------|--|--|
| Flag             | 1           |  | 1                   |  |  |
| Time             | 0           |  | 1                   |  |  |

| B/D  | B                   | D    | B    | D    | B                 | D    | B    | D    |  |
|------|---------------------|------|------|------|-------------------|------|------|------|--|
| 32=b | 22-1                | 22-3 | 22-5 | 22-7 | 23-1              | 23-3 | 23-5 | 23-7 |  |
| 16-b | 22-2                | 22-4 | 22-6 | 22-8 | 23-2              | 23-4 | 23-6 | 23-8 |  |
|      | 以越死区为事件所报数值是越死区时的数值 |      |      |      | 以达到冻结值为事件并上报冻结时的值 |      |      |      |  |

- \* 凡无标志者，都隐含 **Flag=1**(即对象在线，;其它标志位均为常态)。
- \* 凡有时间段的 **20** 系列对象，其时间均按照 **ms** 计算，并自 **1970** 年 **1** 月 **1** 日午夜零时、零分、零毫秒起算。
- \* 凡有标志段 **20** 系列对象，其标志均为：

```

FLAG  ={
On-line  = BS1[0]          <0,off-line;1,on-line>
Restart  = BS1[1]          <0,normal;1,restart>
Communication lost = BS1[2] <0,normal;1,lost>
Remote forced data = BS1[3] <0,normal;1,forced>
Local forced data  = BS1[4] <0, normal;1,forced>
Roll-over  = BS1[4]        <0,normal;1,roll-over>
Roll-over  = BS1[5]        <0,normal;1,roll-over>
Reserved   = BS1[6]        <0>
Reserved   = BS1[7]        <0>      }

```

### 6.1 32 位模拟量输入

**Data Object 30—Variation 01      Type: Static**

“**32 位模拟量输入**”是一种表现一个硬件或软件模拟量点的信息对象。它的有符号 **32** 位值可以表现一个数字化的信号或计算值。

“标志”段与以往的对象具有同样的意义，但增加这两点：

• 参准校验”（**reference check**）段表示用以数字化模拟输入的参准信号不稳定，以数字化的值可能不正确。

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 31 |   |   |   |   |   |   | 0 |

当前值

**Frozen Value = 0132[0..31]**

 $\langle 2^{32} - 1 \dots - 2^{31} \rangle \}$ 

**On-line = BS1[0]**

**<0,off-line;1,on-line>**

**Restart = BS1[1]**

**<0,normal;1,restart>**

**Communication lost = BS1[2]**

**<0,normal;1,lost>**

**Remote forced data = BS1[3]**

**<0,normal;1,forced>**

**Local forced data = BS1[4]**

**<0 normal;1,forced>**

**Roll-over = BS1[4]**

**<0,normal;1,roll-over>**

**Roll-over = BS1[5]**

**<0,normal;1,roll-over>**

**Reserved = BS1[6]**

**<0>**

Reserved = BS1[7] <0>

}

## 6. 2 16 位模拟量输入

Data Object 30—Variation 02 Type: Static

说明:

与 32 位模拟量输入相同，不同之处在“超出量程”(over-range)段表明数字化的信号或计算值已超过  $2^{15}-1$  或低于  $-2^{15}$ 。“数值”段就可以不计了，因为它的数值未被定义。

对象编码:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
| 15 |   |   |   |   |   |   | 0 |

标志  
当前值

## 6. 3 32 位不带标志的模拟量输入

Data Object 30—Variation 03 Type: Static

说明:

“32 位模拟量输入”是一种表现一个硬件或软件模拟量点的信息对象。它的有符号 32 位值可以表现一个数字化的信号或计算值。

“数值”段显示在上报时模拟量输入之当时值或来自源发设备的上次上报值。

对象编码:

|    |   |
|----|---|
| 31 | 0 |
|----|---|

当前值

SQ2 { Current Value = UI32 [0...31] < $2^{32}-1$ ..... $-2^{31}$ > }

注意: 该变体用于输入点是在线的，且其余各标志均为正常的条件下（即此变体用于 flag=1）

## 6. 4 16 位不带标志的模拟量输入

Data Object 30—Variation 04 Type: Static

说明: (同前)

对象编码:

|    |   |
|----|---|
| 15 | 0 |
|----|---|

当前值

SQ2 { Current Value = UI32 [0...31] < $2^{15}-1$ ..... $-2^{15}$ > }

注意: (同前)

## 6. 5 32 位冻结了的模拟量输入

**Data Object 30—Variation 01    Type: Frozen Static**

说明:

“32 位冻结了的模拟量输入”是用以表现一个硬件或软件模拟量点的信息对象。有符号 32 位值可以表现一个数字化的信号或计算值。

“冻结值”段显示在该点上次实现冻结令时的模拟量输入值。

“标志”段与以往的对象具有同样的意义，但增加了

• “over-range”段指明数字化的信号或计算值已超过  $2^{31}-1$  或低于  $-2^{31}$ 。“数值”段的值实际已不可计，因为它的值未被定义。

“参准校验”（reference check）段表明用以数字化模拟输入的参准信号已不稳定，从而数字化的值也可能不正确。

对象编码:

|    |   |     |
|----|---|-----|
| 7  | 0 | 标志  |
| 31 | 0 | 冻结值 |

**SQ2 {FLAG = BS8 [0...7]**

**Current Value = UI32[0...31] < $2^{32}-1$ .... $-2^{31}$ >}**

**FLAG = {**

**On-line = BS1[0] <0,off-line;1,on-line>**

**Restart = BS1[1] <0,normal;1,restart>**

**Communication lost = BS1[2] <0,normal;1,lost>**

**Remote forced data = BS1[3] <0,normal;1,forced>**

**Local forced data = BS1[4] <0, normal;1,forced>**

**Roll-over = BS1[4] <0,normal;1, over-range>**

**Roll-over = BS1[5] <0,normal;1,over-range>**

**Reserved = BS1[6] <0,normal;1,error>**

**Reserved = BS1[7] <0>**

**}**

**6. 6    16 位冻结了的模拟量输入**

**Data Object 30—Variation 02    Type: Frozen Static**

说明: 同前，仪量程改为  $+2^{15}$  至  $-2^{15}$ 。

对象编码:

|    |   |    |
|----|---|----|
| 7  | 0 | 标志 |
| 15 | 0 |    |

## 冻结值

SQ2 {FLAG = BS8 [0...7]

Current Value = UI16[0...15]  $\langle 2^{15}-1 \dots -2^{15} \rangle$

}

### 6. 7 32 位带冻结时间的冻结了的模拟量输入

Data Object 31—Variation 03 Type: Frozen Static

说明:

“32 位带冻结时间的冻结了的模拟量输入”是用以表现一个硬件或软件模拟量点的信息对象。有符号 32 位值可以表现一个数字化的信号或计算值。

“冻结值”段显示规定冻结时的模拟量输入值。

“冻结时”段显示“冻结值”等于模拟输入当前值的时间，这两个数值的相等等效于收到一个模拟量的冻结令。

“标志”段与以往的对象具有同样的意义，但增加了

- “over-range”段它可以表示数字化的输入或计算值已超过  $2^{31}-1$  或低于  $-2^{31}$ 。其“数值”段的值实际已不可计，因为它的这种数值无定义。

- 参准校验”段表示用以数字化模拟输入的参准信号已不稳定，因而其数字化的值也可能不正确。

|       |    |   |   |   |   |   |   |   |       |
|-------|----|---|---|---|---|---|---|---|-------|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志    |
|       | 31 |   |   |   |   |   |   | 0 | 冻结值   |
|       | 47 |   |   |   |   |   |   | 0 | 冻结的时间 |

SQ2 {FLAG = BS8 [0...7]

Current Value = UI16[0...15]  $\langle 0 \dots 2^{15} \rangle$

Time of Freeze = UI 48 [0...47]  $\langle 0 \dots 2^{48} \rangle$

}

FLAG (同前)。

### 6. 8 16 位带冻结时间的冻结了的模拟量输入

Data Object 31—Variation 04 Type: Frozen Static

说明: 同前，仅量程改为  $+2^{15}$  至  $-2^{15}$

|       |    |   |   |   |   |   |   |   |    |
|-------|----|---|---|---|---|---|---|---|----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志 |
|       | 15 |   |   |   |   |   |   | 0 |    |
|       | 47 |   |   |   |   |   |   | 0 |    |

冻结值  
冻结的时间

SQ2 {FLAG = BS8 [0...7]  
Time of Freeze =UI 48 [0...47] <0...2<sup>48</sup>-1>  
Current Value = UI16[0...15] <0...2<sup>15</sup>.....-2<sup>15</sup>>  
}  
FLAG (同前)。

#### 6. 9 32 位无标志冻结了的模拟量输入

Data Object 31—Variation 05 Type: Frozen Static

说明:

“32 位无标志冻结了的模拟量输入”是用以表示一个冻结了的硬件或软件模拟输入之信息对象。有符号 32 位值能表示一个数字化的信号或计算值。

“冻结值”段显示在该点实现上一次“冻结”令时的模拟输入之值。

对象编码:



SQ2 { Current Value =UI32 [0...31] <2<sup>31</sup>-1.....-2<sup>31</sup>>  
}

注意: (该变体用于 flag=1 之时)

#### 6. 10 16 位无标志冻结了的模拟输入

Data Object 31—Variation 06 Type: Frozen Static

说明: (同上)

对象编码:



SQ2 { Current Value =UI16[0...15] <2<sup>15</sup>-1.....-2<sup>15</sup>>  
}

#### 6. 11 不带时间的 32 位模拟量变化事件

Data Object 32—Variation 01 Type: Event

说明:

“不带时间的 32 位的模拟变化事件”是用以表现一个硬件或软件模拟量点的信息对象。

32 位有符号值能表示一个数字化的信号或计算值。

当前的“数值”段显示在上报时模拟输入的当前值或来源设备的上次报告值。这个对象仅当“当前值”与上次上报值之差超过了可组态的死区值时方才上报。它滤除了通常所谓的死区内的摆动值（**dead bading**）。

“标志”段与以往的对象具有同样的意义，但增加了

- “**over-range**”段它可以表示数字化的信号或计算值已超过 $+2^{31}-1$ 或低于 $-2^{31}$ 。实际的“数值”段的值可以不计，因为此值无定义。

- 参准校验”段表示用表明以数字化模拟输入的参准信号不稳定，因而其数字化的值也可能不正确。

对象编码:

|    |   |     |
|----|---|-----|
| 7  | 0 | 标志  |
| 31 | 0 | 当前值 |

SQ2 {FLAG = BS8 [0...7]

Current Value = UI32[0...31]  $<+2^{31}-1 \dots -2^{31}>$

FLAG(同 32 位冻结了的模拟输入)

6. 12 16 位不带时间的模拟量变化事件

Data Object 32—Variation 02 Type: Event

说明: (同前) 仅量程改为 $+2^{15}-1$ 至 $-2^{15}$

对象编码:

|    |   |     |
|----|---|-----|
| 7  | 0 | 标志  |
| 15 | 0 | 当前值 |

SQ2 {FLAG = BS8 [0...7]

Current Value = UI16[0...15]  $<2^{15}-1 \dots -2^{15}>$

FLAG(同上)

6. 13 带时间的 32 位模拟量变化事件

Data Object 32—Variation 03 Type: Event

说明:

“带时间的 16 位模拟量变化事件”是用以表示一个改变了的硬件或软件模拟点之信息对象。带符号的 32 位值可以表示一个数字化的信号或计算值。

“当前值”显示在“时间”段所规定的时间上模拟输入之值。

“时间”段显示处理造成事件的时间。

“标志”段的意义与以往的对象相同，但增加 • “Over-range”段，（同 32—01）。

• 参准校验”段，（同 32—01）。

对象编码：

|    |   |
|----|---|
| 7  | 0 |
| 31 | 0 |
| 47 | 0 |

标志

数值

时间

SQ3 {FLAG = BS8 [0...7]

Value =UI 32 [0...31] <2<sup>31</sup>-1....- 2<sup>31</sup>>

Time = UI48[0...47] <0....-2<sup>48</sup>-1>

}

FLAG （同 31-03）。

6. 14 带时间的 16 位模拟量变化事件

Data Object 32—Variation 04 Type: Event

说明：同前，仅量程改为 2<sup>15</sup>-1 至 -2<sup>15</sup>。

对象编码：

|    |   |
|----|---|
| 7  | 0 |
| 15 | 0 |
| 47 | 0 |

标志

数值

时间

SQ2 {FLAG = BS8 [0...7]

Time = UI48[0...47] <0....- 2<sup>48</sup>-1>

Value =UI16[0...15] <2<sup>15</sup>-1....- 2<sup>15</sup>>

}

FLAG （同前）。

6. 15 不带时间的 32 位冻结了的模拟量事件

Data Object 33—Variation: 01 Type: Frozen Event

说明：

“不带时间的 32 位冻结了的模拟量事件”是用以表示一个冻结了的硬件或软件模拟量点之信息对象。该点被作为事件上报。有符号的 32 位值可以表示一个数字化的信号或计算值。

“冻结值”显示在 “冻结”时的模拟量输入之值。

“标志”段（同 31—03）

|       |    |   |     |
|-------|----|---|-----|
| 对象编码: | 7  | 0 | 标志  |
|       | 31 | 0 | 冻结值 |

SQ2 {FLAG = BS8 [0...7]}

Frozen Value = UI32[0...31]  $\langle 2^{31}-1 \dots -2^{31} \rangle$

FLAG(同 31-03)

## 6. 16 不带时间的 16 位冻结了的模拟量事件

Data Object 33-Variation 02 Type: Frozen Event

说明: (同 33-01) 仅量程改为  $+2^{15}-1$  至  $-2^{15}$

|       |    |   |     |
|-------|----|---|-----|
| 对象编码: | 7  | 0 | 标志  |
|       | 15 | 0 | 冻结值 |

SQ2 {FLAG = BS8 [0...7]}

Frozen Value = UI16[0...15]  $\langle 2^{15}-1 \dots -2^{15} \rangle$

FLAG(同 31-01)

## 6. 17 带时间的 32 位冻结了的模拟量事件

Data Object 33-Variation 03 Type: Frozen Event

说明:

同 33-01, 但增加了“冻结时间”显示“冻结值”。等于模拟输入“当前值”的时间。这两个数值相等时等于收到一条模拟量的冻结令。

|       |    |   |      |
|-------|----|---|------|
| 对象编码: | 7  | 0 | 标志   |
|       | 31 | 0 | 冻结值  |
|       | 47 | 0 | 冻结时间 |

SQ2 { FLAG = BS8 [0...7]}

Frozen Value = UI32[0...31]  $\langle 2^{31}-1 \dots -2^{31} \rangle$

Time of Freeze= UI48[0...47]  $\langle 0 \dots 2^{48} \rangle$

FLAG (同 31-03)。

## 6. 18 带时间的 16 位冻结了的模拟量事件

Data Object 33-Variation 04 Type: Frozen Event

说明: (同 33-03), 仅量程改为  $2^{15}$  至  $-2^{15}$ 。

|       |    |   |      |
|-------|----|---|------|
| 对象编码: | 7  | 0 | 标志   |
|       | 31 | 0 | 冻结值  |
|       | 47 | 0 | 冻结时间 |

SQ2 { FLAG = BS8 [0...7]  
Freezen Value =UI16[0...15] <2<sup>15</sup>-1....- 2<sup>15</sup>>  
Time of Freeze= UI48[0...47]  
FLAG (同前)。

译者按：本章小结

| Type Data Object | Static 30 |       | Frozen Static 31 |               |          | Event 32 |        | Frozen Event 33 |              |  |
|------------------|-----------|-------|------------------|---------------|----------|----------|--------|-----------------|--------------|--|
| Flag             | 1         | 0     | 1                |               | 0        | 1        |        | 1               |              |  |
| Time             | 0         | 0     | 0                | 1             | 0        | 0        | 1      | 0               | 1            |  |
| 32-b             | 30-01     | 30-03 | 31-01            | 31-03         | 31-05    | 32-01    | 32-03  | 33-01           | 33-03        |  |
| 16-b             | 30-02     | 30-04 | 31-02            | 31-04         | 31-06    | 32-02    | 32-04  | 33-02           | 33-04        |  |
|                  |           |       | 由冻结令实现冻结         | 由当前值等于冻结值实现冻结 | 由冻结令实现冻结 | 以越死区定为事件 | 由处理定事件 | 由冻结并作为件报        | 当前值于冻结时冻并事上报 |  |

- 凡无 FLAG 段者，都隐含 flag=1 的条件 •
- 凡有 FLAG 者，共

FLAG={

On-line = BS1[0]

<0,off-line;1,on-line>

Restart = BS1[1]

<0,normal;1,restart>

Communication lost = BS1[2]

<0,normal;1,lost>

Remote forced data = BS1[3]

<0,normal;1,forced>

Local forced data = BS1[4]

<0, normal;1,forced>

Roll-over = BS1[5]

<0,normal;1, over-range>

Roll-over = BS1[6]

<0,normal;1,error>

Reserved = BS1[7]

<0>

}

- **Over-range** 的数值段:

|       | Over-range | Under-rauge |
|-------|------------|-------------|
| 32bit | $2^{31}-1$ | $-2^{31}$   |
| 16bit | $2^{15}-1$ | $-2^{15}$   |

## 第七章：模拟量输出对象的定义

本章使用第一章内所建立的规则来定义模拟量输出之数据对象。

### 7. 1 32 位模拟量输出之状态

**Data Object 40—Variation 01 Type: Static**

说明:

“32 位模拟输出的状态信息”表现一个模拟输出或软件点之实际值及其有关的状态。

“实际值”字段含有模拟输出的当前值。

“标志”段具有与以往对象相同的意义。

对象编码:

|    |   |   |   |   |   |   |   |     |
|----|---|---|---|---|---|---|---|-----|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
| 31 |   |   |   |   |   |   | 0 | 当前值 |

**SQ3 {FLAG = BS8 [0...7]**

**Frozen Value = UI32[0...31]**  $\langle 2^{31}-1 \dots -2^{31} \rangle$

**FLAG={**

**On-line = BS1[0]**  $\langle 0, \text{off-line}; 1, \text{on-line} \rangle$

**Restart = BS1[1]**  $\langle 0, \text{normal}; 1, \text{restart} \rangle$

**Communication lost = BS1[2]**  $\langle 0, \text{normal}; 1, \text{lost} \rangle$

**Remote forced data = BS1[3]**  $\langle 0, \text{normal}; 1, \text{forced} \rangle$

**Reserved = BS1[4]**  $\langle 0 \rangle$

**Reserved = BS1[5]**  $\langle 0 \rangle$

**Reserved = BS1[6]**  $\langle 0 \rangle$

**Reserved = BS1[7]**  $\langle 0 \rangle$

**}**

叙记: 在一次模拟量输出操作之后, 该对象能够返回以便判定操作是否成功。

### 7. 2 16 位模拟量输出之状态

**Data Object 40—Variation 02 Type: Static**

说明:

“16 位模拟输出之状态”这个信息对象表现一个 DAC 模拟输出硬件或软件点之实际值及其有关的状态。

“实际值”字段含有模拟输出之当前值。

“标志”段具有与以往对象相同的意义

|       |    |   |   |   |   |   |   |   |     |
|-------|----|---|---|---|---|---|---|---|-----|
| 对象编码: | 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 | 标志  |
|       | 15 |   |   |   |   |   |   | 0 | 当前值 |

SQ3 {FLAG = BS8 [0...7]

Current Value = UI16[0...15]  $\langle 2^{15}-1 \dots -2^{15} \rangle$

}

FLAG = (同前)

### 7. 3 32 位模拟量输出块

Data Object 41—Variation 01 Type: Static

说明:

“32 位模拟输出块”这个信息对象表现一个硬件 DAC 模拟输出或软件点所要求的值。此值仅为一个逻辑值，因为在任何输出电平被设定之前这种数值还要被定标与 / 运标。

“请求值”(requested value) 字段含有模拟输出的要求值。模拟输出的实际值被回送在“模拟输出状态”对象内。

“控制状态”(control status) 表明模拟控制操作的状态，就象 control delay output block 所作的一样。该段的定义与 control relay output block 中一样。

|       |    |   |                     |
|-------|----|---|---------------------|
| 对象编码: | 31 | 0 | Requested Value 请求值 |
|       | 7  | 0 | Control Status 控制状态 |

Requested Value = UI32[0...31]  $\langle 2^{31}-1 \dots -2^{31} \rangle$

Status = UI8[0...7]  $\langle 0 \dots 255 \rangle$

这是用于一个 Control 的请求 / 响应中的。

### 7. 4 16 位模拟量输出块

Data Object 41—Variation 02 Type: Static

说明: (同上)

对象编码: (同上), 仅 Requested Value 段改为 15...0.

Requested Value = UI16[0...15]  $\langle 2^{15}-1 \dots -2^{15} \rangle$

**Status** = UI8[0...7] <0.....255>

对象编码:

|    |   |
|----|---|
| 15 | 0 |
| 7  | 0 |

请求值(Requested Value)

控制状态(Control Status)

DocuCom PDF Trial  
www.pdfwizard.com

## 第八章： 时间对象的定义

本章使用第一章内所建立的规则来定义时间数据的对象。

### 8. 1 时间与日期

#### Data Object 50—Variation 01

说明：

“时间与日期”对象是表现日期与当日绝对时间的信息对象。该对象应该用于时间同步的情况下。

对象编码：

| 绝对时间 |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|
| 7    | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 15   | 14 | 13 | 12 | 11 | 10 | 9  | 8  |
| 23   | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| 31   | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
| 39   | 38 | 37 | 36 | 35 | 34 | 32 | 31 |
| 47   | 46 | 45 | 44 | 43 | 42 | 41 | 40 |

绝对时间= UI48[0...47] <0... $2^{48}-1$ , msec>毫秒

叙记：“绝对时间”按毫秒记录，自1970年1月1日午夜零时零分零秒和零毫秒开始。

### 8. 2 带有时间间隔的时间与日期

#### Data Object 50—Variation : 02

说明：

“带有时间间隔的时间与日期”既表示一个绝对时间，也表示了一个时间间隔。“绝对时间”是一个起动时间（或时基），而间隔时间是自时基起的一段正向偏移。这种时间间隔也可以对一个时基多次使用以便得到一个周期性时间的序列。

例如，该对象可用以为对象周期性的冻结规定一个时间序列。

“绝对时间”段规定了时基，这个时间是一日内的实时。

“时间间隔”段规定了周期性的时间间隔或自时基起始的偏移时间。

对象编码：

| 绝对时间 |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|
| 7    | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 15   | 14 | 13 | 12 | 11 | 10 | 9  | 8  |
| 23   | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| 31   | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
| 39   | 38 | 37 | 36 | 35 | 34 | 33 | 32 |
| 47   | 46 | 45 | 44 | 43 | 42 | 41 | 40 |
| 间隔时间 |    |    |    |    |    |    |    |
| 7    | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 15   | 14 | 13 | 12 | 11 | 10 | 9  | 8  |
| 23   | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| 31   | 30 | 29 | 28 | 27 | 26 | 25 | 24 |

**Absolute time**

**Interval**

**SQ2{Absolute time = UI48[0...47]} <0...2<sup>48</sup>-1, msec>**

**Interval = UI32[0...31] <0...2<sup>32</sup>-1, msec>**

叙记：“绝对时间”按毫秒记录，自 1970 年 1 月 1 日午夜零时，零分，零秒，零毫秒起始。

### 8. 3 CTO 的时间与日期

**Data Object 51—Variation: 01**

说明：

“CTO（事件的共用时间）”是表示日之绝对时间的一个对象，这个对象应与另一个包含时间参准值的对象联用。此对象是一个时基，可以将一个相对的（递增的）时间可以加向此时基或自此时基上减速去，以便得到另一个绝对的基准时间。

对象编码：

| 绝对时间 |    |    |    |    |    |    |    |
|------|----|----|----|----|----|----|----|
| 7    | 6  | 5  | 4  | 3  | 2  | 1  | 0  |
| 15   | 14 | 13 | 12 | 11 | 10 | 9  | 8  |
| 23   | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
| 31   | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
| 39   | 38 | 37 | 36 | 35 | 34 | 32 | 31 |
| 47   | 46 | 45 | 44 | 43 | 42 | 41 | 40 |

**Absolute time = UI48[0...47] <0...2<sup>48</sup>-1, msec>**

叙记：“绝对时间”按毫秒记录，自 1970 年午夜零时，零分，零秒，零毫秒起始。

## 8. 4 非同步的 CTO 时间与日期

### Data Object 51—Variation: 02

说明:

“非同步的 CTO 时间与日期”是表示相对绝对的日时间之信息对象。这个对象应与其它包含时间参准的（若干）对象联用。本对象是一个相对的时基，可以将一个相对的（递增的）时间加于其上或自其中减去，以便得另一个相对的绝对时间参准。真正的“绝对时间”则可以由报文接收机根据含有此对象的报文之收到时间计算而得。跟在此对象之后的并且在下一个含有相对时间的 CTO（共用时间对象）来到之前的任何对象都必需用此相对的绝对时间值校正。

“相对时的绝对时间”是发送此对象的站上（即响应站）非同步的日时间（time-of-day）。

对象编码:

| 相对绝对时间 |    |    |    |    |    |    |    | Relative      |
|--------|----|----|----|----|----|----|----|---------------|
| 7      | 6  | 5  | 4  | 3  | 2  | 1  | 0  | Absolute Time |
| 15     | 14 | 13 | 12 | 11 | 10 | 9  | 8  |               |
| 23     | 22 | 21 | 20 | 19 | 18 | 17 | 16 |               |
| 31     | 30 | 29 | 28 | 27 | 26 | 25 | 24 |               |
| 39     | 38 | 37 | 36 | 35 | 34 | 32 | 31 |               |
| 47     | 46 | 45 | 44 | 43 | 42 | 41 | 40 |               |

Relative-absolute time = U148[0...47] <0...2<sup>48</sup>-1, ms>

叙记: “相对绝对时间”按毫秒计的，自 1970 年 1 月 1 日午夜零时,零分,零秒,零毫秒起始。

## 8. 5 粗延时

### Data Object 52—Variation: 01

说明:

“粗延时信息”对象表明一段相对时间，它是自报文收到时开始的一段时间周期。

对象编码:

| 秒  |    |    |    |    |    |   |   | Seconds |
|----|----|----|----|----|----|---|---|---------|
| 7  | 6  | 5  | 4  | 3  | 2  | 1 | 0 |         |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |         |

Seconds = U116[0...15] <0...2<sup>16</sup>-1, 秒>

8. 6 精延时

Data Object 52—Variation: 02

说明:

“精延时信息”对象表示一个相对时间，它是自报文收到时间开始的一段时间周期。该对象可用于时间整步去实现精延时的测量计算或用于要求时基刻度的其它功能。

对象编码:

| 毫秒 |    |    |    |    |    |   |   |
|----|----|----|----|----|----|---|---|
| 7  | 6  | 5  | 4  | 3  | 2  | 1 | 0 |
| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |

Milliseconds

Milliseconds = UI16[0...15] <0...2<sup>16</sup>-1,msec>

## 第九章：级类对象的定义

---

本章使用第一章内所建立的规则来定义类的数据对象。

### 9.1 0类数据

#### Data Object 60—Variation: 01

说明：

“0类数据”对象是一种对象的位座，它被规定为零类或更多的信息单元。这种单元可以是全部的对象类型，一种指定的变体(**variation**)，变体的某些点，或是以上这些的任意组合，由该对象类型所规定的的数据，在响应站内是可组态的。

未被归入 1, 2 或 3 的数据就潜在地属于“0类数据”。

对象编码：

无

叙记：

“0类数据”对象本身并不带任何信息，因此也就没有对象编码。“0类”是一种无束缚的类，任何未归属其它类别的数据对象，都可以根据预置而归于它。

### 9.2 1类数据

#### Data Object 60—Variation: 02

说明：

“1类数据”是规定一类无信息单元或多信息单元的对象位座，这些单元(**elements**)可以是整个类型，一种指定的变体，变体的某些点或以上的任何组合，由该数据类型所规定的的数据，在响应站内是可组态的。

响应不发送“1类数据”对象，因为它不含有实际的信息，它仅是其它对象的识别符。

对象编码：

无。

叙记：

“1类数据”对象被用以去请求一个已组态的组，通常是来自一个响应站的信息对象之变化。这个数据对象本身不带任何信息，所以也就没有对象编码。

“1类数据”，典型地比 2 类、3 类及 0 类数据的优先级都更高。

### 9.3 2类数据

#### Data Object 60—Variation: 03

说明:

“2类数据”规定一类无信息单元或多个信息单元的对象位座，这些单元可以是整个对象类型，一种指定的变体，变体的某些点或以上任意的组合，由该数据类型所规定的数据在响应站内是可组态的。

响应不发送“2类数据”对象，因为它不含有任何实际信息，它仅是其它对象的识别符。

对象编码:

无。

叙记:

“2类数据”对象被用来请求一个已组态的组，通常是来自一个响应站的信息对象之变化。这个数据对象本身不带任何信息，所以也就没有对象编码。

### 9.4 3类数据

#### Data Object 60—Variation: 04

说明:

“3类数据”规定一类无信息单元或多信息单元的对象位座，这些单元可以是整个的对象类型，一种指定的变体，变体的某些点或以上任意的组合，由该数据类型所规定的数据在响应站内是可组态的。

响应不发送“3类数据”对象，因为它不含有任何实际信息，它仅是其它对象的识别符。

对象编码:

无。

叙记:

“3类数据”对象被用来请求一个已组态的组，通常是来自一个响应站的信息对象之变化。这个数据对象本身不带任何信息，所以也就没有对象编码。

## 第十章：文件对象的定义

本章使用第一章内所建立的规则来定义文件数据对象。

### 10. 1 文件识别码

#### Data Object 70—Variation: 01

说明：

“文件识别码”对象是一种信息对象，它表示在一个网络文件系统内，关于一个文件的信息。该对象准备用以传送一种不遵循现有数据对象格式的大数据块。该对象特别适宜于上装与下装组态文件至远方装置或数据集中器。

为此对象还定义了文件操作，可允许对一个文件的复制（考见），删除等等。文件对象的内容以及在文件上的正确执行步骤将不由“应用层”去解释，所以必须由“应用层”的用户去实现。

网络的能力：（Networking Capability）

“文件名”（File - Name）段定义一个文件或设备的逻辑名。接收的应用程序应解释此“名”作一个网络文件名。“根”（root 或 1）表示主机文件系统或接收节点的根。根的子目录（sub-directories）被解释为根的分支转移。一个分支可以是一个新的目录，或者更重要的这个目录可以是一个远方的文献系统，它驻留于可自当前节点访问的一个远方设备。

当一个应用程序收到一份指定一个目录的文件时或收到不驻留于当前文件系统的文件时，应用程序必需做任何必要的事以便自所指定的设备获得此文件。

若远方设备是一台 DNP3 的设备，那就应用以下规则：

- 收到对一个非本地文件的请求后，应用程序应将请求（以它的整体）向前送至恰当的 DNP3 设备。

- 文件名（File-name）段被修改以致远方设备的根被为文件名段所指定。这意谓着撕去已驱动远方设备名的任何住（PC）名。

例如，一个 IED（智能电子设备）的组态被从主机通过数据集器 DC1 及 DC2 发出，从主机到 DC1 的名字为：

\DC2\IED\config1

自 DC1 到 DC2 的名字为：

## \IED\config 1

自 DC2 到 IED 的名字为:

### \Config1

在这种情况下, DC2 和 IED 是逻辑目录, 它规定远方设备及 Config 1 是物理文件的名字。

对象编码:

这不是一个固定格式的对象, 而是一个格式 / 规模可变的对象, 这部份对象在请求的报文中送出。

|                          |    |                |    |                       |   |                    |   |
|--------------------------|----|----------------|----|-----------------------|---|--------------------|---|
| 31                       | 24 | 23             | 16 | 15                    | 8 | 7                  | 0 |
| Attributes 属性            |    | File_Size 文件类型 |    | Name_Size 名称_规模       |   |                    |   |
| End_Record 文件的结束记录号      |    |                |    | Start-Record 文件的起始记录号 |   |                    |   |
| File_Size 文件规模           |    |                |    |                       |   |                    |   |
| Time_of_Creation 文件的生成时间 |    |                |    |                       |   |                    |   |
| Permission 允许            |    |                |    |                       |   |                    |   |
| File_ID                  |    |                |    |                       |   |                    |   |
| Owner_ID                 |    |                |    |                       |   |                    |   |
| Group_ID                 |    |                |    |                       |   |                    |   |
|                          |    |                |    | Status 状态             |   | File_Function 文件功能 |   |
| X                        |    | File_Name 文件名  |    |                       |   |                    | 0 |

“File-Name”段包含 0...X 比特, 这里  $X = \text{Name\_Size} \times 8$

而  $\text{Name\_Size} = 0 \dots 65535 (2^{16} - 1)$

“file identifier”(文件识别符, 即 File\_ID)对象是在一个 DNP “应用请求”的报文内送给一个远方的设备, 使用“应用层的‘写’请求”功能码。一个设备用一个“应用层的‘响应’或‘非请求的响应’功能码(按需要)去响应”。“文件识别符”对象的请求(或自发的报告)。

**File-Name** (文件名):

要执行操作的文件名。它包含 1 个或多个字符, 仅包括 A...Z, a...z, 0...9, “.”, “-”。“\”和 “-”, 这里的 “-” 定界符, 还有其中的短横符不可用于文件名的第一个字符。该段的规模取决于下面的 **Name-Size** 段。名称可以包含自文件系统的根(即 “\”)以后的全部经

信息，名称内可以包含空格符仅用以将文件的径 / 名与程序的变元分隔开来。

**Name-Size(名称的规模):**

确定 **File-Name(文件名)** 的字符数。

**File-Function(文件功能):**

对指定的文件记录或对在用户层的文件系统所要执行的功能。它包括以下：**APPEND**（附加），**DELETE**（删除），**INSERT**（插入），**WRITE**（写），**ERASE**（擦除），**INFO**（取信息），**CWD**（改变工作的索引），**PWD**（现在的工作索引），**EXECUTE**（执行），**READ**（读）。以下的数值被定义为：

**0=APPEND（附加）:**

将数据记录加在文件的末尾（**END**）。**Start\_Record(起始记录)**段指明要附加于文件的记录号，并也是报文中跟在“**file identifier**”(文件识别符)报头后面的数据记录号。跟在报头后面的数据记录被说明于下面的 **WRITE** 功能码中。设备应该用“**File identifier**”的对象去响应，对象的有用段都要填写全，其中 **File\_function(文件功能)**段应置于 **RESPONSE(响应)**，而 **status(状态)**段则置入所请求操作之状态。

**1=DELETE（删除）:**

从文件中去除掉所指定的记录（即文件收缩）。**Start\_Record** 指明所要删除的第一个记录，而 **End\_Record** 指明所要删除的最后一个记录。这个请求中没有数据记录。设备应该用 **file\_identifier** 对象去响应，全部有用的段都要填好，其中 **File\_Function** 段置于 **RESPONSE** 而 **status** 段则置入所请求操作之状态。

**2=INSERT（插入）:**

自 **Start\_Record** 段所指定的位置插入这些记录，连续插入至 **End\_Record** 段所指定的位置为止（即文件中规模增大）。跟在文件中 **file\_identifier** 报头后面的文件记录数为 **End\_Record-Start\_Record+1**。跟在报头后面的数据记录说明于下文中的 **WRITE** 功能码。设备应该用 **file identifier** 对象去响应，全部有用的段都要填好，其中 **File\_Function** 段置于 **RESPONSE**，而 **Status** 段则置入所请求操作之状态。

**3=WRITE（写）:**

将这些记录放在由 **Start\_Record** 段所指定的地方，并延续至 **End\_Record** 段所指的地方（即文件可以由此生成，并将原先的数据去除，替之以这些记录。）报文中跟在 **file identifier** 报头之数量为 **End\_Record-Start\_Record+1**。跟在报头后面的数据记录描述

如下。设备应该用 **file identifier** 对象去响应，全部有用的段都要填好，其中 **File\_Function** 段置于 **RESPONSE**，而 **Status** 段则置入所请求操作之状态。

| Data |   | Record_Size(记录规模) |   |
|------|---|-------------------|---|
| X    | 0 | 15                | 0 |

其中  $X = \text{Record\_Size} * 8$

$\text{Record\_Size} = 0..65535$

#### 4=ERASE(擦除):

将 **Start** 与 **End** 在与记录段所指定的全部记录清除掉或擦除掉(即文件仍保持同一规模，但数据被清除了)。在此报文中没有数据记录。设备应该用 **file identifier** 对象去响应。全部有用的段都要填好，其中 **File\_Function** 段置于 **RESPONSE**，而 **status** 段则置入所请求的操作之状态。

#### 5=INFO (取消息):

该消息系用以在指定的文件中获取信息。设备应该用 **file identifier** 对象去响应，它的报头各段都要填好，其中 **File\_Function** 段置以 **RESPONSE**，而 **status** 段则置入所请求的操作之状态， **File\_Name** 段可以用一个专用的名字“/”表示所有的(**ALL**)文件。设备应该用在设备文件系统内每个文件的 **file identifier** 对象的报头去响应。若设备只有一个文件(并且没有索引)那应该一个文件的 **file identifier** 对象的报头就该回送。

#### 6=CWD (改变工作索引):

改变工作索引(**Change Working Directory-CWD**)至 **File\_Name** 内规定的路径。设备应该用 **file identifier** 对象去响应，所有有用的段都要填好，其中 **File\_Function** 段置成 **RESPONSE**，**File\_Name** 段置成新的工作索引，**status** 段则置入所请求的操作之状态。

#### 7=PWD (现在的工作索引):

回送现在在 **File\_Name** 内的工作索引(**PWD**)。设备应该用 **file identifier** 对象去响应，所有有用的段都要填好，**File\_Function** 段置成 **RESPONSE**，段置入当前的工作索引，以及状态段置入所请求操作之状态。

#### 8=EXEC (执行):

起动或执行 **File\_Name** 所指定的应用程序并将 **File\_Name** 中文件名部分后随的参数(以空格分隔的)传给它。设备应该用 **file identifier** 对象去响应，所有有用的段都要填好，**File\_Function** 段置成 **RESPONSE**，以及在 **status** 段置入所请操作之状态。

#### 9=READ (读):

读取所指定的文件记录。**Start\_Record** 指定要读的第一个记录, **End\_Record** 指定要读的最末一个记录。如果 **start\_Record** 为 0, **End\_Record** 为 65535, 则设备应以全部可用的记录去响应。响应中的 **File\_Size** 段应表明文件的总规模。设备应该用 **file identifier** 对象(所有有用的段都要填好)和所有被请求的数据记录(如属可能)去响应, **File\_Function** 段置成 **RESPONSE**, **Start\_Record** 和 **End\_Record** 应置于在响应中回送的始端和末端的数据记录, 段则置入所请求操作的状态。报文内 **file identifier** 报头随的数据记录数为 **End\_Record-Start\_Record+1**。跟随于报头后的数据记录已说明于 **WRITE** 功能码。

**255=RESP:**

该功能码用以标明对一个请求的响应。该报文的内容由请求报文内的功能码所规定。

**Permission** (允许):

**Permission** 段规定文件所有者, 文件所有者的群组, 以及“world”(所其它人)对 **READ**, **WRITE** 与 **EXECUTE** 之特权。**READ** 特权使用户有权去读文件 (**READ**, **PWD**, **CWD**, **INFO** 等请求)。**WRITE** 特权使用户有权去修改文件 (**APPEND**, **DELETE**, **INSERT**, **WRITE**, **ERASE**), **EXECUTE** 特权使用户有权去运行指定的应用程序 (**EXEC** 请求)。

**File\_Type** (文件类型):

向接收方的应用程序指明如何解释文件对象的内容。有效的类型有: **8 位二进制**, **8 位 ASCII**, **7 位 ASCII**, **EBCDIC** (扩充的二进制编码之十进制交换码), **BCD** (十进制), **Baudot** (博多码—用于某些电传机), **International Baudot**。其代码值如下:

**0=8 位二进制** (非编码的二进制字节)

**1=8 位 ASCII** (扩展的 ASCII 字符)

**2=7 位 ASCII** (ASCII 字符)

**3=EBCDIC** (扩展的二进制编码之十进制)

**4=BCD** (二十进制)

**5=Baudot** (5 位的 Baudot 码)

**6=International Baudot** (6 位 Baudot 码)

**Attributes**(属性):

文件的属性包括有: **regular** (常规), **temporary** (暂时), **directory** (目录) 或 **FIFO** (先进先出) 代码值如下:

**0: Regular**(常规)文件是暂时的并且如果要将变化过程保留下来就必需将它保存起来。

**1: Temporary**（暂时的）文件是暂时的并且如果要将变化过程保留下来就必需将它保存(saved)起来。

**2: Directory**（目录）是多个文件的文件（不能被读）。

**3: FIFO** 是一个先进的队列并可用作过程间的通信（就像一个插口或管道）

**Status(状态):**所请求的操作之状态，他包括：**OK, Doesn't, exist, Out\_of\_space, No\_permission** 和 **File\_Busy**,在代码值如下：

**0: OK** 表示请求的操作是成功。

**1: Doesn't\_exist**（不存在）表示文件的操作使文件无此文件名。

**2: Out\_of\_space**（越出空间）表明文件的操作使文件超出它由 **UserID, Group ID** 和 **Permission** 所决定的最大规模。

**3: No\_Permission**（不允许）表明文件的所有者对于他所谓求的操作没有足够的特权。

**4: File\_Name**（文件忙）表明文件不能够送达目的地。

**File\_Size(文件规模):**

由 **File\_Name** 所规定的文件之心字节数。

**Start\_Record(起始记录):**

文件起始的记录号。**0** 值的起始记录表示文件之起始，而数值为 **65535** 的起始记录表示这是最后一个记录。

**End\_Record(结尾记录):**

文件的结尾记录号与 **Start\_record** 相似。在一个 **READ** 请求中若始记录为 **0**，尾记录为 **65535** 则表明是整个文件。

**Owner\_ID(所有者的识别符):**

对于文件的所有者这是唯一的（**Unique**）识别符。

**Group\_ID(组识别符):**

对于所有者的群组这是唯一的识别符。

**File\_ID(文件识别符):**

对于文件有一个唯一的识别符，本段也可以被用来容纳文件的检误（典型的是 **16** 位 **CRC**）。在这种情况下，仅当 **File\_Name** 和 **Time\_of\_Creation** 连接起来时，**File\_ID** 才是唯一的。

**Data(数据):**

构成文件记录的实际数据字节。这种 8 毕特的对象是根据 **File\_Type** 段解释的，数据段的内容不由 **DNP** 应用层解释的。

DocuCom PDF Trial  
www.pdfwizard.com

# 第十一章： 设备对象的定义

本章采用第一章内所建立的规则来定义设备数据对象。

## 11. 1 内部信号（Internal Indications）

### Data Object80-Variation:01

说明：

内部信号是一种信息单元，用以传送响应站的内部状态和诊断结果。这种信息可以为接收站所使用去实现差错的恢复或其它功能。

在报文中所发送的内部信号对象之数量是与设备有关的。

对象编码：

0

BS1 [0...0]

State = BS1[0] <0,1>

叙记：

数据对象的传输总是以整字节去实现的，其中不使用的码位均置 0。下面举例说明 n 个这种数据对象的组装。

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 7  | 6  | 5  | 4  | 3   | 2   | 1   | 0   |
| 15 | 14 | 13 | 12 | 11  | 10  | 9   | 8   |
|    |    |    |    |     |     |     |     |
| 0  | 0  | 0  | n  | n-1 | n-2 | N-3 | n-4 |

## 11.2 存储对象（Storage Object）

### Data Object81 - Variation:01

说明：

存储对象（“storage object”）是一种信息单元，用以传送内部缓存器的状态和特定数据类型的存储区之状态。

其 “group”段表明 status 段所对应的组（或数据类型）。

其 “variation”段表明 status 段所对应的对象之变体。

“group”与 “variation”两段合在一起指定了数据的的确切类型。其 status 段展示所

分配的缓存空间已被当前数据类型用去百分之几。**Overflow**（溢出位）可以表明所指定的数据类型已将缓存空间用过头，因而已失落了数据对象。

对象编码：

| 状态 |   |   |   |   |   |   |   | Status<br>Group<br>Variation |
|----|---|---|---|---|---|---|---|------------------------------|
| 7  | 6 | 5 | 4 | 3 | 2 | 1 | 0 |                              |
| 7  |   | 组 |   |   |   |   | 0 |                              |
| 7  |   | 变 | 体 |   |   |   | 0 |                              |

**Storage Object = {**

**Status = BS8[0...7]**

**Group = UI8[0...7] <0...255>**

**Variation = UI8[0...7] <0...255>**

**}**

其中 **Status = {**

**Percent = 7[0...6] <0...100>**

**Overflow = BS1[7] <0,1;溢出>**

**}**

叙记：

存储对象用以表明发送设备内的缓存区，队列或其它存储区之状态，这个对象由设备生成，它在设备内组态并在特定的设备简表中被描述。

### 11.3 设备简表 (Device Profile)

**Data Object82 - Variation:01**

说明：

“**device profile**”对象在不同的 **DNP** 设备之间提供互操作性，这些设备只使用 **DNP** “应用层”功能码和数据对象的一个子集。这个对象将站的作用作为一个主站 (**master**) 和从站设备所支持的有效功能码和对象来加以描述。此外，还给出每个对象 / 变体的 **indices** 变程 (**range**)，以至可以在一个动态基础上组态。

操作原理：

仅当主站所发出的请求是不可识别的，不可分析的或在请求中所援引的对象是不被支持的，从站设备才打算发送 **device profile** 对象与此报文一致的是“内部信号”将根据分析指明问题。

另一种情况，如果从站被组态于一个静止 (**quiescent**) 的环境中，从站可以在启动

(**startup**) 时，自发地报告设备简表 (**device profile**) 对象。

主站收到此对象后，能够改变其轮询方案，发出请求报文，限制或扩充对从站所设想的功能性或者用此对象内所规定的对象对主站数据库重新组态。

如果主站不那么高级，从站可以被标以“离线”，有必要用人工重组态以重新得到正确的通信。

“**device profile**”含有两个部分，第一部分是“功能”，它指定所支持的“DNP 应用层”功能码。第二部分是“对象”它规定了每个对象 / 变体组的 **indices** 之变程。“对象”部分基本上是采样主站呼叫每个对象 / 变体组之对象报头。能对实现什么类型的操作（功能则隐含于对象的类型之中，所以无需对一组对象变换 (**map**) 每个功能码。

“**function**”段是一个比特阵列，表明对每个功能码的支持或不支持。码位 **0...63** 相应于“DNP 应用层功能码”之 **0** 至 **63**。对于 **63** 以外的请求功能码，可以在“**Object Headers**”之后尾随另一个“**function**”段。

“**Num Objects**”段指定后随有多少个采样对象的报头 (**Object header**)。

“**Object Header**”段拥有与一个 DNP 应用层 **Object header** 相同的形式。**header** 最少含有 **Object, Variation, qualifier** (限定值) 和 **8** 比特的数量。这意思说要描述大多数的变体 (**variation**) 只要用 **4** 个字节。

编码对象:

|                        |                      |           |   |           |   |        |   |
|------------------------|----------------------|-----------|---|-----------|---|--------|---|
| 63                     | Function(功能)         |           |   |           |   |        | 0 |
| 15                     | Num Objects(n) (对象数) |           |   |           |   |        | 0 |
| Object Header1(1 号对象头) |                      |           |   |           |   |        |   |
| Quantity               |                      | Qualifier |   | Variation |   | Object |   |
| 7                      | 0                    | 7         | 0 | 7         | 0 | 7      | 0 |
| Object Header2(2 号对象头) |                      |           |   |           |   |        |   |
| Quauntity              |                      | Qualifier |   | Variation |   | Object |   |
| 7                      | 0                    | 7         | 0 | 7         | 0 | 7      | 0 |
| Object Header3(3 号对象头) |                      |           |   |           |   |        |   |
| Quauntity              |                      | Qualifier |   | Variation |   | Object |   |
| 7                      | 0                    | 7         | 0 | 7         | 0 | 7      | 0 |

```

SQ2 {Function      = BS64 [0...63]
      Num Objects  = UI16 [0...15]    <0...65535>
    }

```

后随的每个对象头拥有一个可变格式，取决于构筑应用层对象头的规则。



象有一个固定的报头提供给透明的数据传输，还有一个后随对象的唯一登记号。对内容的说明完全由供货商自由处理。

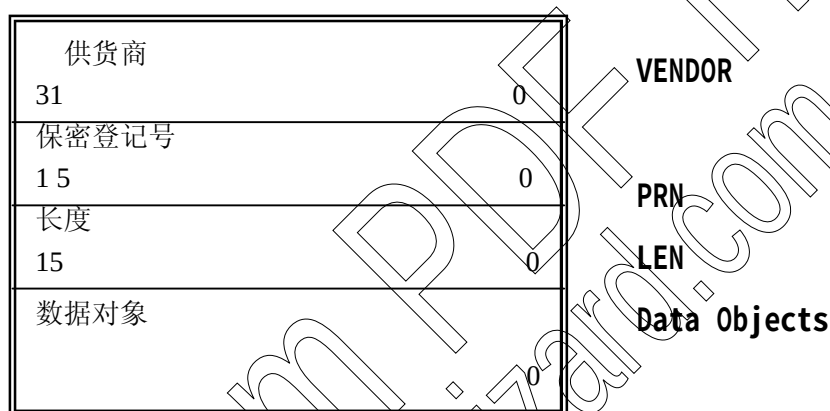
“**Vendor**”（供货商）段是一个供货商的四字节 **ASCII** 码名称。

“**Private Registration Number**”（**PRN**）（保密登记号）段是供货商所指派的一个对象识别符（**I、D**）。

“**Len**”段含有 **Data object** 段的长度（字节数）。

“**Data Objects**”（数据对象）段包含如 **PROD** 对象所描述的供货商之数据（规模与格式可变）。

对象编码：



**SQX { Vendor = UI31 [0..31] <0...2<sup>32</sup>-1>**

**PRN = UI16 [0..15] <0...2<sup>16</sup>-1>**

**LEN = UI16 [0..15] <0...2<sup>16</sup>-1> 字节**

**SQn = n 个基本 DNP 对象之序列**

**}**

## 11. 5 保密登记对象的说明项（**PROD**）

**Data Object 83 - Variation:02**

**Type: Static**

说明：

这类型的对象是为供货商“保密登记对象的说明”而保留的。这个对象和它的 **PRO** 对象是一一对应的，该对象含有一个固定的报头以提供透明的数据传输，和一个后随对象的唯一登记号。内容的说明完全由供货商自由处理。

“**private registration object description**”(**PROD**)对象对于 **PRD** 对象保持一对一的关系，并且可以用它在处理中的 **DNP** 对象内分析 **PRO**。对于 **PRO** 的每个相应的单元，**PROD** 就含有一个单元，每个单元依次含有一套 **DNP** 对象与变体的号码。

“Vendor”（供货商）段是一个四字节 ASCII 码的供货商名称。

“PRN”（计数）段是供货商所指派的一个对象识别符（I、D）。

“Count”（计数）段规定尾随此段的对象定义之数量。每个对象定义包含三段二数量，对象与变体。

“Quantity”（数量）段指定对象的数量，这个数量是由 PRO 对象内的对象与变体段所规定的。

对象编码：

|              |  |   |
|--------------|--|---|
| 供货商 Vendor   |  |   |
| 31           |  | 0 |
| 保密登记号 PRN    |  |   |
| 15           |  | 0 |
| 计数 Count     |  |   |
| 15           |  | 0 |
| 数量 Quantity  |  |   |
| 15           |  | 0 |
| 对象 Object    |  |   |
| 7            |  | 0 |
| 变体 Variation |  |   |
| 7            |  | 0 |
| 数量 Quantity  |  |   |
| 15           |  | 0 |
| 对象 Object    |  |   |
| 7            |  | 0 |
| 变体 Variation |  |   |
| 7            |  | 0 |
| 数量 Quantity  |  |   |
| 15           |  | 0 |
| 对象 Object    |  |   |
| 7            |  | 0 |
| 变体 Variation |  |   |
| 7            |  | 0 |

SQX { Vendor = UI31 [0...31] <0...2<sup>32</sup>-1>  
PRN = UI16 [0...15] <0...2<sup>16</sup>-1>  
LEN = UI16 [0...15] <0...2<sup>16</sup>-1> 字节  
SQn = n 个基本 DNP 对象 序列之定义  
{ Object = UI8 [0...7] <0...2<sup>8</sup>-1>

Variation = UI8 [0...7] <0...2<sup>8</sup>-1>  
 quantity = UI15 [0...15] <0...2<sup>16</sup>-1>  
 }

}

下面举例作更清楚的说明：

PROD:

| 空格<br>(blank) | B | B | A  |
|---------------|---|---|----|
|               |   | 0 | 16 |
|               |   | 0 | 3  |
|               |   |   | 2  |
|               |   |   | 1  |
|               |   |   | 2  |
|               |   |   | 2  |
|               |   |   | 21 |
|               |   |   | 2  |
|               |   |   | 5  |
|               |   |   | 30 |
|               |   |   | 1  |

在此例中：

- B, B 及 A 代表供货商名称。
- PRN 是供货商 ABB 的 #16 保密对象。
- Count 段规定尾随三个定义，每个定义包含 **quality**, **Object** 和 **Variation** 三者。

相应的 PRO 对象为：

| 空格 (blank) | B | B        | A             |
|------------|---|----------|---------------|
|            |   | 0        | 16            |
|            |   | 0        | 33            |
|            |   |          | Binary Input1 |
|            |   |          | Binary Input2 |
|            |   | Counter1 |               |
|            |   | Counter2 |               |
|            |   | Analog1  |               |
|            |   | Analog2  |               |
|            |   | Analog3  |               |
|            |   | Analog4  |               |
|            |   | Analog5  |               |

- Object=01**  
**Variation=02** 8 位二进制
- Object=21** 16 位冻结计数  
**Variation=2** (带 FLAG)
- Object=30** 32 位模拟输入  
**Variation=1** (带 FLAG)

## 第十二章： 应用程序对象的定义

---

本章采用第一章所建立的规则去定义应用程序数据对象。

### 12. 1 应用程序的识别符

#### Data Object 90- Variation 01

说明：

“**application identifier**”(应用程序识别符)是一种信息对象，用以表示在一个设备内 的一个应用程序或操作系统的过程。这个对象应用层的应用功能联用，以控制软件的应用。

该对象具有不定的格式并且就是用作一种位座 (**place holder**)。应该使用应用层的三格式限定码(**three format qualifier**)去识别所要用的应用程序，或者若应用程序是未知的，就必须用 **ALL** 限定码限定全部有关的应用程序。

### 第十三章： 另一种数码对象的定义

本章采用在第一章内建立的规则去定义另一种或定制的数据对象表。

#### 13. 1 短浮点

##### Data Object100- Variation:01

说明:

“short floating point”(短浮点)信息对象表现一个计算所得或测量所得的科学值。该对象的格式遵循 IEEE—754 关于浮点数据表现法的标准。“Value”(数值)段含有实际的浮点数并遵从 IEEE—754 标准所规定的 “short real”格式。

“flag”(标志)段保持关于点的信息和先前的对象具有同样的意义。

“unit”(单位)段决定 value 段的单位, 这是计算值或测量值的科学单位或工程单位。

对象编码:

|           |  |              |  |                   |  |  |  |
|-----------|--|--------------|--|-------------------|--|--|--|
| 单位(Units) |  |              |  | 标志 (Flag)         |  |  |  |
| 0         |  |              |  | 0                 |  |  |  |
| 值 (Value) |  |              |  |                   |  |  |  |
| 符号 (Sign) |  | 指数(Exponent) |  | 有效值 (Significant) |  |  |  |
| 0         |  | 7            |  | 0                 |  |  |  |

Flag = {  
On-line = BS1[0] <0,离线; 1, 在线>  
Restart = BS1[1] <0,正常;1,再起动力>  
Communication lost = BS1[2] <0,正常;1,失落>  
Remote forced data = BS1[3] <0,正常;1,远方强置数>  
Local forced data = BS1[4] <0, normal;1,forced>  
Roll-over = BS1[4] <0>  
Roll-over = BS1[5] <0,正常;1,越程>  
Reserved = BS1[6] <0, 正常; 1, 出错>  
Reserved = BS1[7] <0>  
}

叙记:

绝对值自 “value”段推导出来, 如下:

$$\text{Absolute\_Value} = 1.S_0S_1S_2S_3S_4\dots S_{22} \times 2^{(\text{Exp}-127)}$$

式中  $S_i$  = 有效值  $[i\dots i]$  与  $\text{Exp}$  = 指数 $[0\dots 7]$

**Sign**: 正数的符号为 0, 负数的符号为 1

**Exponent**: 应用于 (1, 有效值) 的 2 之幂。

**Significant**: 二进制的数字的有效值

“Unit” (单位) 段具有以下的定义值:

- |                           |                            |
|---------------------------|----------------------------|
| 0: Volt p-p               | (电压, 峰—峰值之伏)               |
| 1: Amperes p-p            | (电流的峰—峰值之安)                |
| 2: Volt RMS               | (电压有效值之伏)                  |
| 3: Amperes RMS            | (电流有效值之安)                  |
| 4: KW 或 Kilowatts         | (有效功之千瓦或阻性千伏安)             |
| 5: KVA 或 Kilo volt-ampere | (视在功率之伏安)                  |
| 6: KVAR 或 Kilovars        | (无功功率之千瓦, 或感性的伏安)          |
| 7: KWH                    | (千瓦小时)                     |
| 8: KWARH                  | (千乏小时)                     |
| 9: PF                     | (功率因数)                     |
| 10: Hz                    | (赫芝)                       |
| 11: W                     | (用弧度计的频率)                  |
| 12: C                     | (摄氏度数)                     |
| 13: F                     | (华氏度数)                     |
| 14: K                     | (K 氏度数, 即绝对温度)             |
| 15: N                     | (力之牛顿单位)                   |
| 16: Kg                    | (按公斤计的质量)                  |
| 17: m/s <sup>2</sup>      | (加速度, 米 / 秒 <sup>2</sup> ) |
| 18: N / m <sup>2</sup>    | (压力单位, 牛顿每平方米)             |
| 19: N * m                 | (力矩单位, 牛顿一米)               |

### 13.2 长浮点

## Data Object100- Variation:02

说明:

“**long floating point**”(长浮点)信息对象表现一个计算得到的或测量得到的科学值。这个对象的格式遵循浮点数表现方法的 **IEEE—754** 标准。

“**Value**”(数值)段含有实际的浮点数并遵从 **IEEE—754** 标准所规定的 “**long real**” 格式。

“**flag**”(标志)段含有关于点的信息并且具有和先前的对象相同的意义。

“**unit**”(单位)段决定 “**value**”段的单位, 这是计算值和测量值的科学的或工程的单位。

对象编码:

```
Flag = {  
  On-line   = BS1[0]  <0, 离线; 1, 在线>  
  Restart   = BS1[1]  <0, 正常; 1, 再启动>  
  Communication lost = BS1[2]  <0, 正常; 1, 通信失落>  
  Remote forced data = BS1[3]  <0, 正常; 1, 远方强置数>  
  Local forced data  = BS1[4]  <0>  
  Over—range = BS1[5] <0, 正常; 1, 越程>  
  Reserved check = BS1[6]  <0, 正常; 1, 参准出错>  
  Reserved   = BS1[7]  <0>  
}
```

叙记:

“绝对值”可自 “**value**”段推导出来, 如下:

$$\text{Absolute\_Value} = 1.S_0S_1S_2S_3S_4\dots S_{51} \times 2^{(\text{Exp}-1023)}$$

式中  $S_i$  = 有效值  $[i\dots i]$  和  $\text{Exp} = \text{Exponent}[0\dots 7]$

**Sign**: 正数的符号为 0, 负数的符号为 1

**Exponent**: 施加于 (1, 有效值) 的 2 之幕。

**Significant**: 二进制的数字的有效值

“**Unit**”(单位)段具有以下的定义值: (同短浮点)

### 13.3 扩充的浮点

**Data Object100- Variation:03**

说明:

“**extended floating point**”(扩充的浮点)信息对象表现一个计算得到的或测量得到的科学值。这个对象的格式遵循浮点数表现方法的 **IEEE—754** 标准。

“**Value**”(数值)段含有实际的浮点数并遵从 **IEEE—754** 标准所规定的 “**long real**” 格式。

“**flag**”(标志)段含有关于点的信息并且具有和先前的对象相同的意义。

“**unit**”(单位)段决定 “**value**”段的单位，这是计算值和测量值的科学的或工程的单位。

对象编码:

|           |  |              |  |                   |  |  |  |
|-----------|--|--------------|--|-------------------|--|--|--|
| 单位(Units) |  |              |  | 标志 (Flag)         |  |  |  |
| 0         |  |              |  | 7                 |  |  |  |
| 值 (Value) |  |              |  | 0                 |  |  |  |
| 符号 (Sign) |  | 指数(Exponent) |  | 有效值 (Significant) |  |  |  |
| 0         |  | 14           |  | 63                |  |  |  |
| 0         |  | 0            |  | 0                 |  |  |  |

**Flag** = {  
**On-line** = BS1[0] <0, 离线; 1, 在线>  
**Restart** = BS1[1] <0, 正常; 1, 再起动力>  
**Communication lost** = BS1[2] <0, 正常; 1, 通信失落>  
**Remote forced data** = BS1[3] <0, 正常; 1, 强置数>  
**Local forced data** = BS1[4] <0>  
**Over-range** = BS1[5] <0, 正常; 1, 越程>  
**Reserved check** = BS1[6] <0, 正常; 1, 出错>  
**Reserved** = BS1[7] <0>  
}

叙记:

“绝对值”可自 “**value**”段推导如下:

$$\text{Absolute\_Value} = 1.S_0 S_1 S_2 S_3 S_4 \dots S_{51} \times 2^{(\text{Exp}-1023)}$$

式中 **Si** = 有效值 [1..i]和 **Exp** = Exponent[0..7]

**Sign** :正数的符号为 0，负数的符号为 1

**Exponent:** 施加于（1，有效值）的 2 之幂。

**Significant:** 二进制的数字的有效值

“Unit”（单位）段具有以下的定义值：（同短浮点）

### 13. 4 小包装的二进制编码十进数（SPBCD）

**Data Object 101 - Variation: 01**

说明：

“small-packed binary coded decimal”（小包装的二进制编码的十进数）信息对象表现一个 BCD 数字序列。每个 BCD 数字能表现各种信息，从控制输出到模拟输入。

对象编码：

| 数字 4 (Digit4) | 数字 3(Digit3) | 数字 2(Digit2) | 数字 1 (Digit1) |
|---------------|--------------|--------------|---------------|
| 3 0           | 3 0          | 3 0          | 3 0           |

SPBCD = SQ4{

Digit1 = UI4 [0...3] <0...9>

Digit2 = UI4 [0...3] <0...9>

Digit3 = UI4 [0...3] <0...9>

Digit4 = UI4 [0...3] <0...9>

}

### 13.5 中包装的二进制编码十进数（MPBCD）

**Data Object 101 - Variation: 02**

说明：

“medium-packed binary coded decimal”（中包装的二进制编码的十进数）信息对象表现 BCD 数字一个序列。每个 BCD 数字能表现各种信息，从控制输出到模拟输入。

对象编码：

| Digit 4 | Digit 3 | Digit 2 | Digit 1 |
|---------|---------|---------|---------|
| 3 0     | 3 0     | 3 0     | 3 0     |
| Digit 8 | Digit 7 | Digit 6 | Digit 5 |
| 3 0     | 3 0     | 3 0     | 3 0     |

MPBCD = SQ8{

Digit 1 = UI4 [0...3] <0...9>

Digit 2 = UI4 [0...3] <0...9>

Digit 3 = UI4 [0...3] <0...9>  
 Digit 4 = UI4 [0...3] <0...9>  
 Digit 5 = UI4 [0...3] <0...9>  
 Digit 6 = UI4 [0...3] <0...9>  
 Digit 7 = UI4 [0...3] <0...9>  
 Digit 8 = UI4 [0...3] <0...9>  
 }

### 13.6 大包装的二进制编码十进数 (LPBCD)

Data Object101 - Variation:03

说明:

“Large-packed binary coded decimal” (LPBCD) 信息对象表现 BCD 数字一个序列。  
 每个 BCD 数字能表现各种信息，从控制输出到模拟输入。

对象编码:

|                 |                 |                 |                 |
|-----------------|-----------------|-----------------|-----------------|
| Digit 4<br>3 0  | Digit 3<br>3 0  | Digit 2<br>3 0  | Digit 1<br>3 0  |
| Digit 8<br>3 0  | Digit 7<br>3 0  | Digit 6<br>3 0  | Digit 5<br>3 0  |
| Digit 12<br>3 0 | Digit 11<br>3 0 | Digit 10<br>3 0 | Digit 9<br>3 0  |
| Digit 16<br>3 0 | Digit 15<br>3 0 | Digit 14<br>3 0 | Digit 13<br>3 0 |

LPBCD = SQ16{

Digit1 = UI4 [0...3] <0...9>  
 Digit2 = UI4 [0...3] <0...9>  
 Digit3 = UI4 [0...3] <0...9>  
 Digit4 = UI4 [0...3] <0...9>  
 Digit5 = UI4 [0...3] <0...9>  
 Digit6 = UI4 [0...3] <0...9>  
 Digit7 = UI4 [0...3] <0...9>  
 Digit8 = UI4 [0...3] <0...9>  
 Digit9 = UI4 [0...3] <0...9>

```
Digit10 = UI4 [0...3] <0...9>
Digit11 = UI4 [0...3] <0...9>
Digit12 = UI4 [0...3] <0...9>
Digit13 = UI4 [0...3] <0...9>
Digit14 = UI4 [0...3] <0...9>
Digit15 = UI4 [0...3] <0...9>
}
```

## 术 语 词 汇

---

### **application** (应用程序)

含有一个或多个过程和支持功能的一段软件 (一个程序)。

### **ASCII** 信息交换的美国标准码

它是一种 7 位码, 为国际通用的一套字母与数的字符提供了标准码。

**Binary** 二进制数。

**Bit** 毕特。

**Byte** 字节、IEC 定义它为八位字节 ( )。

### **Change of state** (变位)

由一个特定系统所定义的显著变化。通常用双位状态表示。缩写为 **COS**。

**COS** 参照 **change of state**

**CRC** 循环余校验

**Decimal** 十进数

**Host** 主站, 亦称 **mester** 或 **master station**。

**NUL** 零

**Port(口)** 连接通信信道或设备的接口。

**RTU** 远方终端

### **Sub-remote unit**

收集数据, 作某些处理并以其对远方的主机 (**MRU**) 通信的一种物理 设备 (例如外设板, **D20 RTU**, 非 **D20 RTU**, 表或其它智能化) 设备。

**Trip/close** 脱扣 / 合闸, 通常用两个继电器, 一个主继电器, 一个选择继电。

## 缩 写 的 清 单

---

|              |                                                |
|--------------|------------------------------------------------|
| <b>BCD</b>   | 二十进制码, <b>binary coded decimal</b> (二进制编码的十进数) |
| <b>BIN</b>   | 二进制码                                           |
| <b>BS</b>    | 二进制串                                           |
| <b>CTO</b>   | <b>common time of occurrence</b>               |
| <b>CTO</b>   | <b>common time object</b>                      |
| <b>DA</b>    | 分布式自动化                                         |
| <b>DAC</b>   | 数据采集与控制                                        |
| <b>DAC</b>   | 数 / 模转换器                                       |
| <b>DNP</b>   | 分布网 <b>EXEC</b> 执行                             |
| <b>F</b>     | 固定点                                            |
| <b>FIFO</b>  | 先进先出                                           |
| <b>I</b>     | 整数                                             |
| <b>ID</b>    | 识别符                                            |
| <b>IEC</b>   | 国际电工技术委员会                                      |
| <b>IEEE</b>  | 电与电子工程师学会 (美)                                  |
| <b>LEN</b>   | 长度                                             |
| <b>LPBCD</b> | 大包装 <b>BCD</b> 码                               |
| <b>MPBCD</b> | 中包装 <b>BCD</b> 码                               |
| <b>OS</b>    | 字符串                                            |
| <b>PCB</b>   | 模式控制块                                          |
| <b>PRN</b>   | 保密登记数                                          |
| <b>PRO</b>   | 保密登记对象                                         |
| <b>PROD</b>  | 保密登记对象说明项                                      |
| <b>PWD</b>   | (回送) 现在的工作索引                                   |
| <b>R</b>     | 真; 实( <b>red</b> )                             |
| <b>RESP</b>  | 响应                                             |
| <b>RMS</b>   | 均方根                                            |

**SCADA**     监视控制与数据采集

**SPBCD**    小包装 **BCD** 码

**UF**        无符号固定点

**UI**        无符号整数

DocuCom PDF Trial  
www.pdfwizard.com